# Physics Simulator

Name: Dominic Easton

Candidate Number: 3056

Centre Name: Wren Academy

Centre Number: 12258

OCR H446-03

# Table of Contents

1. Analysis ..... 7
Problem Identification ..... 7
Stakeholders ..... 8
Interview with Dante ..... 8
Survey ..... 10
1. What proportion of your time revising Physics is spent on Paper 3 (Required Practical) content? ..... 10
2. Which required practical activities would you most like to see in a physics simulator? ..... 10
3. Rank the required practical activities by how useful it would be to have them in the simulator ..... 11
4. In the next 8 questions, I asked how far the user would agree with certain statements to gauge how important certain features would be to have in the simulator. ..... 11
5. Are there any non-required Practical Activities that you would like to see in a physics simulator? ..... 15
6. What are the essential features of a Physics simulator? ..... 15
7. How formal should my Physics simulator be? (1- very informal 10- very formal) ..... 16
Research into similar software – PhET Interactive Simulations Approach ..... 17
Wave Interference (University of Colorado, 2024) ..... 18
Circuit Construction ..... 21
Interview with Ian about PhET ..... 23
Research into similar software – SimPhy (SimPhy Micrometer Simulator, n.d.) ..... 24
Summary ..... 25
Research into similar software – GeoGebra (Gilbert Bett, n.d.) ..... 26
Features of the proposed solution ..... 27
Further Limitations: ..... 30
Features that make it solvable by computational methods ..... 31
Abstraction ..... 31
Thinking Ahead ..... 31
Thinking Procedurally ..... 31
Thinking Logically ..... 31
Decomposition ..... 32
Concurrency ..... 32
Hardware and software Requirements ..... 33
Additional Requirements ..... 33
Success Criteria ..... 34

2. Design ..... 37
Systems Diagram ..... 38
Priority Order ..... 40
Explanation of each module ..... 41
Storing data ..... 41
Simulators ..... 41
Menu ..... 44
Choosing Development Language ..... 45
Results Design ..... 46
Algorithm Design ..... 46
Key Variables and Data Structures ..... 47
Validation ..... 47
In Development Test data ..... 47
Simulators – Required Practical 1 – Stationary Waves visuals – Moving Parts – Abstract and Realistic View ..... 48
Algorithm Design ..... 49
Key Variables and Data Structures ..... 55
In-Development Test Data ..... 56
Control Panel – Sliders ..... 58
Class Diagrams ..... 58
Attributes and Methods Used ..... 59
Validation required ..... 60
Algorithm Design ..... 61
In-Development Test Data ..... 64
Second Iteration – Algorithm Design ..... 65
Discrete Slider – Class Diagram ..... 66
Discrete Slider – Algorithm Design ..... 68
Discrete Slider – Validation required ..... 69
Discrete Slider - Test Data ..... 69
Equipment – Mass Hook and Masses ..... 70
Class Diagram ..... 70
Attributes and Methods Used ..... 71
Validation Required ..... 72
Algorithm Design ..... 73
In-development Test Data ..... 77
Equipment box ..... 78

Class Diagram ..... 79
Attributes and Methods Used ..... 80
Algorithm Design ..... 81
Validation Required ..... 82
In Development Test Data ..... 82
Main Menu ..... 83
Algorithm Design ..... 83
Class Diagram ..... 86
Attributes and Methods Used ..... 86
In-Development Test Data ..... 87
Home Icon ..... 88
Class Diagrams ..... 88
Attributes and Methods Used ..... 88
Algorithm Design ..... 88
In-Development Test Data ..... 89
Second Prototype ..... 90
Algorithm Design ..... 90
Key Variables and Data Structures ..... 95
In-Development Test Data ..... 96
Usability features ..... 97
RP 01 Visuals ..... 97
Sliders ..... 99
Mass Hook ..... 100
Main Menu ..... 101
Home Icon ..... 103
Font ..... 103
Stakeholder Feedback ..... 104
Post-Development Test Data ..... 105
3. Development ..... 107
Libraries Used ..... 107
Naming Conventions ..... 107
Stationary Waves Results ..... 107
Calculate Frequency ..... 107
Calculate Speed ..... 107
Testing ..... 108

Review ..... 108
Stationary Waves Visuals: ..... 108
Closest Harmonic ..... 108
dispReal ..... 109
dispAbstract ..... 115
Control Panel - RP 01 Sliders ..... 120
Input Box ..... 124
Sliders – 2nd Iteration ..... 130
Discrete Slider ..... 133
Equipment – RP 01 ..... 136
Mass Hook ..... 136
Masses ..... 145
Equipment Box ..... 149
First Prototype ..... 154
Main Menu ..... 158
Home Icon ..... 160
Second Prototype ..... 162
Menu ..... 162
RP 01 – Realistic View ..... 166
RP 01 Abstracted View ..... 168
4. Evaluation ..... 174
Post Development testing ..... 174
Ian run through ..... 174
Dante Run Through ..... 174
Mr Moran Run through ..... 174
Functionality Testing ..... 174
Robustness Testing ..... 177
Usability Testing /Usability Features ..... 179
Menu – Test 19 ..... 180
Test 20 ..... 181
Test 21 ..... 181
Test 22 ..... 181
Test 23 ..... 182
Test 24 ..... 184
Limitations of the Solution ..... 185

Functionality Limitations....185
Usability Limitations....186
Maintenance....187
Corrective Maintenance....187
Adaptive maintenance....187
Perfective Maintenance....187
5. Bibliography....188
6. Appendices....190
Appendix A - PyGame Methods....190
Appendix B – Glossary....190
Appendix C - Code Listing....191
Waves.py....191
Sliders.py....193
Masses.py....198
Menu.py....203
RP 01.py....207

# 1. Analysis

## Problem Identification

As an AQA A-Level Physics (AQA, n.d.) student, I have had to undertake examinations on the required practical activities (RPs) performed in class. Students are tested on their knowledge of these required practical activities in A-Level Physics Paper 3. This paper includes questions on the methods involved in performing the practical activities, the errors that can arise from different methods and the safety precautions that must be upheld.

To revise for this part of the subject, students usually review their notes taken while performing the practical activities, consult notes found online, such as those found on Physics and Maths Tutor (Physics & Maths Tutor, 2024) or by watching videos of others performing the practical activities. The problem with revising from notes taken during the practical activity is that the exams usually occur long after the activities are performed meaning that students may struggle to recall specifics and try to remember what they had done during the required practical in the exam.

The problem with revising from notes online is that different methods are often used, which the student is not familiar with, so the student may not be able to fully visualise what is going on in the practical it is describing. Watching videos of others does allow the student to visualise what is going on in the practical but does not allow them to explore changing different variables for themselves which is key to understanding how one practical activity works. Additionally, practical activities are occasionally rushed in class due to time constraints. This results in students not being able to fully explore all variables and observe what errors arise. Therefore, students have to rely solely on notes for their revision which are more easily forgotten than recalling the experience of performing the practical.

An ideal solution to this problem would be to allow students to re-perform the practical activities closer to the exam period help students grasp the nuances of the practical activities. However, this is often not possible because each practical requires specialist equipment and expertise that is not always available due to it being used by other year groups. Moreover, students would have to come into a Physics Lab since practical activities cannot be performed at home. This could be inconvenient to a student who is revising.

If a computational approach was applied to this situation, a practical simulator could be developed which would allow students to experience setting up equipment, exploring the change of different variables and performing a required practical activity without the need for specialist equipment. The simulator would enable the user to use different methods so online notes could be understood better by the student as they would be able to fully visualise the method. It would also still be able to show systematic and random errors that can arise due to the setup of the practical activity. Therefore, I have chosen to create a Physics Practical simulator for my Computer Science NEA.

## Stakeholders

My target audience for this software will be current A-Level Physics students looking for help in revising A -Level Physics. This is because my software would allow these students to revise more effectively by being able to virtually conduct Physics practical activities. My software will be appropriate to their needs as these students would be looking to find a way to re-perform the practical activities to aid their revision. More specifically, my main stakeholder will be Dante Andersen, who I will have contact with throughout the creation of this software. Dante is currently a Year 13 student studying A-level Physics and said he struggled on Physics Paper 3 in his mocks. I will also use Ian Lam as another stakeholder to give me a second insight as another Year 13 student. Dante has said that he often struggles to recall performing the practical activities and re-learn what he did by watching videos which do not give him a full understanding of the practical activities. The solution I am creating would be appropriate as it would allow him to explore the practical actvities for himself.

I will also use Mr Moran, an A-level Physics teacher as my third stakeholder to give me a perspective of someone who has been teaching Physics for years and knows what would be beneficial to students. Mr Moran currently recommends PhET Simulations which has interactive simulations similar to the Required Practical Activities we learn in the AQA A-Level Physics course but are not course specific and do not always show the setup of the practical activity. Mr Moran said that it would be useful to have a software to recommend to students revising that is course specific and shows the setup, meaning that my solution is suitable for him.

The following is a table of my stakeholders, their role, availability and their interactions with the software.

| Stakeholder | Role | Interaction with solution | Availability |
|---|---|---|---|
| **Ian Lam** | Year 13 Physics Student | Performing the practical activities | Twice a week |
| **Dante Andersen** | Year 13 Physics student | Performing the practical activities | Twice a week |
| **Mr Moran** | Year 13 Physics Teacher | Setting the students assignments | Once a month |

## Interview with Dante

This is the transcript from an interview with Dante to gain a deeper understanding of the problem and to gain an idea of the basic features of the desired solution.

1. How do you usually revise Paper 3 Physics content?
   I watch videos on YouTube of others performing the required practical activities, make notes and then do practice questions.
2. If you could, would you re-perform the required practical activities closer to your exams?
   Yes. I think this would be ideal.
3. Why?
   It would help me remember the method better as I'm actually doing it.
4. Do you think having an app that could replicate this experience would be beneficial?
   Yes. I would use something like this if I had the option.
5. Do you think this app could be used as a revision tool as well as just simulating practical activities?

Yes. It would be useful if it let you setup the practical activities yourself and checked your answers.

6. Would you want to do this by dragging and dropping virtual equipment into its correct place?
   Yes, sounds good.
7. Would you like the teachers to set you assignments on a required practical?
   Not really. I don't really see the point.
8. Would you like to record results from a required practical in a table and then have the software draw a graph from these results.
   Yes, I would also like to be able to draw the graph myself and compare answers.
9. Would you like to have your own account so you can save your results and graphs from performing a required practical?
   Yes, that sounds good.

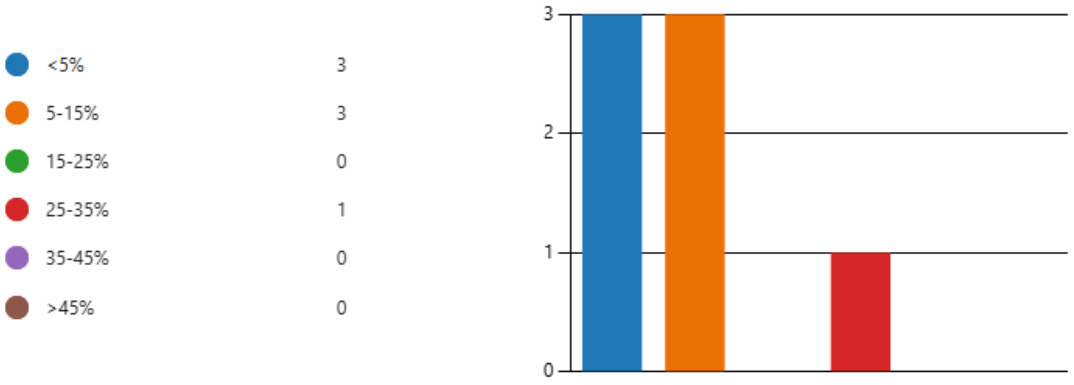
This interview has shown that a Physics simulator is necessary in order to solve the problem of not being able to re-perform a required practical activity in class. It has also shown that being able to setup the equipment is a key feature of the solution and having teachers set students assignments is low priority.

## Survey

I created a survey and I sent it to students currently studying A-Level Physics to get a wider range of insights into my solution and to get a general consensus on certain features.

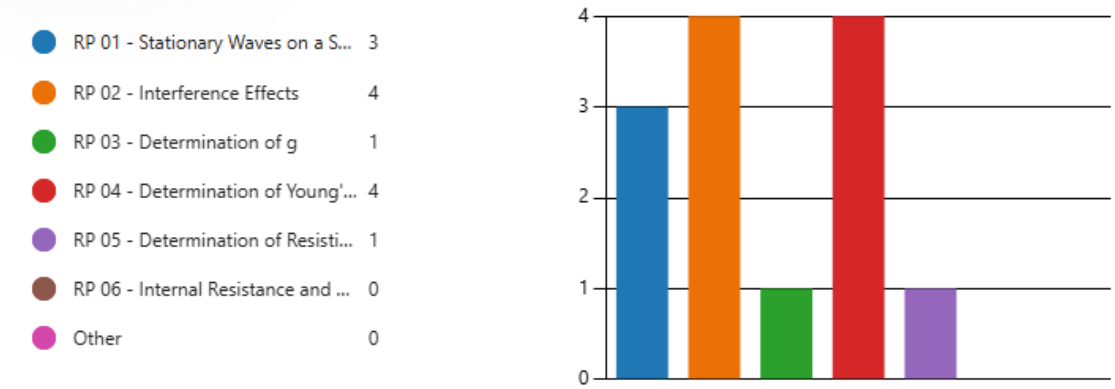
RPs (Required Practical activities) are referred to a lot in this section. To see what each RP is in A-level Physics, see this page from Physics and Maths Tutor:
https://www.physicsandmathstutor.com/physics-revision/a-level-aqa/practical-skills/

### 1. What proportion of your time revising Physics is spent on Paper 3 (Required Practical) content?




15% of the marks in the Physics papers relate to practical work. The estimated mean from this graph shows that around 10% of revision time is spent on Paper 3. Ian Lam said this was because he did not have an effective way of revising, so spent more time on other topics in Physics that he felt that he was making more progress on in an hour's worth of revision. This means that my solution should be engaging and have an effective revision element so students spend more time on required practical content.

### 2. Which required practical activities would you most like to see in a physics simulator?




Respondents were allowed to select more than one option in this question. The purpose of this question was to see which practical activities have a lower priority as they would be the ones not selected

As you can see, RP 06 was not selected. This is probably because there is a PhET simulation that allows you to replicate the practical activity .Given a strict development timeframe, it would not be an effective use of time to focus on RP 06 more than other practical activities.

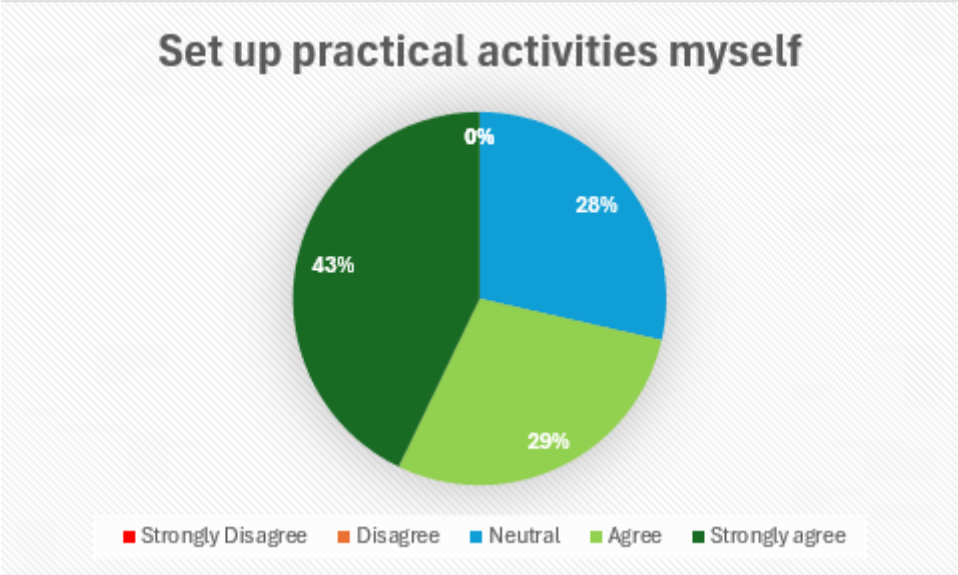
3. Rank the required practical activities by how useful it would be to have them in the simulator




This question allowed me to see the order of priority for the recreation of required practical activities as RPs with a higher ranking mean that more people would find it useful in a simulator so it should have more time allocated to it in the development stage.
RP 02, RP 04 and RP 01 are the highest ranked RPs to be implemented into a simulator. I will make sure that these three are given a high priority in development as they are the most important RPs for my stakeholders.

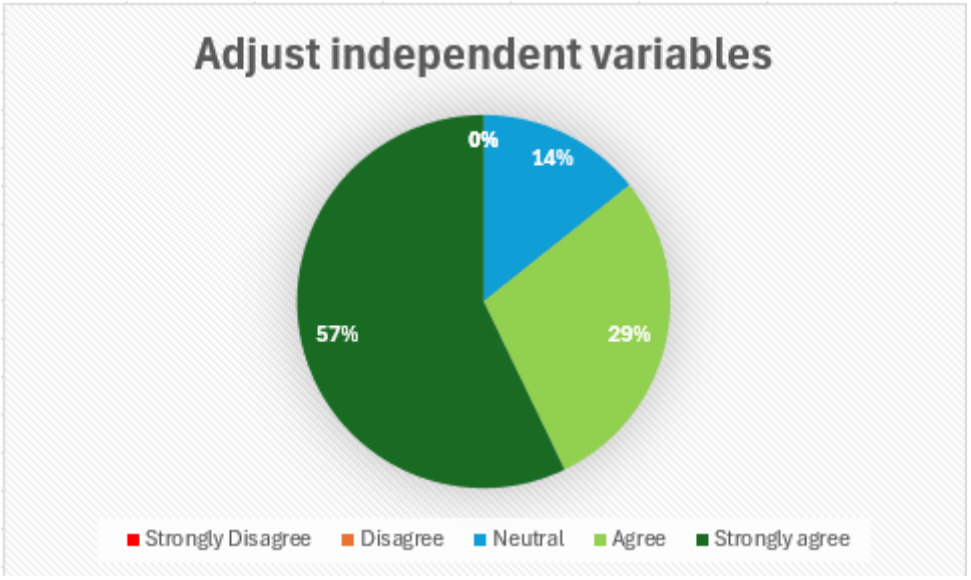
4. In the next 8 questions, I asked how far the user would agree with certain statements to gauge how important certain features would be to have in the simulator.

a) *I would like to set up RPs using virtual equipment myself.*



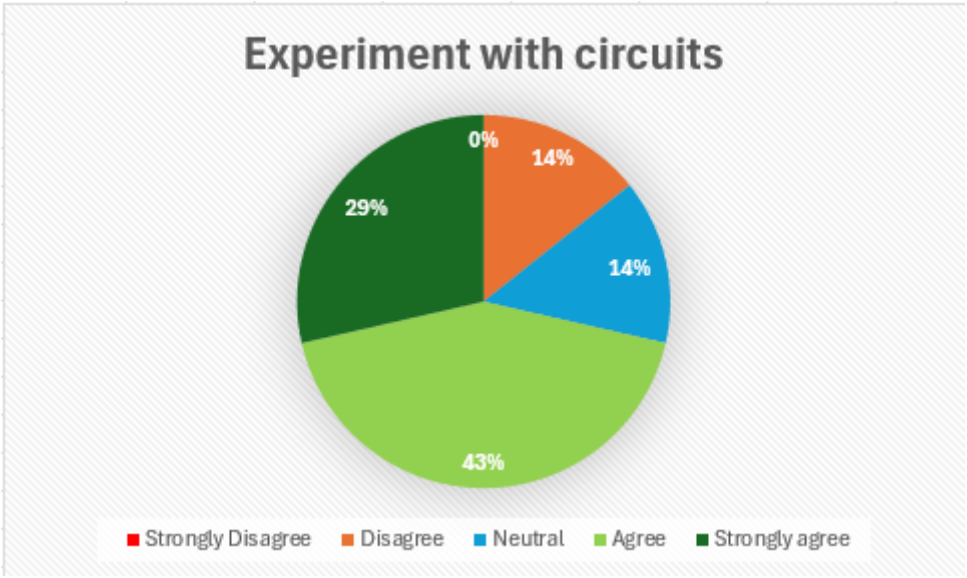

This pie chart shows that giving the user the ability to set up RPs themselves is a highly desirable feature. This is expected because this feature would allow students to have a lab-like experience of setting up the practical, which would allow them to effectively revise the methods of the RPs which they could be tested upon in their exams.

b) *I would like to adjust independent variable and be shown their effects.*



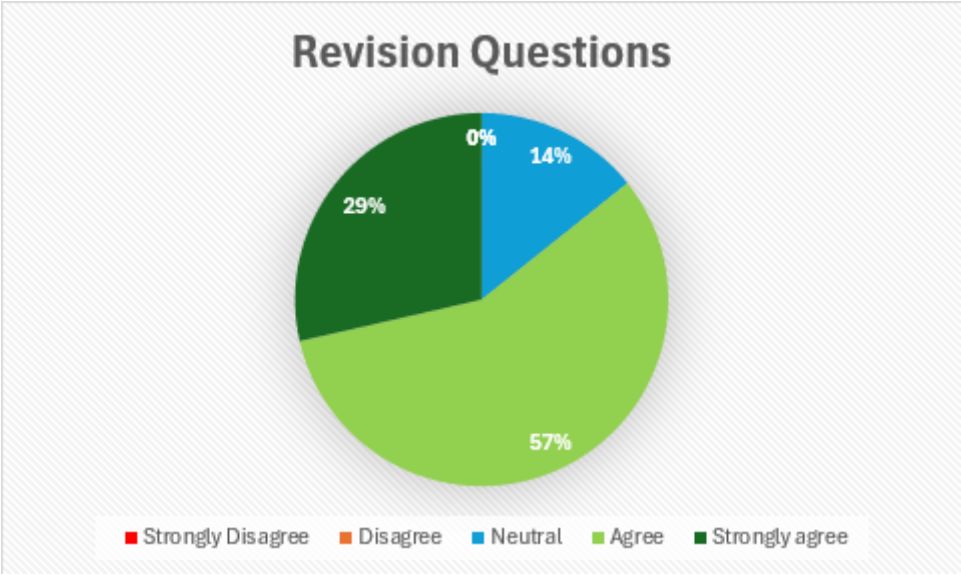

This shows that adjusting variables is an important feature of my Physics simulator and is highly desired. This is expected because it allows the user to gain an intuitive understanding of the effects of independent variables, a key part of a required practical activity.

c) *I would like to experiment with electrical circuits myself (not follow a particular RP)*



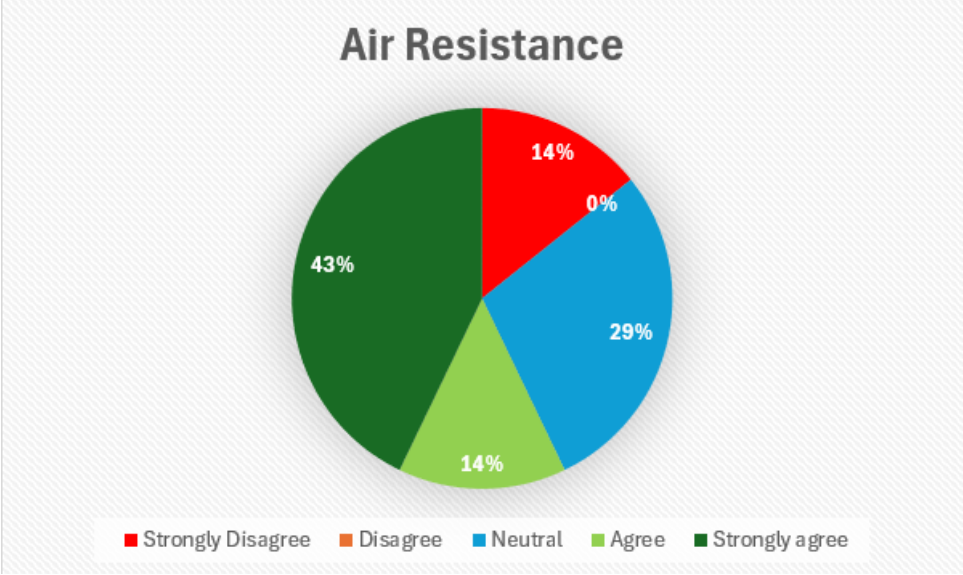

Most agree that a virtual circuit lab similar to the one in PhET simulations described in Circuit Construction would be useful in a physics simulator. However, since not as many people strongly agreed with this feature as they did with others and an almost complete solution already exists for this particular feature, it will have a lower priority than other features.

d) *I would like revision questions as part of the simulator.*



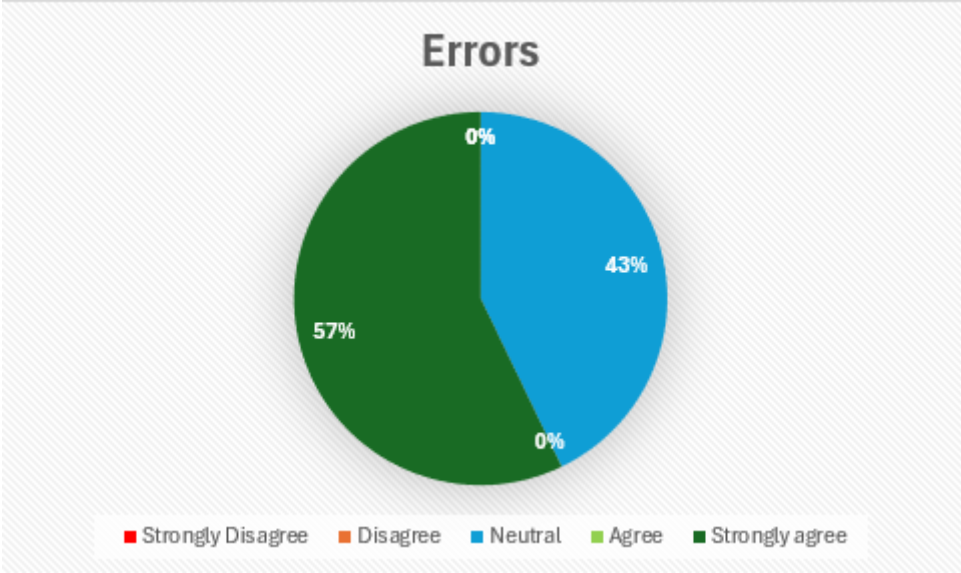

Again, most respondents agreed that this feature would be useful as it means that the software can help students with preparation for their examinations. This would also be a low priority feature as not many strongly agreed and revision applications already exist.

e) *I would like to have the option to add the effects of air resistance for RP 03.*



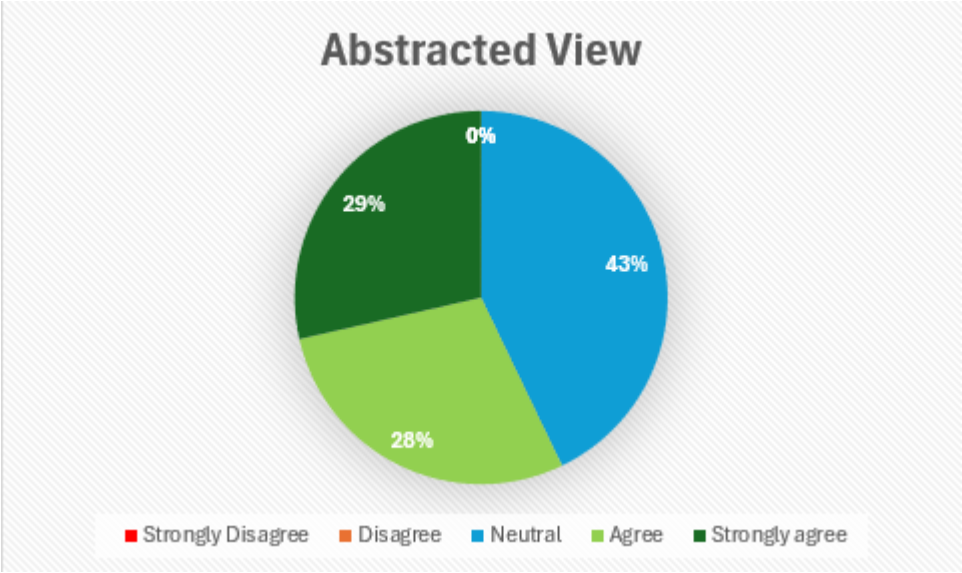

This feature had mixed results from the respondents, so having the option to turn air resistance on or off would suit all parties. This feature would be low priority due to the mixed results.

f) *I would like the option to add the effects of errors that may come up when performing the RP in a lab.*



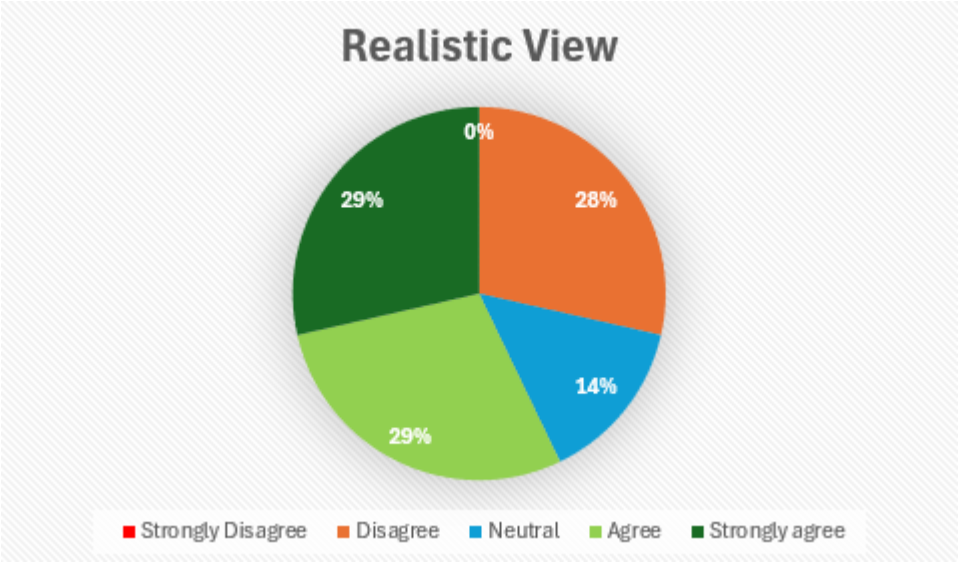

This is a highly desired feature so will set to a high priority. This feature would not only give the user a lab-like experience but would also benefit their revision for their exams as there have been many Physics Paper 3 questions on errors in a practical activity.

g) *I would like the software to give me an abstracted view to give me conceptual understanding (E.g. showing wave fronts for RP 02).*




This feature is also desired by the majority of the respondents so will be included in the final solution. This feature is used in PhET simulations described here - Research into similar software – PhET Interactive Simulations so this will be researched into to give me an idea of how to implement an abstracted view.

h) *I would like a realistic view of the RPs to fully simulate the lab experience.*




There are mixed results from respondents to add a feature to have a realistic looking view. As most agreed with this feature and it would help students remember better what they did in the lab, it will be included in my final solution. An option to the user will be given, so they can switch between abstracted views and realistic views.

5. Are there any non-required Practical Activities that you would like to see in a physics simulator?

- Floating object, simulating upthrust in water *2
- Stern Gerlach experiment
- Strong force particle simulator

This question allowed me to see if students would like the simulator to cover practical activities not tested on in A-level Physics so I can add them if I have the time. Multiple people requested a floating object simulator, so will be included if there is time. These additional practical activities will be a low priority as the students are not getting tested directly on them.

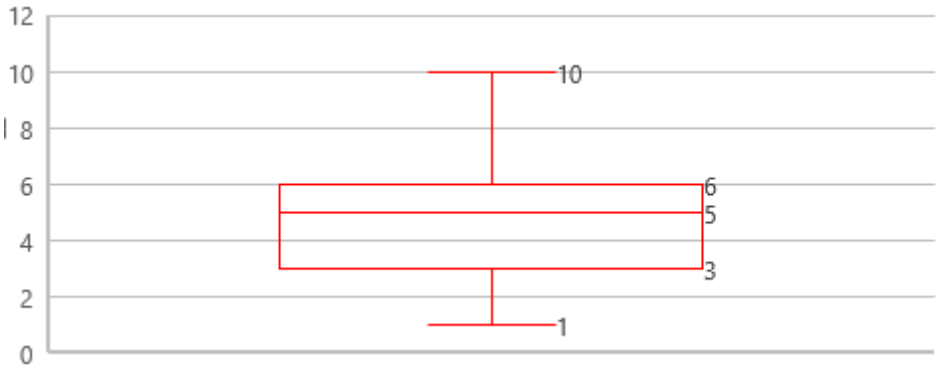
6. What are the essential features of a Physics simulator?

- Similar to real life results
  This has already been established to be a key feature of the simulator as it would allow the user to re-perform the RP just like how they would in real life and experience the same errors.
- 2x speed
- Ability to move back and forward in a practical
- Ability to put 2 snapshots of a practical side by side
  These three features would allow the user to compare the results from different values of independent variables and how the practical changes over time.
- Ability to remove errors
  This feature will be added so users can get both a lab-like experience and a perfect scenario to give them more of a conceptual understanding. This will be added in the form of a slider so the user can adjust the effect of the errors.

- Simple Interface
  The solution will be intuitive and simple to use.
- Ability to change variables and see the result in real time
  This is a key feature that will give the user a good understanding of the effects of variables and a true lab-like experience.

7. How formal should my Physics simulator be? (1- very informal 10- very formal)

The values entered for this question are: 1,3,5,5,6,6,10
The mean is therefore 5.14 and the standard deviation is 2.6.
Here is a boxplot for this data




As you can see, both the median and the mean are around 5 so a standard, not too formal, not too fun and informal interface is needed. However, the standard deviation and the range of the results are large so if time allows it, different themes could be chosen by the user so all users are happy.

## Research into similar software – PhET Interactive Simulations Approach

"The PhET [Physics Education Technology] Interactive Simulations project at the University of Colorado creates free interactive math and science simulations". (University of Colorado, 2024) This includes physics simulations that are similar to the required practical activities in the AQA A-level specification. It does not, however, include all of the RPs listed in the A-Level Specification. For example, it does not have a simulator similar to RP 04 – Determination of Young's Modulus. I will look at the simulations it does have and note the features I like and the features I do not. PhET's interactive simulations are more for conceptual understanding than recreation of a lab practical but still share similarities with the A-Level RPs.

PhET simulations use an agile methodology where the team at PhET go through iterations of creating an initial design, showing it to students and teachers, getting feedback and then adjusting the design. This focus on user satisfaction is a key part of the agile methodology, which will also be key to my approach. The agile methodology used is highlighted by the diagram presented by PhET in *Figure 0*. I will also use an agile methodology because it will allow me to collaborate with my stakeholders and respond to changing in user requirements.

PhET also have a constantly changing document called Look and Feel (PhET Colorado, 2006). This outlines the underlying features that every simulation should have. This is what I would look to create in Features of the proposed solution and like PhET, repetitively review these features with my stakeholders. This will allow me to tailor the Physics simulations to my stakeholders'.

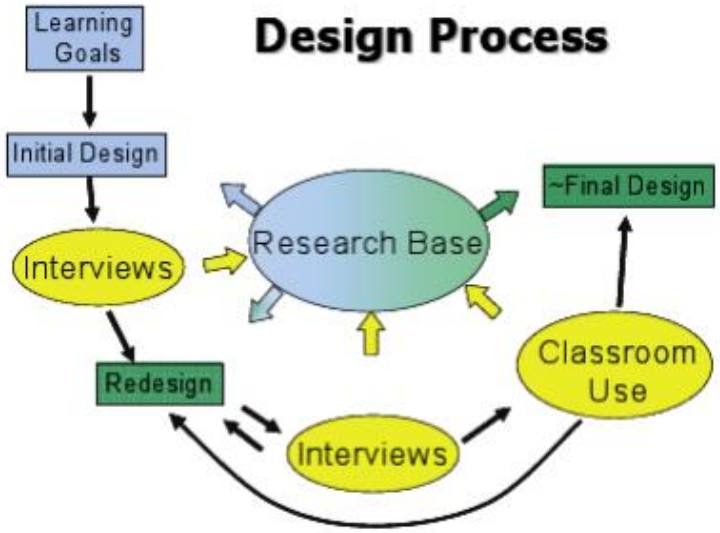



*Figure 0 (PhET Colorado, n.d.)*

## Wave Interference (University of Colorado, 2024)

Firstly I will look at a simulation close to the double slit part of Required Practical activity 02 – Interference Effects[1] ( Wave Interference, colorado.edu). (PhET, n.d.)

*Figure 1.1*

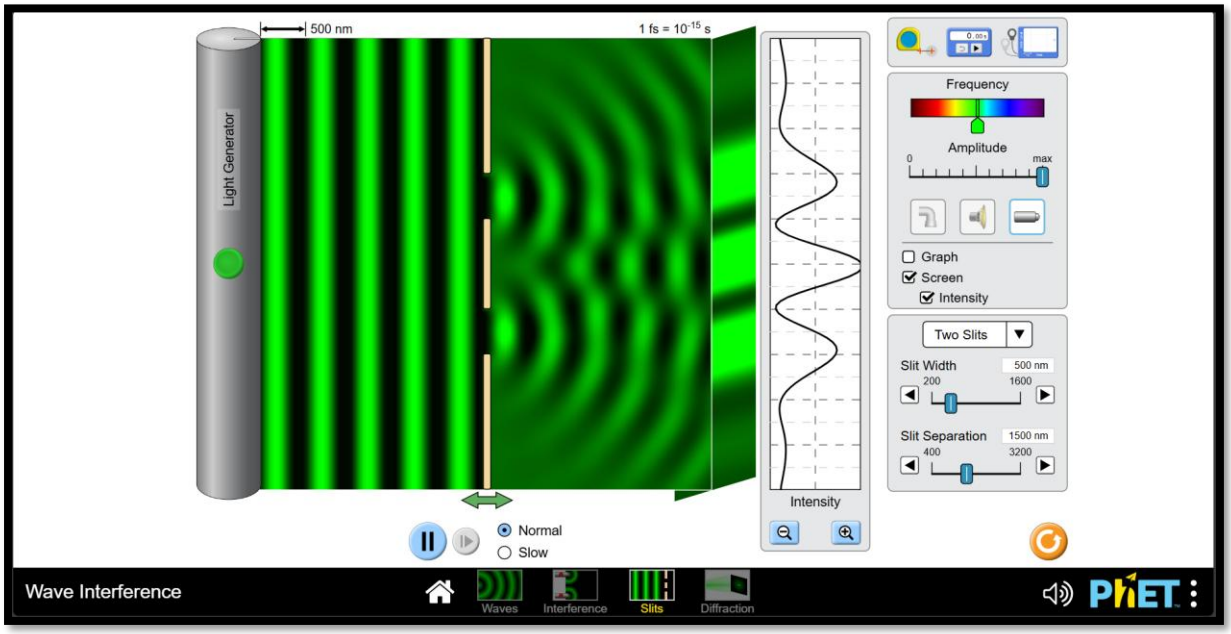



*Figure 1.1* shows a still of the interactive simulator for Young's Double slit experiment. Note that when using the software, the waves appear to move from left to right.

In the main display of the experiment, the wavefronts of each wave are shown passing through the double slit, diffracting, interfering and producing bars on the screen on the right hand side.

In real life, you will not be able to see these wave fronts but only be able to see the fringes on the right hand side. Despite this, this is still a useful feature because it allows the student to understand how the physics behind Young's Double slit experiment works.

[1] A description of the method of RP2 can be found here - RP 02 - Interference Effects.pdf (physicsandmathstutor.com)
A video explaining Youngs Double Slit Experiment - Young's double slit introduction (video) | Khan Academy

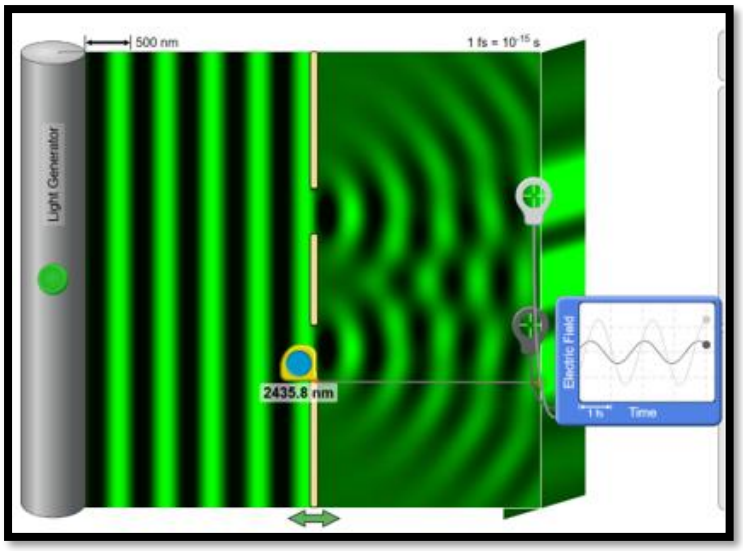



*Figure 1.2*

At the top right of the screen, the user can choose from measuring instruments that allow the user to measure distances between points so that they can measure the distance between fringes to calculate the wavelength of the light. This is shown in *Figure 1.2*. This is an important feature in my solution as there will be a heavy emphasis on replicating the lab experience. What this solution is missing is a way to plot these results on a graph.

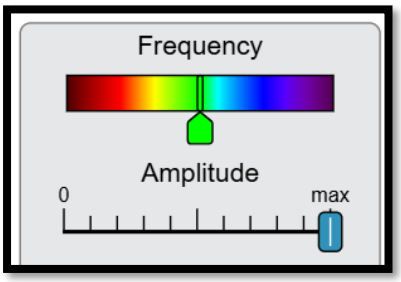



*Figure 1.3*

*Figure 1.3* shows the sliders located on the right hand side of the display that allow the user to adjust the frequency and amplitude of the wave. This is a good feature as it allows the user to experiment with different frequencies of light and their effect on the practical activity which you would not always get the chance to do in a science lab.

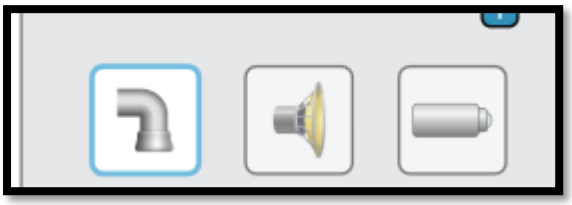


*Figure 1.4*

PhET also allows the user to represent the waves in different ways, including water, sound and light. This enables the user to visualise the experiment in various ways.

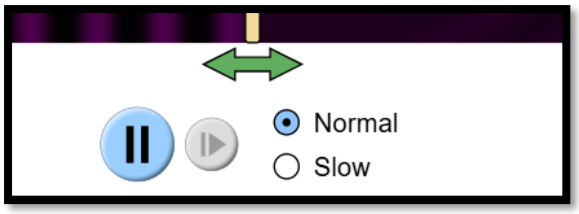



*Figure 1.5*

*Figure 1.5* shows a pause icon that allows the user to pause the practical activity in the middle of the simulation. This allows the user to get a freeze frame of the practical activity so that they can take measurements and gain an understanding by stopping the activity to see what is happening in that moment. This is something you cannot do in real life and the simulator will provide.

*Figure 1.5* also shows the slow option and the arrow which allows you to explore the activity frame by frame. This would allow the user to gain a conceptual understanding as they can see the physics taking place in slow-motion.

*Summary:*

Features I will include (Red – High priority, Amber – Medium Priority, Green – Low priority ;priorities were chosen by Dante Andersen):

- Representation of wave fronts to give the student a conceptual understanding in an abstracted view that a student would not be able to see in real life.
- Measuring instruments including a tape measure to give the student a lab-like experience.
- Different media to explore waves through (water, sound and light)
- Sliders to adjust frequency and amplitude to allow the user to explore effects of changing variables.

## Circuit Construction

I will now look at the Circuit Construction Interactive Simulation provided by PhET (Circuit Construction Kit: DC, colorado.edu). (PhET, n.d.)This can be used to replicate Required Preactical Activity 6 (RP 06) - Internal Resistance and emf[2] and has features that can be used in RP 05 - Determination of Resistivity of a Wire.[3]

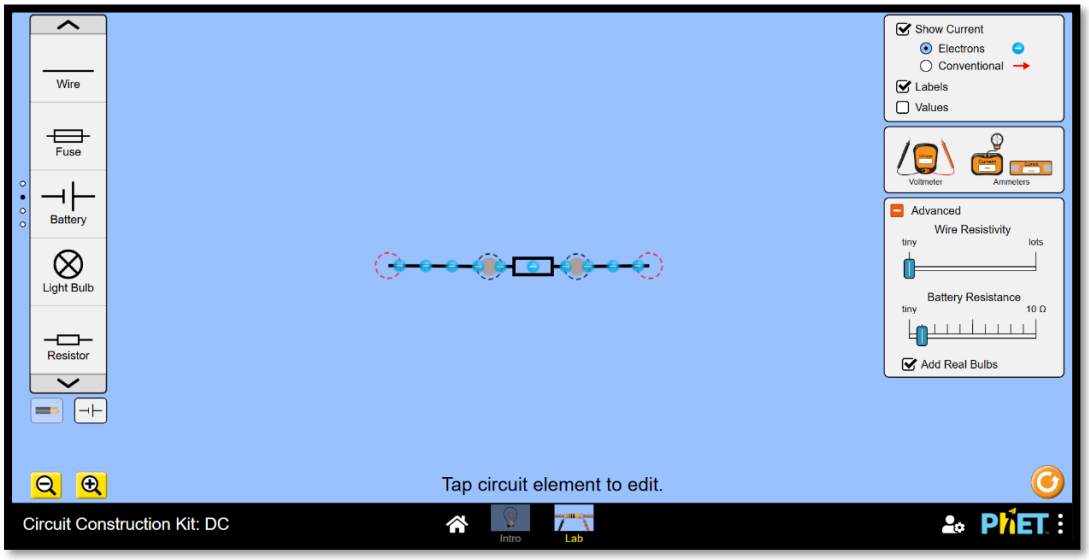



*Figure 2.1*

As seen in *Figure 2.1*, circuits of the user's choice can be created by dragging out components from the box on the left-hand-side. This is a good feature to be included in my solution because it allows the student to set up the Required Practical Activities for themselves, giving them an understanding of the setup of equipment.

It looks like this simulator was developed using an object oriented approach with each component having its own class so that multiple components can be instantiated quickly. Therefore, I will also use an object orientated approach as this is appropriate.

[2] RP 06 - Internal Resistance and emf – PMT description of the Required Practical Activity

[3] RP 05 - Determination of Resistivity of a Wire – PMT description of the Required Practical Activity

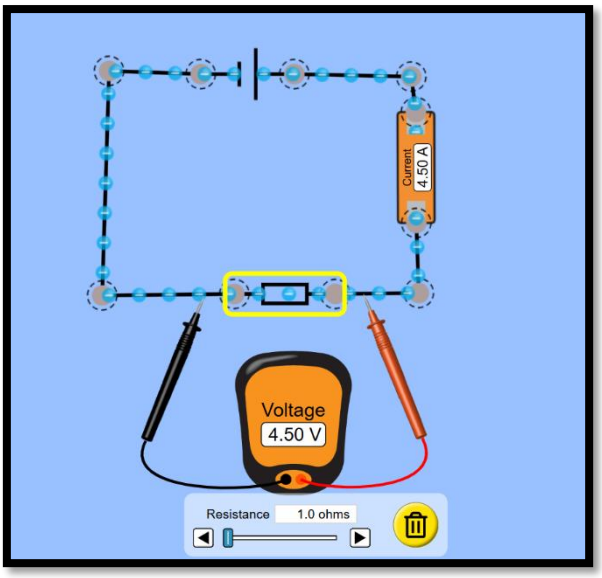



Figure 2.2

Figure 2.2 shows the setup of RP 06, which can be achieved through this PhET simulation. It shows how voltmeters and ammeters can be used in the simulation to investigate different components like you would be able to in a lab. PhET also shows the current through many electrons flowing through the circuit, allowing the user to conceptually understand how the physics behind the circuits works. The user can also choose to see conventional current rather than electrons to view the difference in direction of flow as seen in Figure 2.3.

I have noticed that PhET does not show the effects of the wire and battery heating up. This is one of the causes of error when performing RP 06 in the lab as this can increase the resistance of components. To give students a full replication of a lab experience, I will include an option to add errors like the heating of components.

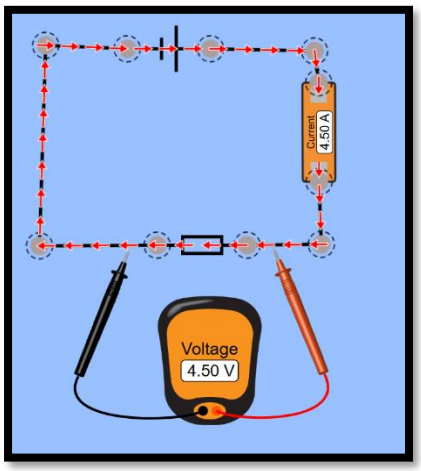



Figure 2.3

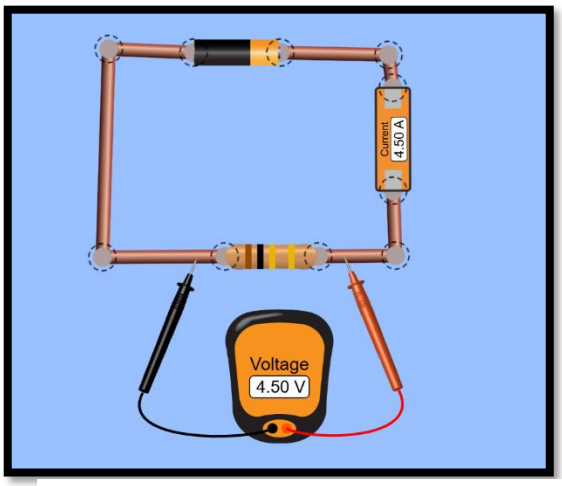



Figure 2.4

PhET allows the user to choose between viewing the circuit as circuit symbols as shown in Figure 2.2 and 2.3 or as images depicting the physical components of the circuit as shown in Figure 2.4. This allows the user to view what components look like in real life, giving them a lab-like experience.

*Summary*

Features to include (Red – High priority, Amber – Medium priority, Green – Low priority):

- Representation of different current conventions (arrows and electrons).
- Drag and drop of circuit components to allow the user to build their own circuit.
- Sliders to adjust resistance and potential difference of components.
- Ability to view the circuit as circuit symbols or as representations of the physical component.

## Interview with Ian about PhET

The following is the transcript of an interview with Ian about other solutions available

1. Do you use PhET simulator to help revise for Physics Paper 3?
   For me not really because sometimes it is confusing.
2. How is it confusing?
   You cannot see the effect of changing a variable as it only shows the data for one value at a time. If 2 simulations for different values could be placed side by side for me to compare it, it would be less confusing.
3. What features do you like from PhET?
   It gives me a good conceptual understanding as it gives a view of the physics behind a practical.

This shows that I should add a comparison feature that allows the user to compare two freeze frames as this would allow the users to gain more out of my solution than what PhET currently provides.

## Research into similar software – SimPhy (SimPhy Micrometer Simulator, n.d.)

SimPhy is a Physics simulation software that has many Physics simulations similar to required practical activities but most must be paid for so I will look into the free simulations they provide.

*Figure 3.1 – SimPhy's Micrometer Simulator ( Simphy Software)*

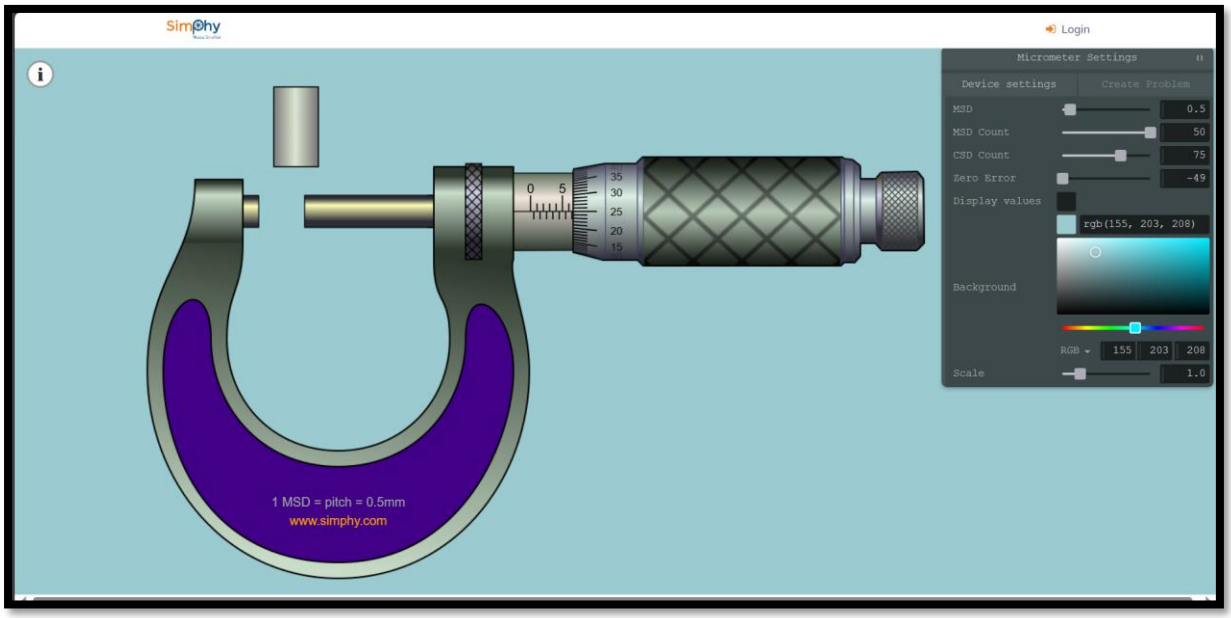



*Figure 3.1* shows a simulator for a micrometer, a piece of measuring equipment commonly used in RP 05 and RP 06. In this simulator, you can turn the ratchet and the thimble using your mouse, causing the spindle would move in or out like it would with a real micrometer. This is a feature I would like to include in my solution as there are questions in the Physics exams that are on the use and operation of micrometers.

*Figure 3.2*

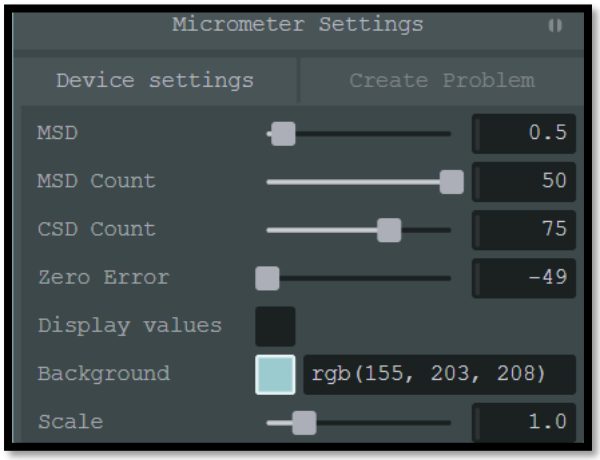



*Figure 3.2* shows the device settings for the micrometer. Note that the zero error can be changed and also be randomised under the Create Problem tab. This would allow users to practice with micrometers they do not have at home and would be a key feature to my solution. I will also apply similar features to other pieces of measuring equipment.

## Summary

Features to include (Red – High priority, Amber – Medium Priority, Green – Low priority):

- Realistic equipment that the user can adjust and use in the same way they do in real life.
- Randomly generated systematic errors added to the measuring equipment to give a lab-like experience.

## Research into similar software – GeoGebra (Gilbert Bett, n.d.)

GeoGebra is a website that holds a collection of free to use maths tools including graphing calculators. Users can create their own maths simulators and post them on the website for other users to use. I will look into a simulator that simulates stationary waves like the ones in RP 01- stationary waves on a string to give me a good idea of how to represent this practical activity digitally.

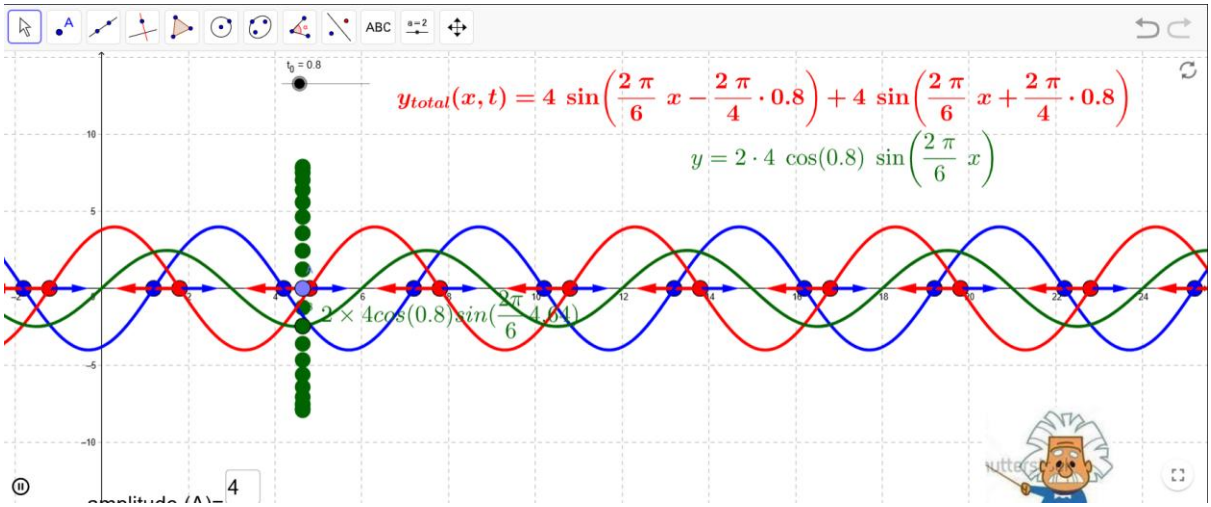



*Figure 4.1*

*Figure 4.1* shows a stationary wave simulator made by Gilbert Bett and Tom Owsiak on GeoGebra. The simulator shows an abstract, mathematical view on graph paper so the user can see where the nodes and antinodes are on the wave. This is a very abstracted view as Bett and Owsiak removed the real-life details like the details of the string and wave generators and they highlighted the details of the wave and its reflected wave that superpose to form one stationary wave. This gives users a conceptual understanding as they do not need to worry about other details. In my solution, I will give an option to show a view similar to this for RP 01 – Stationary Waves to give users a conceptual understanding.

This solution is simplistic and does not show the waves being reflected at a boundary to produce the second wave. The solution also does not allow users to change values like frequency and mass per unit length of the string. These are key features in my solution so they will be included.

The solution shows the mathematics behind the practical activity and shows each initial wave (in red and blue) as sine waves and the resulting green wave of these two superposing as the sum of these sine waves. This is a good approach to the problem and will be used in my solution as it would be relatively simple to implement.

## Features of the proposed solution

| Feature | Evidence | Justification | Limitations | Priority |
|---|---|---|---|---|
| **User Login**<br>User enters a username and password. When signing up, users enter their school and full name | Interview with Dante Question 9 | This allows the user to save progress when performing revision questions and allows the user to save data from the practical activities run. Full names and schools are required to allow assignments to be sent to students from teachers. | Each student having their own account is not essential to the functionality of the program so the user login is not of high priority. Therefore, I will only develop it if I have time after developing the practical activities. I could allow teachers to link their accounts to their pupils so that they can assign work but this would extend development time out of the timeframe I have been given so will not be included. | Low |
| **Ability to set up practical activities**<br>This would include placing equipment in its position using drag and drop functionality. | Survey Question 4a | This allows the user to gain a lab like experience of setting up an RP and helps them revise for method questions in the exam. | | High priority due to results from question 4a |
| **Sliders to adjust independent variables and show their effects in real-time**<br>The solution will also have an input box so the user can input numerical values | Survey Question 4b<br><br>Survey Question 6<br><br>PhET simulations Figure 1.3 | This gives the user a fundamental understanding of what is happening during the practical activity as it would show the effect of changes of one independent variable on the dependent variable. | | Very high priority due to survey results |

| Feature | Evidence | Justification | Limitations | Priority |
|---|---|---|---|---|
| **Have simulators for all required practical activities** (RP 01 – RP 12) | AQA-A level Physics specification | Paper 3 in A-level Physics has questions on all 12 required practical activities so all should be covered in the simulator to assist revision | At the time of analysis I have not learnt RPs 07-12 so it would not be possible to create the simulator for these as I do not know the theory myself. Creating simulators for the other 6 practical activities may not be possible in the time available so simulators will be created in order of the rankings created by respondents to Survey Question 3. | See Survey Question 3 |
| **Ability to experiment with virtual circuits** | Survey Question 4c<br><br>PhET circuit construction | This would allow users to gain a conceptual understanding of electricity and circuits whilst also allowing users to set up variants of RP 05 and 06 themselves which is a high priority feature so should be included. | | Medium priority due to survey question 4c results. |
| **Revision Questions** This would include getting the user to setup the required practical by dragging and dropping equipment and asking questions on what errors are caused and safety considerations that must take place. | Survey question 4d | This feature would help users revise for their exam and not just re-perform the RP.<br>A-Level Physics Paper 3 often includes questions on safety and error considerations so this would benefit students using my solution. | General Revision questions will not be included in the final solution as it will increase coding time and the simulations need to be prioritised more. | Error questions – medium priority<br><br>General questions – low priority |

| Feature | Evidence | Justification | Limitations | Priority |
|---|---|---|---|---|
| **Option to add errors** This would include air resistance, components heating up in the circuit construction simulator and stabilising equipment. Errors can be randomised or set by teachers. | Survey question 4e and 4f SimPhy micrometer | This feature would give the user a lab-like experience and allow the user to perform better in questions in their examinations on errors. | There are very few errors to introduce in RP 01 which will likely be developed first so may be hard | High priority due to results in survey questions. |
| **Option to have an abstract view and realistic views to the RPs** The abstract view would show the Physics behind the RP while the realistic view would only show what you see in real life. | Survey Question 4g and 4h<br><br>PhET simulations figure 1.1<br><br>Research into similar software – GeoGebra | This feature would allow the user to gain a conceptual understanding of each RP as well as being able to recall what the practical activity was like in the lab. | | Realistic View – High priority<br><br>Background physics view – medium priority |
| **Have simulators for practical activities not in the A-level specification** | Survey Question 6 | This feature would allow users to gain an understanding of theory learnt in A-Level Physics that does not have a relevant required practical activity. | This will not be possible in the allocated time for coding as the focus would be on the required practical activities as this is the main purpose of the solution. | Very low priority |
| **Measuring equipment from drag box** This will include adding systematic errors to the equipment if the user chooses to | PhET simulations Figure 1.2 SimPhy micrometer | This feature would allow users to measure equipment as if they were in a lab and gain experience of systematic errors in the equipment. | | Medium Priority |

| Feature | Evidence | Justification | Limitations | Priority |
|---|---|---|---|---|
| **The user is able to record results in a table and the software plots this in a graph** | Interview with Dante Question 8 | This feature would allow students to gain experience of how to draw graphs from results of a required practical and would give the user a better understanding of the relationships between variables. | | Medium Priority |
| **The user can view 2 simulations at the same time** | Interview with Ian about PhET | This would allow users to see the effect of a variable by comparing the displays of two simulations. This would give users a deeper understanding of the practical activity. | | Medium |
| **The user can pause the simulation at any time and there is a slow mode option.** There will be a pause icon at the bottom of the screen allowing the user to get a freeze frame of the RP. | PhET Wave Interference Figure 1.5 | This would allow the user to think through the practical activity more thoroughly as they would be able to see what happens at each stage. | | Medium |

## Further Limitations:

As discussed in the row "Have simulators for all required practical activities", I will not have time to create simulators for all of the required practical activities. I prioritised the development of simulators for RP 1, RP 2 and RP 4 as this is what was asked for in survey questions 2 and 3. However, given the restricted timeframe, I will not be able to make simulators for all three of these but will focus on one. It makes sense to start with required practical activity 1 so that the simulators will be released somewhat sequentially.

2 simulators will be made based on this required practical activity. One will display a realistic view that would somewhat match what the activity looks like in real life and the other will display an abstracted view of the practical activity, showing two waves moving in 2 different directions, adding up to form one stationary wave.

I decided to create 2 separate views due to responses from the survey (Questions 4g and 4h).

## Features that make it solvable by computational methods

### Abstraction

Abstraction is needed to remove unnecessary details and highlight details that are important. This allows me to solve a problem more easily because the solution becomes clearer after using abstraction. A key example of abstraction in my project is the different views of the practical activities that the user can choose. I could make the simulator look completely realistic and minimise abstraction to create a full lab experience or I could create an abstract view like PhET simulations to give a conceptual understanding. I will focus on the realistic view because my stakeholders want an application that will recreate a Physics experiment as if it were in a laboratory. However, this realistic view will still use some abstraction as details like the design of an ammeter are not important, as long as the ammeter reads off the current.

### Thinking Ahead

Thinking ahead allows me to see any problems that I may encounter when developing the software. The project is solvable because it has inputs such as the values of independent variables, where equipment has been dropped and options such as the view of the practical activity and the choice of what errors can occur. The system should then produce outputs of the results from performing the practical activity as well as a visual representation.

Preconditions must also be considered when thinking ahead. The values of independent variables must be within a specific range and in the correct format (cannot enter symbols).

Reusable components must be used to shorten development time and reduce redundant code. There are 2 RPs (required practical activities) that both involve waves (RP 01 and RP 02) so components calculating these waves must be reused from one RP in the solution to the other. RP 05 and RP 06 both use circuits so components involving circuit calculations can be reused. The sliders to adjust independent variables can be reused across all RPs. Measuring equipment, control panels and the controls a user has are all components that can be reused.

Caching is used to save time when retrieving data. RPs recently performed will be cached to quickly return to progress.

### Thinking Procedurally

Thinking procedurally would allow me to break down the problem and give my solution a structure.

The problem can be broken down into each RP and each required practical can be broken into displaying results, displaying the practical and the measuring equipment used.

Additionally, the program will follow the steps of signing in, choosing an RP, choosing whether the user wants to set up the RP themselves, choosing the variables and then displaying the effect of this change.

### Thinking Logically

Thinking logically allows me to see where decisions have to be made and allows me to see the flow of the program.

This project is solvable because it requires logic. For example, calculations and simulations are performed if the user chooses to start the simulation. When the home icon is clicked, the program returns to a home page where different simulations can be chosen. The correct simulator is displayed if it is selected from the home page.

## Decomposition

Decomposition will allow me to break down the problem into more manageable sub-problems that are easier to solve. The problem can be broken down into each RP, the controls and different parts of the user interface. I will create a top-down diagram in the Design section which will show a more detailed decomposition of the problem.

## Concurrency

Thinking concurrently will allow me to see where processes have to occur simultaneously. An example of this is where I need to update the measurements recorded by measuring equipment at the same time as the simulator displaying the RP. I could use parallel processing to solve this problem but to reduce the time to code the solution, I will allow each process to run sequentially, but seemingly concurrently.

## Hardware and software Requirements

| Requirements: | Explanation |
|---|---|
| **Hardware** | |
| Mouse | To allow the user to interact with the simulator. |
| Keyboard | To allow the user to input values for different variables. |
| A display with resolution of at least 1366 * 768 | This will allow the user interface to be properly displayed without layout issues. |
| 1GB RAM | To allow space for the program to be run. |
| 2GB free storage | To allow space to store the program. |
| **Software** | |
| Pygame | A python library to use for the user interface |
| NumPy | Python library for maths calculations |
| Windows, Mac or Linux Operating System | These are the operating systems that are supported by python. |
| Matplotlib | A python library that allows you to plot graphs. |
| Python 3.x installed | So the program can be run. |

## Additional Requirements

For this project to be successful, I will need the ability to install Python libraries because the user must have the correct libraries installed in order for the program to function. The user will also need a basic understanding of stationary waves and Physics so that they understand what is being shown.

## Success Criteria

The following points are the success criteria for my solution that will mean that it meets the requirements set by my stakeholders. Due to the limited time allocated for development, I have split the criteria into essential (black) and desirable (red) so that I can ensure that I have the essential features covered if I do not have time to cover all features. The second column shows how I will illustrate how each of the criteria have been met in the evaluation stage. The third column shows why the particular criteria is necessary. I have split up the criteria into functionality (how fit for purpose the software is), usability (how easy the interface is to use) and robustness (if the software is in good condition).

| Success criteria - Functionality | Measured By | Justification |
|---|---|---|
| **Users sign in with username and password. Users have to enter there password twice in sign up.** | Testing video | This gives each user an individual account so that they can store their own data. |
| **A window in the corner of the simulator allows users to change speed settings and have two snapshots side by side** | Testing video | This was required by Ian in his interview and would allow users to compare results. |
| **The practical activity can be set up by dragging equipment into its desired location from a window containing all equipment.** | Testing video | This is needed to give users an understanding of the setup of Required Practical activities which is needed for their exams. |
| **Each user has a profile that saves their progress in questions and results from performing practical activities.** | Testing video | This would allow users to look back at previous data values they have recorded and continue working with these rather than re-performing the practical activity. |
| **The simulator should replicate how the practical activity looks in real life.** | Ask stakeholders for feedback | This is a key feature as the simulator must replicate a Required Practical Activity lesson experience. |
| **The simulator should help a student better understand the Physics behind a Physics required practical activity.** | Ask stakeholders for feedback | This is a part of the problem that I look to fix with my solution. |
| **The simulator should help a student who has missed a required practical lesson or has forgotten the lesson get experience of performing the practical activity.** | Ask stakeholders for feedback | This is the problem that was set out at the beginning of my document so the program must meet this criteria. |
| **Systematic Errors can be added to measuring equipment and can be randomised.** | Testing video | This would give users a lab-like experience of errors gained in class. |
| **The simulator should follow the laws of physics and match the equations that students learn in A-level Physics.** | Ask stakeholders for feedback, Testing video. | This is a key feature as the simulator is intended to replicate real life. |
| **System should load within 5 seconds of opening the program** | Testing video | This means that the user would not get tired of waiting for the system to load. I could have chosen a shorter time but I |

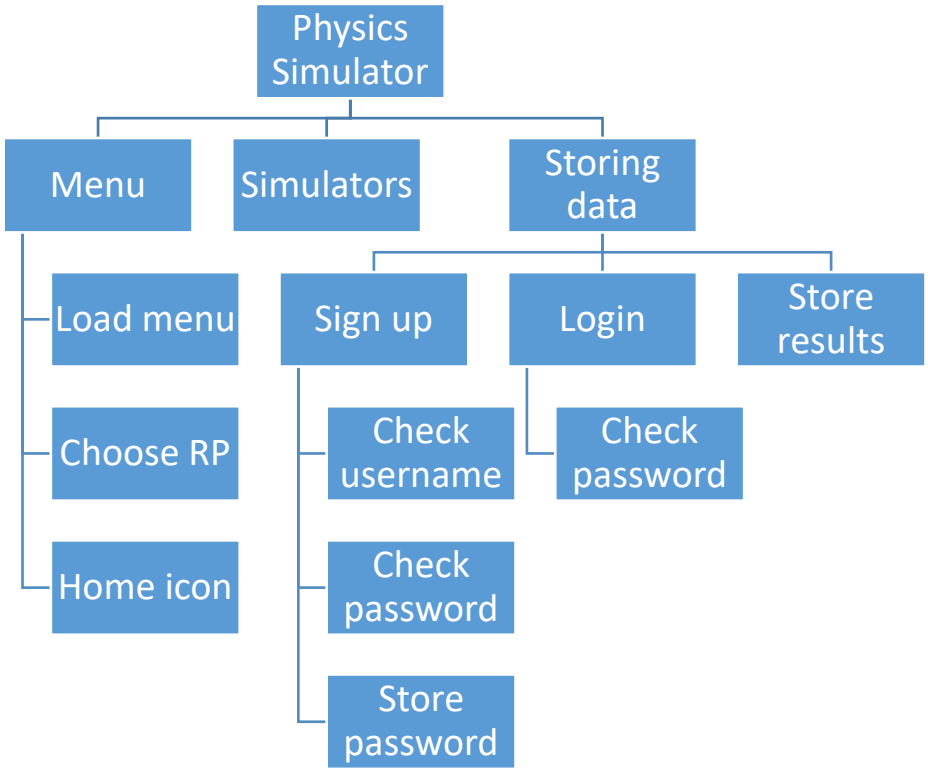
| Success criteria - Functionality | Measured By | Justification |
|---|---|---|
| | | thought it was important to be realistic at this stage. |
| **Each simulator should load within 10 seconds of being selected** | Testing video | This means that the user does not get tired of waiting for the RP to load. |
| **The effects of a change in a variable should be shown within 1 second of adjusting it.** | Testing video | This would give the user a feeling of real time adjustment of independent variables. |
| **When running the simulators, they should move smoothly and not freeze.** | Testing video | This would make the simulations more realistic as they would not freeze in real life. |
| **The program should be a viable model for future development** | Code review | It is important that the code is easy to develop into future models as there are always features that can be added to the solution. |

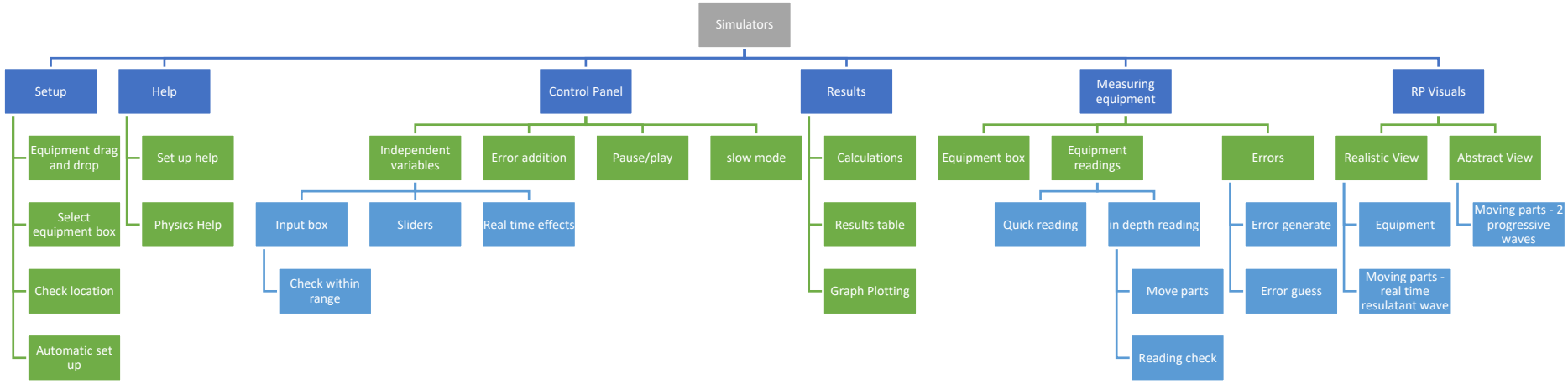
| Success criteria - Usability | Measured by | Justification |
|---|---|---|
| **The user is presented with a menu that allows the user to select a simulator. This should be intuitive and easy to use.** | Testing video/ User feedback | This would give the user an intuitive user interface that would allow them to choose a specific simulator. |
| **A home icon is on every page which allows users to return to the home page to select another simulator. The function of the icon should be obvious by the way it looks.** | Testing video/ user feedback | This allows the user to easily move between pages on the application. |
| **An undo button is available on each simulator so that changes to variables can be reversed. It should be obvious from first glance what this button does.** | Testing video | This would allow the user to reverse changes to variables and equipment without having to start again from the beginning. |
| **The user interface should exaggerate colours of equipment in the simulators.** | Testing video/ user feedback. | This would give the user an attractive user interface so they use the application more and it would emphasise equipment used in the practical activity. |
| **The user interface should be intuitive and users should understand what certain components do. For example, they should be able to understand what a slider does just from a glance.** | User feedback | Making the user interface intuitive to use is important as they are likely to not use the solution if they cannot understand how it works. |
| **Equipment should be instantly recognisable** | User Feedback | The user needs to know what equipment does from a glance so that they recognise it from the lab and know what the equipment does. |

| Success Criteria - Robustness | How to evidence | Justification |
|---|---|---|
| **The password is checked to be 6 characters long and is covered up by *s.** | Testing video | Ensures that the user's data is secure as passwords are checked for strength.<br>This is desirable because sign up is desirable |
| **A required practical activity simulator should be able to be re-loaded and started from the beginning.** | Testing video | This would mean that the simulator can be run multiple times without error. |
| **The program should deliver correct outputs every time.** | Testing video/ User Feedback | Users would be discouraged to use the software if it delivers incorrect outputs. |
| **The program should allow for two separate simulations to be run sequentially without error.** | Testing video | Users would be discouraged to use the software if they can only access one simulation at a time. |

# 2. Design

## Systems Diagram




Simulators
Setup
Help
Control Panel
Results
Measuring equipment
RP Visuals
Equipment drag and drop
Select equipment box
Check location
Automatic set up
Set up help
Physics Help
Independent variables
Error addition
Pause/play
slow mode
Input box
Sliders
Real time effects
Check within range
Calculations
Results table
Graph Plotting
Equipment box
Equipment readings
Errors
Quick reading
in depth reading
Move parts
Reading check
Error generate
Error guess
Realistic View
Abstract View
Equipment
Moving parts - real time resulatant wave
Moving parts - 2 progressive waves

I have created a top-down design diagram to decompose the problem. This allows me to break down the large task of creating a Physics simulator into smaller, more manageable tasks that are easier to solve. This means that it is easier for me to manage my code and test smaller chunks. It also allows me to have a clear view of what needs to be done in order to successfully create a Physics simulator. I could have used a table to present this data but instead, I chose to use a top-down diagram because it is easier to see which module each sub-module falls under and it is easier to view the problem as a whole.

The first diagram shows the top-down diagram for the whole solution split into Menu, the Simulators and Storing Data. I have split up the solution like this to easily separate what happens within each simulator and what happens outside of the simulations. I have then created a separate top-down diagram for the simulators as this is the main section of the solution and should be focused on.

## Priority Order

I could work on each of these branches in order from left to right but I have decided to order each branch in terms of priority to allow me to create modules that other modules are dependent on first. It also allows me to focus on modules that are more important for the overall solution to be a success before looking at the "desirable" features set in the analysis stage. This is the order that I shall use:

1. **Results calculations**. This is because the visuals need the results to already be calculated so a key part of the project is dependent on this.
2. **Simulator visuals.** This will be done second because the main point of the solution is to help visualise Physics practical activities so it is very important that this is completed.
3. **Control Panel.** This is another key part of the solution as users need to be able to experiment with adjusting variables and seeing their effects. This could have been swapped with the simulator visuals but I have chosen this order because the design of the effect of independent variables will be easier after the visuals have been developed.
4. **Menu** This is needed in order to select and load the simulator. The solution would not be functional without it.
5. **Results.** This is a key part of the practical activity process so should be included.
6. **Help.** This would make the system easier to use. If the system is already intuitive, this may not be essential.
7. **Measuring Equipment.** This is not a key part of the solution but is needed for users to grasp a full understanding of the practical activity process.
8. **Setup.** The solution will be functional without this so will be done last because my priority is to make the solution functional in the allowed timeframe.
9. **Storing Data.** This is not essential because at first there will not be much to store as results displayed as graphs is a low priority feature.

## Explanation of each module

### Storing data

This is to allow users to save results from a practical activity so that they can continue to work on a specific Required Practical activity (RP) even after restarting the program or shutting down the computer.

#### *Sign up*

Allows new users to register and create an account so results are saved to their account. It will ask the user for their full name, username, password and the school they go to.

##### Check Username

Checks that the username entered is unique so it can be used as a primary key.

##### Check Password

Makes the user enter their password twice so that they do not mistype it. Ensures that the password is more than 7 characters long.

##### Store password

A hashing algorithm is used to store passwords.

#### *Login*

The user needs to log in every time they open the program or sign in as a guest.

##### Check password

Checks the hash of password entered with the hash stored in the database.

#### *Store results*

This will store the results from an RP that has been performed in a database.

### Simulators

This is what happens within each simulator. I have split this into general features that the simulator of the realistic view and abstract view of RP 1 share so it is clear what modules can be re-used across both simulators. I discovered that the main difference between the two simulators was the waves that are shown. This is why I split up the RP visuals into “realistic view” and “abstract view”

#### *Setup*

Allows the user to set up the required practical activities.

##### Equipment drag and drop

The user can select the equipment to use for the RP and drag it near to where it should be located.

##### Equipment selection box

The user can select what equipment they want to use

##### Equipment check location

This would load the equipment in the correct place if it has been dropped near the correct location.

##### Automatic set up

The user is presented with an option to go to the RP without setting it up. If this option is selected, all equipment will be loaded in the correct location.

### *Help*

This would give guidance on the physics behind the practical activity. The user interface should be intuitive to use but help will be given if required.

#### Set up help

A help icon will appear on the simulator page. If the help icon is selected, a window will appear and give the user a hint on how to set up the RP.

#### Physics Help

A help icon will appear as results are being processed and if clicked on, a window will appear explaining the physics behind the RP.

### *Control Panel*

This is a box on the side of each RP where components of the practical can be adjusted. This is an example of a component that will be used across all of the RPs but will have specific features for different RPs.

#### Independent variables

The control panel will allow independent variables to be adjusted by a slider or inputting a specific value. The range of values that can be chosen from changes for each RP.

##### Input box

Values input as independent variables are checked that they are within the given range.

##### Sliders

Values can be adjusted by a slider.

##### Real-time effects

Calls the relevant display practical function for the new value of the variable.

#### Error addition

A tick box that allows the user to add errors to the simulation to simulate a real life environment. This will only be available in the RP 04 – Determination of Young's Modulus simulator because errors in real life are caused by many factors and are often negligible in RP 01 – Stationary Waves and RP 02 – Interference effects.

#### Pause/play

A button that allows the user to get a freeze frame of the RP or continue from where it left off.

#### Slow mode

An option that plays the simulation at one quarter speed or a speed selected by the user.

### *Results*

The following is about the calculations and manipulation of the results from the practical activities. These are very specific to each RP so I have separated each RP in the systems diagram. I have separated this from the visuals because the results modules will produce accurate results no matter the setup, allowing users to compare the error free results with the realistic ones if they have chosen to add errors.

#### Calculations

Given the input of the independent variable and the setting of control variables, the dependent variable will be calculated and output. Calculations are needed for each of the RPs.

#### Results table

The output from the calculations can be placed into a results table which can be pre-generated or generated as users try different independent variables.

#### Graph plotting

An ideal graph with a smooth line can be plotted as well as graphs from the results table.

### *Measuring equipment*

This is the measuring equipment that the users use to get their readings and can play with in order to understand how the equipment works, so they have a better understanding for their exams. I have grouped this as 1 module for all RPs as these components are reusable. For example, a ruler is used in multiple RPs

#### Equipment box

A box on the side of the screen that allows measuring equipment to be drag and dropped into the desired location

#### Equipment readings

##### Quick reading

This allows the user to place the measuring equipment over the relevant parts of the simulation and it shows the value of the measurement.

##### In depth reading

This would open a new window that allows the user to explore the equipment like it were real to give them a lab like experience.

###### *Move parts*

This would allow the user to move parts of the equipment and see the effect of changing it. This is needed for the micrometer and vernier caliper simulators only.

###### *Reading check*

The user is given an input box to input the reading and its uncertainty based on the display and this is checked against the true reading value. A pop-up window confirms whether the user is correct or incorrect.

#### Errors

An option to include systematic errors to the equipment

##### Error Generation

A randomly generated addition to values given by measuring equipment.

##### Error guess

The user can work out what the systematic error is and enter it into an input box. The user is told if they are correct.

### *RP Visuals*

I have divided this into each RP as each RP has its unique visuals. I have then subdivided this into the stationary equipment and the moving parts of the practical activity so I can tackle each separately.

#### RP 01 – Stationary waves – Realistic View

##### Equipment

For RP 01 I need to display the signal generator, the mass hook, the vibration generator, the wooden block, the table and the string all in the correct position.

##### Moving parts

A wave should be shown oscillating up and down. The relevant harmonic has to be displayed on the string given the correct frequency. If the frequency is not at a certain harmonic, there is not a stationary wave displayed at the maximum amplitude.

#### RP 01 – Stationary Waves – Abstract View

##### Equipment

Only the mass hook is shown so that the tension can be adjusted, but other equipment is removed so that the user can focus on the formation of the waves.

##### Moving Parts

The waves move slower for the abstract view. The same stationary wave moving up and down is shown but slower. Two progressive waves are also shown moving in opposite directions. The mathematical sum of these waves should form the stationary wave.

## Menu

### *Options between RPs*

An icon showing a still from each RP is presented and if the user clicks on it, it takes them to that RP.

### *Load menu*

Loads the main menu once the user has signed in.

### *Home icon*

A home icon that is located on every page to take them back to the menu screen.

## Choosing Development Language

For this project, I will use the language Python. This is because it a language that is very easy to read so will be easier to maintain my project. It also has an extensive collection of libraries that I can use to display the simulators well.

For my IDE, I will use Visual Studio Code because this provides good highlighting and autocompletion.

## Results Design

I will start my design by designing the calculation, storage and representation of the results of the simulations. This is because the results are an important part of the practical activities. Parts like the RP visuals cannot be started until the results calculations are performed, and it is a quick task that will give a quick win at the start of my project.

### Algorithm Design

Thinking ahead, the inputs needed for this module are the tension in the string, the length and the mass per unit length of the string. This module needs to output the frequency where each harmonic of a stationary wave is observed. Preconditions that must be met are that the tension, length and mass per unit length must be positive.

The speed of the wave also needs to be calculated and can be done by v = sqrt(Tension/mass per unit length of the string). This needs to be calculated so I know how fast the progressive waves in the abstract view need to be.

```
function calcFreq(T, mu, l)
      // calculates the frequency of the first harmonic
      h1Freq = (1/(2*l))*sqrt(T/mu)
      return h1Freq
endfunction


function calcSpeed(T,mu)
      // calculates the speed of the
      return sqrt(T/mu)
endfunction
```

These both use the mathematical calculations of the first harmonic and speed of a wave learnt in A-level Physics. Both of these functions can be written in one line so could be written out instead of calling these functions. However, I have used abstraction because when accessing the frequency of the first harmonic, I do not need to consider how it is being calculated, I just call these functions.

## Key Variables and Data Structures

| Variable name | Data type | Description | Justification |
|---|---|---|---|
| **mu** | Float | Mass per unit length of the string | This is needed to find the speed of the waves so that the frequency of harmonics can be found. "Mu" is used because the Greek letter $\mu$ is used to represent the mass per unit length of an object. |
| **T** | Float | The tension in the string caused by adding weights to the end of it. | This is needed to find the speed of the waves so that the frequency of harmonics can be found. The symbol "T" is used for tension in Physics so will be used in development. |
| **l** | Float | The length of the string | This is needed to calculate the speed of the stationary waves on the string. The variable name "l" is used because this is usually the symbol used for the length of the string in Physics. |

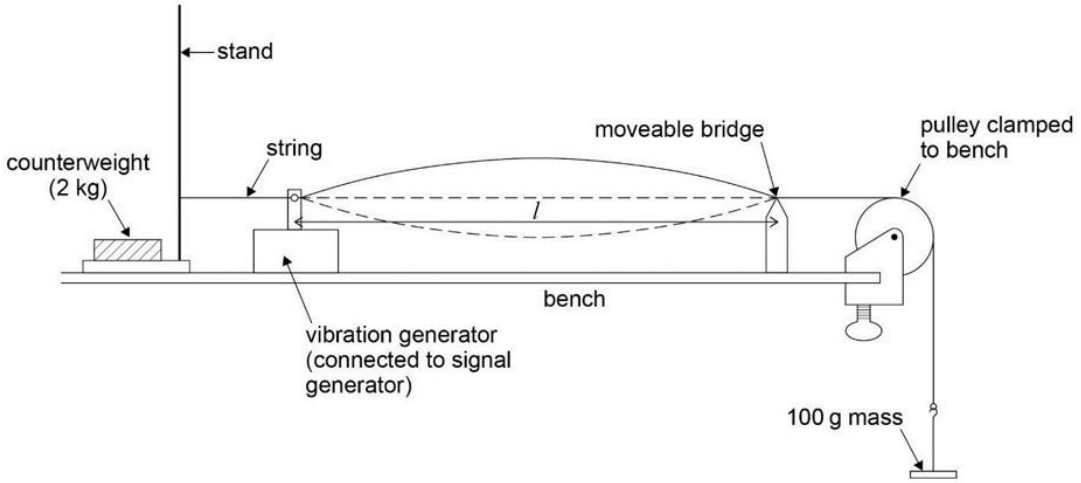
## Validation

No validation is used in this section because there is no user input.

## In Development Test data

| Module tested | Data | Test data type | Expected Output | Justification |
|---|---|---|---|---|
| **calcFreq** | T = 1.96<br>Mu = 0.0007<br>l = 1 | Normal | 26.458 | T = 1.96 because it is the force caused by a 200g mass. 0.0007 is a usual length for the string in this practical activity |
| **calcFreq** | T = 0.98<br>Mu = 0.001<br>l = 1.5 | Normal | 20.870 | I could have tested with one set of values but I chose to use a second set of values to ensure it works every time. |
| **calcSpeed** | T = 2.94<br>Mu = 0.0002 | Normal | 121.2 | I have only included one set of test data because this function is simple and I am confident that it should work. |

## Simulators – Required Practical 1 – Stationary Waves visuals – Moving Parts – Abstract and Realistic View




This image is from PMT (Physics and Maths Tutor, 2024) and shows the Practical Activity I am trying to simulate.

This section is the design of the algorithms that will display the visuals of the first required practical activity. More specifically, it will form and display the stationary waves produced from the frequency the user sets the signal generator to.

I am designing and developing this section early on because it is an integral part of the final solution as the practical activities need to be displayed correctly for the user to understand them. I could have created the main menu section first because the solution would not be intuitive to use without it but I have decided to create the visuals of the practical activity first as this is an essential part of my solution.

## Algorithm Design

Initially, I thought that this problem could be solved by using the sum of multiple sine waves that get reflected by each end of the string. This is simple when the frequency of the wave is equal to the frequency of a certain harmonic because at these frequencies the ends of the strings are stationary nodes so will be fixed in place with the wooden bridge and vibration generator. At these frequencies, the sum of only two sine waves is needed to create the stationary wave.

However, when the frequency is not at one of the harmonic frequencies, this problem becomes much more complex. This is because the wave can no longer be represented by two sine waves while keeping the two ends of the string stationary. Playing around with different waves on Desmos (an online graphing calculator), I tried adding damped waves that get reflected by the vibration generator as well as the moveable bridge but still did not find a solution. If I continued with this, it would increase the size of memory used for each sine wave and the time complexity of the program.

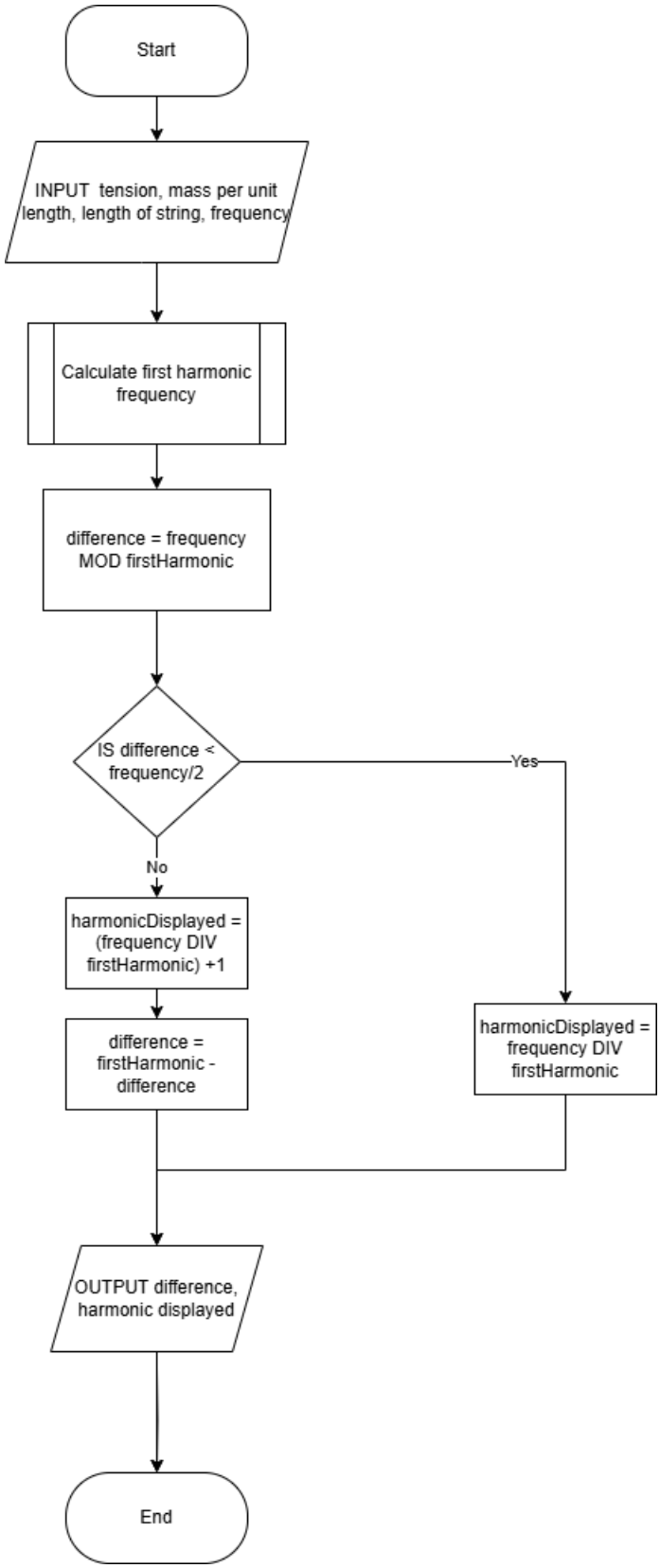
Therefore, I have decided to use a heuristic approach that will deliver a solution that gives a result that is close to real life but not exact. After watching a standing waves demonstration by James Denn (https://www.youtube.com/watch?v=-gr7KmTOrx0), I found that as Denn increases the frequency of the signal generator when the string is at the first harmonic, the amplitude decreases but the string remains at the first harmonic until it becomes closer to the second where it appears to be the second harmonic but with a lower amplitude. Therefore, I have designed the algorithms to calculate the harmonic the frequency is closet to so this harmonic can be displayed.

Before displaying the stationary waves, I needed to create a way of finding the harmonic that is supposed to be displayed. Stationary waves are not only displayed at their harmonic frequencies but are also shown at the frequencies near their harmonic frequencies but with a smaller amplitude. I therefore need to create an algorithm that finds the closest harmonic to the frequency given, the difference between the harmonic frequency and the frequency given so that the amplitude can be adjusted and then display the correct wave.

I have decided to separate this module into two functions. One (closestHarmonic) will calculate the harmonic the wave is closest to and the other (displayHarmonic) will display the harmonic given this information. This will allow me to test the solution more easily as I can more easily spot errors because I have split the function up into functions that only perform one task. It also allows me to use the closest harmonic function alone in later stages to check if the user is near the correct answer when guessing which harmonic the wave is at.

Thinking ahead, I have created a flowchart for the closestHarmonic function to allow me to see where decisions are made and allow me to visualise the solution.

### *Closest Harmonic Flowchart*



This is the pseudocode for the algorithm shown in the flowchart:

```
######################

function closestHarmonic(T, mu, l, f)
      // This function returns the number of the harmonic
      // closest to the frequency the user has set the frequency
      generator to
      harm1Freq = calcFreq(T,mu,l)
      difference = f MOD harm1Freq
      // finds the difference between the frequency the generator is set
      to and the nearest harmonic frequency
      if difference < (f/2) then
      //checks if frequency is closer to the harmonic below it
            harmDisp = f DIV harm1Freq
      // the number of the harmonic is the frequency divided by the 1st
      harmonic rounded down
      else
            harmDisp =( f DIV harm1Freq) +1
      // the number of the harmonic is the frequency divided by the 1st
      harmonic rounded up

      endif

      return harmDisp
endfunction
######################
```

After designing this function, I realised that there is a simpler way to create a function that produces the same output. I noticed that this function is effectively rounding the frequency divided by the frequency of the first harmonic to the whole number. This is the simpler function:

```
######################

Function closestHarmonic(T, mu, l, f)
      // This function returns the number of the harmonic
      // closest to the frequency the user has set the frequency
      generator to
      harm1Freq = calcFreq(T,mu,l)
      closestHarmonic = round(f/harm1Freq)
      //divides the frequency by the frequency of the 1st harmonic
      and rounds to nearest integer
      Return closestHarmonic
EndFunction
######################
```

This first function calculates the harmonic that the frequency is closest to and the difference between this frequency and the frequency of the closest harmonic. This means that the correct harmonic will be displayed and the amplitude will be adjusted as the farther away from the harmonic frequency, the lower the amplitude of the stationary wave.

The next algorithms show how the stationary wave will be displayed.

## Realistic View

This is the pseudocode for the realistic view of the first required practical activity. This will be developed using the library PyGame.

```
######################

Procedure dispReal(T, mu, l, f, A)
// This procedure displays 2 waves, one the reflection of the other
to display a stationary wave that looks like one formed with lab
equipment.
      harmonic = closestHarmonic(T, mu, l, f)
//gives the number of the harmonic by calling the closetHarmonic function
      firstHarmonic = calcFreq(T,mu,l,f)
      harmFreq = harmonic * firstHarmonic
//calculates the frequency of the harmonic to be displayed
      wavelength = v/f
      flip = 1
      while running:
            flip = flip * (-1)
// flips the graph every time to appear stationary
Display flip/sqrt((f^2-harmFreq^2)^2+(bf)^2) * A * (sin(harmFreq * x * π))
      EndWhile
endProcedure
######################
```

After watching the James Denn video (https://www.youtube.com/watch?v=-gr7KmTOrx0), I noticed that you can only really see the stationary wave in two positions: at the top like this



and at the bottom like this



Therefore, I designed this algorithm to display the wave alternating between these two positions to give the illusion of a stationary wave. This was achieved by multiplying the equation by the variable flip and changing the sign of flip every frame.

$$A * (\sin(f * x * \pi - vt) - \sin(f\pi(-x) - vt))$$

This equation was used as it shows the sum of 2 sine waves travelling in opposite directions. X is multiplied by pi so that the wavelength of the wave is 2 and not 2 pi so it is easier to manipulate the graph. X is multiplied by f so that the frequency adjusts the wavelength of the wave appropriately. Vt calculates the distance that the waves move so that they can interfere at particular values of t. The whole formula is then multiplied by A so that the amplitude of the wave can be adjusted.

I originally designed the equation above to be multiplied by $\cos(((f - \text{harmFreq}) * \pi)/(2 * \text{harmonic}))$ as an estimation of how the amplitude changes as the frequency the user inputs gets

further away from the natural frequency of the harmonic. However, after further research, the equation $\frac{T}{mu*l*\sqrt{(f^2-harmFreq^2)^2+\left(\frac{b*f}{mu*l}\right)^2}}$ is more accurate where b is a constant of damping. I found this equation on Desmos (https://www.desmos.com/calculator/cq13mr6fpc).

It took some time to find this equation but I realised that stationary waves are part of a topic in Physics called resonance. I revised this topic in Physics and found this graph on a PMT revisions note page (Physics and Maths Tutor, 2025):

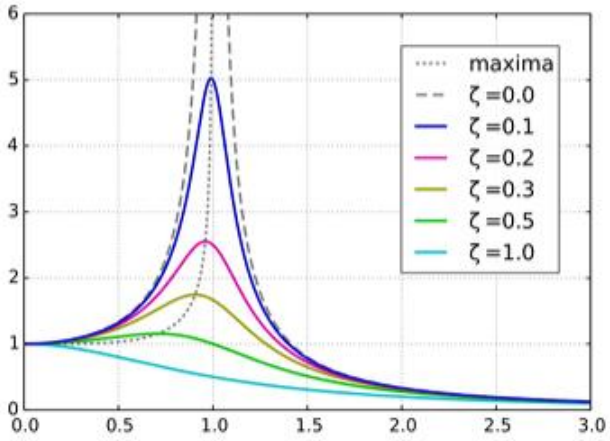



Image source: Geek3,CC BY 3.0

This graph shows the amplitude on the y axis against the proportion of the resonant frequency on the x axis. No equation was supplied with this image so further research had to be done so I can use an equation to calculate the changes in amplitude. I found the one on Desmos and then adjusted the variables in the equation so that they are suited for this procedure.

This equation makes sense because it shows that the larger the difference between the frequency set to and the frequency of that harmonic, the smaller this multiplier will be and therefore the smaller the wave would be. As seen in the James Denn video, the larger the frequency, the smaller the amplitude of the oscillations of the stationary waves. Therefore this part : $\left(\frac{b*f}{mu*l}\right)$ of the equation makes sense because increasing the frequency would cause the size of this multiplier to decrease. I will have to test to find a suitable value for b. I will use b = 0.2 initially.

Abstract View:

In the abstract view, there will be three sine waves shown on the screen. There will be one resultant wave which is the one you can see in real life but moving at a slower speed, selected by the user. There will also be 2 progressive waves moving in opposite directions shown that make up this resultant wave shown to give the user conceptual understanding.

This will only be shown at the harmonic frequencies because more than two waves would have to be drawn if the frequency was not at a harmonic and this would look cluttered and unappealing.

I could create a class for a sine wave and use inheritance to create subclasses for a resultant wave and the 2 progressive waves. This could be done by creating methods that allow me to move the waves. However, it will be easier to create one procedure that displays all three waves at once because the manipulation of these graphs is not complex and can be performed with one calculation.

```
##############

Procedure dispAbstract(T, mu, l, f, A)
// This procedure shows how the abstract view of the required
practical activity. It displays 2 waves travelling in opposite
directions and the resultant wave by summing these 2 waves.
      harmonic = closestHarmonic(T, mu, l, f)
      firstHarmonic = calcFreq(T,mu,l,f)
      v = calcSpeed(T,mu)
      harmFreq = harmonic * firstHarmonic
      wavelength = v/f
      b = 0.2
      while running:
            Display ∗ A ∗ (sin(f ∗ x ∗ π − vt))   in blue
            // Displays one wave moving right
            Display −sin(fπ(−x) − vt)) in red
            // Displays a second wave moving to the left
            Display A ∗ (sin (f ∗ x ∗ π − vt) − sin(fπ(−x) − vt))   in black
            // Displays a resultant wave of the addition of the 2
waves
            Check still running
      endWhile
endProcedure
##################
```

I have removed the multiplier from Desmos from this pseudocode because that multiplier ($\frac{T}{mu*l*\sqrt{(f^2-harmFreq^2)^2+\left(\frac{b*f}{mu*l}\right)^2}}$) reduces the amplitude as the frequency is increased but since this is the abstract view, I want it show at the same amplitude for all harmonics so that it does not become small and hard for the user to see and understand.

## Key Variables and Data Structures

| Module | Identifier | Data Type/ structure | Description | Justification |
|---|---|---|---|---|
| **All** | screenSize | Tuple (int,int) | Hold the x and y value for the size of the screen | Needed to adjust lines on the screen to the size of the screen |
| **RP 01 visuals – closestHarmonic, dispReal, dispAbstract** | f | Float | The frequency of the waves used in simulation | This is a key variable that is needed to find what the wave looks like. The variable name "f" has been used because it is the usual symbol used to represent frequency in physics. |
| | mu | Float | Mass per unit length of the string | This is needed to find the speed of the waves so that the frequency of harmonics can be found. "Mu" is used because the Greek letter $\mu$ is used to represent the mass per unit length of an object. |
| | T | Float | The tension in the string caused by adding weights to the end of it. | This is needed to find the speed of the waves so that the frequency of harmonics can be found. The symbol "T" is used for tension in Physics so will be used in development. |
| | l | Float | The length of the string | This is needed to calculate the speed of the stationary waves on the string. The variable name "l" is used because this is usually the symbol used for the length of the string in Physics. |
| | v | Float | The speed of the stationary waves | The letter "v" is used because it is the symbol used for speed in physics. It is needed to move the waves at the accurate speed. |
| | harmonic | Integer | Holds the value of the harmonic that should be displayed given the frequency input. | This allows the closest harmonic to be displayed. |

| Module | Identifier | Data Type/ structure | Description | Justification |
|---|---|---|---|---|
| | firstHarmonic | float | Holds the value of the frequency of the first harmonic | This allows the difference to be calculated and is used to find the frequencies of different harmonics. |
| | A | Float | The amplitude of the wave to be used in the simulation | This is needed so that the user can adjust the amplitude and see its effects |
| | L2rWaveY | NumPy array | An array containing all of the y values for the progressive wave moving from left to right. | Using a NumPy array because it allows me to quickly perform calculations on all of the x values. Could have used a list but would be much slower because the data is not stored contiguously in memory. |
| | R2lWaveY | NumPy array | An array containing all of the y values for the progressive wave moving from right to left. | Same as above |
| | resultantY | Numpy array | An array containing all of the y values for the resultant stationary wave | Same as above. |

## In-Development Test Data

| Module tested | Data | Test data type | Expected Output | Justification |
|---|---|---|---|---|
| **closestHarmonic** | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 26 | Normal | 1 | |
| | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 1 | Boundary | 1 | Check if outputs correct harmonic number for lowest possible frequency |
| | T = 0.98<br>Mu = 0.0007<br>L = 1<br>f = 99 | Normal | 5 | I need to check that 1 is not returned every time. |

| Module tested | Data | Test data type | Expected Output | Justification |
|---|---|---|---|---|
| **dispReal** | T = 1.96<br>Mu = 0.0007<br>L = 1<br>f = 26.5, 53, 79.5<br>A = 50<br>screenSize = (500,500) | Normal | A graph showing the first harmonic stationary wave with the wave flipping up and down for f = 26.5. It should then show the 2nd harmonic when f = 53 and the 3rd when f = 79.5 | I chose frequencies where the frequency is near a multiple of the first harmonic so that I can first test that it works when the frequency is at a harmonic before testing what happens when the frequency is between harmonics. |
| **dispReal** | T = 0.98<br>Mu = 0.0007<br>L = 1<br>f = 18.7<br>A = 50<br>screenSize = (500,500) | Boundary | A graph showing the first harmonic, constantly flipping. It should have a greater amplitude than the one when T = 1.96 | I have included this to test that the function works for different tensions and not just when T = 1.96 |
| **dispReal** | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 35<br>A = 50<br>screenSize= (500,500) | Normal | A sine wave with little amplitude or none at all constantly flipping. | I could have used f = 39 as this is halfway between the first and second harmonic so definitely should cause the wave to have no amplitude but I chose to use f = 35 because it should still be very small but it will be easier to see how small the wave becomes when non-harmonic frequencies are used. |
| **dispAbstract** | T = 1.96<br>Mu = 0.0007<br>L = 1<br>Harmonic = 1, 2 ,3,4,5<br>A = 60<br>screenSize = (500,500)<br>multiplier = 0.01 | Normal | One standing wave oscillating up and down in 1st , 2nd, 3rd , 4th and 5th harmonic and 2 waves travelling in opposite directions in different colours making up the standing wave. | I chose to set the multiplier is 0.01 to slow the simulation down so I can see clearly if it is working or not. Testing for all harmonics possible so I can see if it works not just for the first harmonic. |

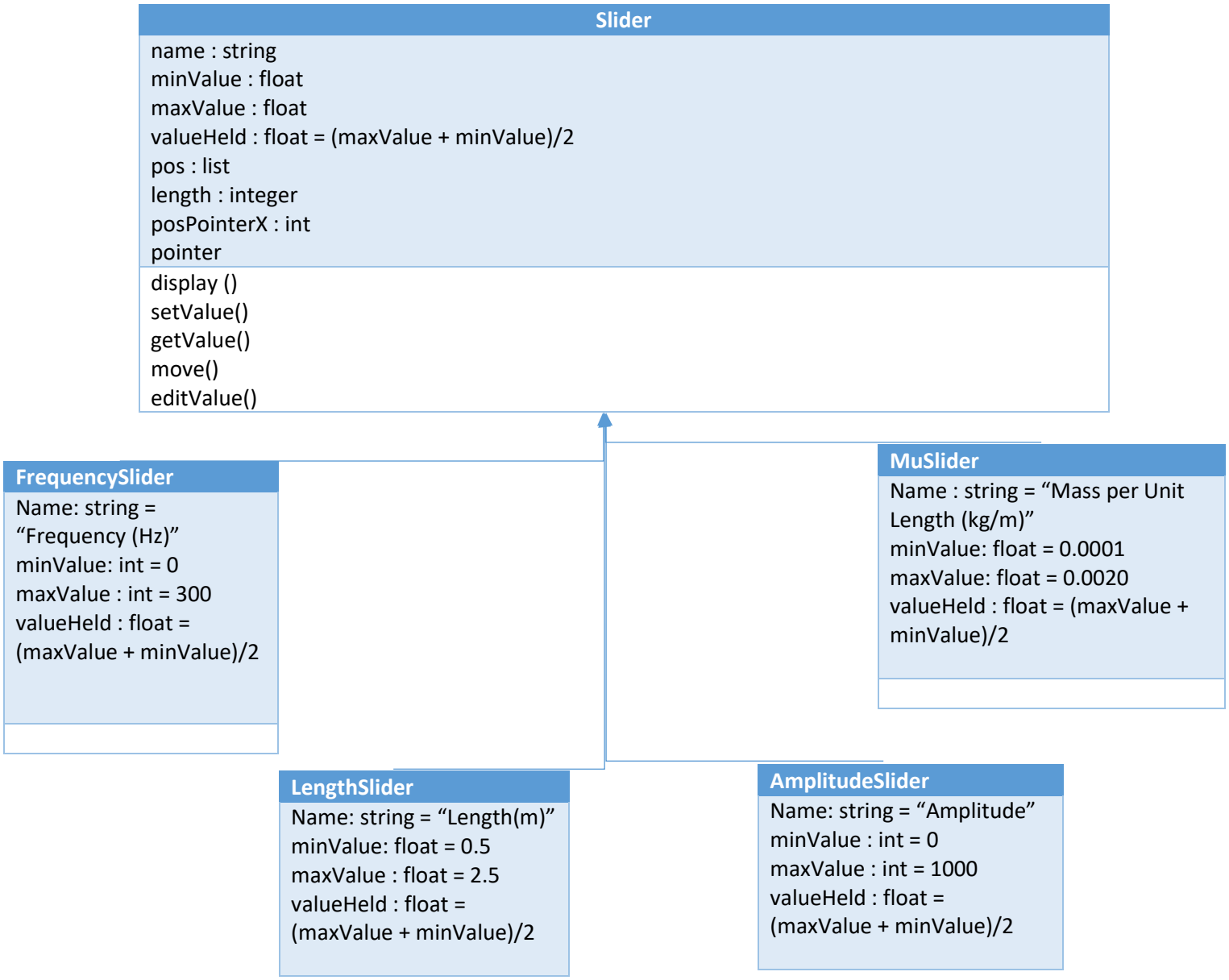
## Control Panel – Sliders

### Class Diagrams

For the sliders module which allows the user to adjust variables by moving a point across a range to a chosen value, I could have used a series of functions for each slider. However, I have chosen to use an object oriented approach so I can use inheritance and have one parent class that has all of the attributes and methods that the sliders share and create child classes for each slider. Encapsulation provided by OOP will mean that I do not accidentally change the values of each variable.

I created classes for specific sliders because the sliders can be used in different cases. For example, FrequencySlider will be used as a slider in the frequency generator in dispReal and at the top of the screen in dispAbstract

Below are the class diagrams for these sliders showing the inheritance of the four subclasses from the superclass “Slider”. Despite tension in the string being a key variable in affecting the shape of the stationary wave, it will not be included as a slider but will be part of the equipment module as it will be changed by the user adding masses to the end of the string.



After creating this class diagram, I realised that there is no need to use inheritance to create subclasses for the specific sliders but they can all be instances of the slider class with different values.

### Attributes and Methods Used

| Identifier | Data Type/ structure/ method type | Description | Justification |
|---|---|---|---|
| **name** | string | Holds the name of the slider to be displayed to give context to the slider. | This allows a procedure to be produced that can display the name with the slider |
| **pos** | Tuple | Holds the position of the centre of the slider. (xCoordinate, yCoordinate) | I used a tuple instead of a list because the values of the coordinates should not change so the immutable nature of a tuple is appropriate. |
| **length** | Int | The length of the line of the slider in pixels | This is needed so the program can calculate what the position of the pointer represents. |
| **valueHeld** | Float | The value that the slider represents. | This is the most important attribute and is needed for other modules to access the value it represents. |
| **posPointerX** | Int | Contains the value of the x coordinate of the centre of the pointer for the slider | This is needed to calculate what the position of the pointer represents. |
| **pointer** | PyGame Rect Object | Contains the location and dimensions of the pointer. | I could have stored the dimensions and the location as 2 separate tuples/ lists but instead I chose to use the PyGame Rect object as it will allow me to know what the coordinates correspond to (top left or centre) |
| | | | |
| **Move()** | procedure | Accepts user input and changes posSlider to the position of the mouse. | This allows the slider to be adjusted. It is a procedure as it changes its own attributes and does not return a value. The slider is not displayed at this point to maintain modularity. |
| **Display()** | Procedure | Displays the slider on the screen | As move() and display() are called every frame, they could be merged into one procedure but are kept separate to maintain modularity so one procedure allows the slider to be moved and the other displays it. |
| **editValue** | Procedure | Allows the user to enter a value into a box and the slider moves to the appropriate location to match the value.. | Same as above. |

## Validation required

Both the move() and editValue() methods accept user input so validation is needed to check that it has been entered correctly. Length and format checks are not applicable to this module.

| Module | Validation Required | Response if not met | justification |
|---|---|---|---|
| **Sliders – editValue** | Presence –input must be entered | Box flashes red and returns to previous value | The program cannot perform calculations on the data if it is not there |
| | Range – The input must be within the range | The input box flashes red and returns to the previous value. | I could make an alert box appear which would help improve usability as users would better understand what went wrong but this would take extra time which could be spent improving other features. |
| | Type – Input must be numerical format | Input box flashes red and box appears saying that input has to be within a range and what range it is. | Same as above. |
| | | | |
| **Sliders - Move** | Range – the movement of the mouse must be within the length of the slider. | If the slider moves past the end of the line, the pointer will move to that end of the line | After talking to my stakeholder, Ian, we have agreed that this is what you would expect from a slider. |

## Algorithm Design

I have decided to design and develop the sliders after the dispReal and dispAbstract modules because the user need to be able to adjust the values that cause the wave to look the way it looks.

The following pseudocode shows the design of the class Slider including its methods that are described above.

```
class Slider
      //This class allows sliders to be instantiated and displayed
      on the screen
      private name
      private minValue
      private maxValue
      private moving // added boolean attribute that is true if the
      pointer of the slider should be moving
      private valueHeld
      private pos //the position on the screen of the centre of the
slider
      private posPointer // position of the pointer to the values
      private colour
      private boxColour

      public procedure new(givenPos, givenLength,
      givenMinValue,givenMaxValue)
            pos = givenPos //the middle of the slider
            length = givenLength //the length of line in pixels
            minValue = givenMinValue
            maxValue = givenMaxValue
            valueHeld =(maxValue+minValue)/2
            moving = False
            posPointer = pos //pointer initially starts in the middle
            boxColour = BLACK
      endProcedure
```

```
    private procedure move()
        //This procedure changes the pointer location to the
        correct position on the slider when moved by the user

        if mouseClicked == True and clickedPointer:
        //checks to see if user is holding mouse down on
        adjustable arrow
            moving = True

        else if mouseClicked == False then
            //sets moving value to False if mouse is no longer
            held down so pointer stops moving
            moving = False

        else if mouseMoving == True and moving == True then
            //changes position of pointer if mouse is moving and
            button is held down
            //Needs to be validated first:

            if mousePos[0] < pos[0] - length/2 and mousePos[0] <
            pos[0] + length/2 then
                //if within range
                posPointer[0] = mousePos[0]
            else if mousePos[0] <= pos[0] - length/2 then
                //if the mouse is to the left of the line
                posPointer[0] = pos[0] - length/2
            else
                //if the mouse is to the right of the line
                posPointer[0] = pos[0] + length/2
            endif
        endif
    endProcedure

    private procedure setValueHeld()
        // changes the value held to the one indicated by the
slider
        range = maxValue - minValue
        relPosPointer = posPointer[0] - (pos[0] - length/2)
        unitsPerPixel = range/ length
        valueHeld = minValue + relPosPointer * unitsPerPixel

    endfunction
```

```
      private procedure editValue()
            //allows the user to enter a value into an input box to
            change the value held
            Display an input box above pos with colour boxColour
            if box is clicked then
                  if valueEntered.isnumeric and valueEntered >
                  minValue and valueEntered < maxValue then
                        // range check and type check - validation
                        valueHeld = float(valueEntered)

                        //updates the pos of pointer:
                        setValueHeld()
                        boxColour = BLACK
                  else
                        boxColour = RED
                  Endif

            Endif
      endProcedure


      Public procedure display()
            //displays the slider on the screen
            Display line across screen with centre pos and length
"length"
            Display minValue as text on left of line
            Display maxValue as text on right end of line
            Midpoint = (maxValue + minValue)/2
            Display midpoint in the middle of line
            Display a pointer at posPointer
            move()
            editValue()

      endProcedure
endClass
```

The constructor method takes in the parameter `givenPos` and `givenLength` so I can later design a way of displaying these without knowing what happens within this class definition (using abstraction).

The method move() only uses the x values of the position of the mouse. This is so that when the user is adjusting the slider, they can have their mouse anywhere on the screen and do not have to keep it over the line.

The method editValue() needs to call the method setValueHeld() because editValue allows the valueHeld to be changed but does not adjust the pointer to the appropriate position. I have made these separate methods to maintain modularity as I could have put all of the code into the editValue method.

## In-Development Test Data

I will test a slider with length 100, minValue 0 and maxValue 100.

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **Sliders – editValue** | 50 | Normal | Pointer to move to a point halfway along the slider | Check that the pointer will move when a valid value is entered. |
| | h | Erroneous – incorrect type | Box to flash red and nothing to change in slider | Having a value for an independent variable as a letter and not a float would cause many errors in my program so I need to check that the validation is working. |
| | Empty box | Erroneous – no data entered | Box to flash red and nothing to change in slider | I could accept an empty box as 0 but I think it makes more sense to assume it was a mistake and get the user to try again |
| | 100 | Boundary - Normal | The pointer to move to the right end of the slider. | This would allow me to see if the slider can move to the very edges |
| | 101 | Boundary - Erroneous | Box to flash red and nothing to change in slider | This is needed to check that the slider does not accept values outside of the range |
| | 55.55 | Normal | The pointer to move to the point represented by 55.5 (just over halfway along) | I need to check that the slider accepts floats as well as integers. |
| **Sliders - move** | Move slider from one end to another | Normal | Value in box should increase from 0 to 100 and slider should stop at right end | This is needed to check that the slider works and is being displayed correctly and that it does not allow itself to be moved past the end of the line. |

## Second Iteration – Algorithm Design

When showing the sliders to Ian (one of my stakeholders) he said that with the sliders you should be able to click anywhere on the line of the slider and the pointer will jump to that position. Therefore, I have come back to design a new move method that moves to a point on the line that is clicked.

```
private procedure move()
          //This procedure changes the pointer location to the
          correct position on the slider when moved by the user


          if mouseClicked == True and clickedPointer:
          //checks to see if user is holding mouse down on
          adjustable arrow
               moving = True

          else if mouseClicked == False then
               //sets moving value to False if mouse is no longer
               held down so pointer stops moving
               moving = False

          else if mouseMoving == True and moving == True then
               //changes position of pointer if mouse is moving and
               button is held down

               if mousePos[0] < pos[0] - length/2 and mousePos[0] <
               pos[0] + length/2 then
                    posPointer[0] = mousePos[0]
               else if mousePos[0] <= pos[0] - length/2 then
                    posPointer[0] = pos[0] - length/2
               else
                    posPointer[0] = pos[0] + length/2
               endif

           else if mouseClicked == True and clickedLine:
               //checks if line is clicked and move pointer to
               where the mouse is clicked
               posPointer[0] = mousePos[0]
          endif
     endProcedure
```

What I have added is highlighted in yellow.

I have added this line as an else if at the end because the user could click the pointer and the slider at the same time as the pointer is located over the line. I would like the dragging option to be prioritised if this is the case so must be else if after this. It also must come after the check that the pointer is moving because it is likely to be dragged along the line and including this line before this check would mean that the new line is run. This may still work, but I would like to know which code is being run at each time so errors are easier to spot.

I have decided to split the editValue method in the Slider class into 2 separate methods. One will remain as editValue which allows the user to enter a number and validates this number. Another method will adjust the position of the slider so it matches the value held. I have decided to do this to make my code have more modularity and so that the adjust slider method can be accessed by the discrete slider I will design next. This will save me from having to re-write code.

This is the pseudocode for the new editValue and adjustPos methods:

```
private procedure editValue()
          //allows the user to enter a value into an input box to
          change the value held
          Display an input box above pos with colour boxColour
          if box is clicked then
               if valueEntered.isnumeric and valueEntered >
               minValue and valueEntered < maxValue then
                    // range check and type check - validation
                    valueHeld = float(valueEntered)
                    adjustValue()
                    boxColour = BLACK
               else
                    boxColour = RED
               Endif
          Endif
endProcedure
```

As you can see this is almost exactly the same as the previous pseudocode but I have got rid of the code that updates the position of the slider and replaced it with a new method that does the same thing.

```
Private procedure adjustValue():
     //updates the pos of pointer:
     range = maxValue – minValue
     pixelsPerUnit = length/range
     relPosPointer = length * (valueHeld-minValue)
     posPointer[0] = relPosPointer + (pos[0] – length/2)
endProcedure
```

This is exactly the same as the code that was previously in the editValue() method but has been changed into its own method to maintain modularity.

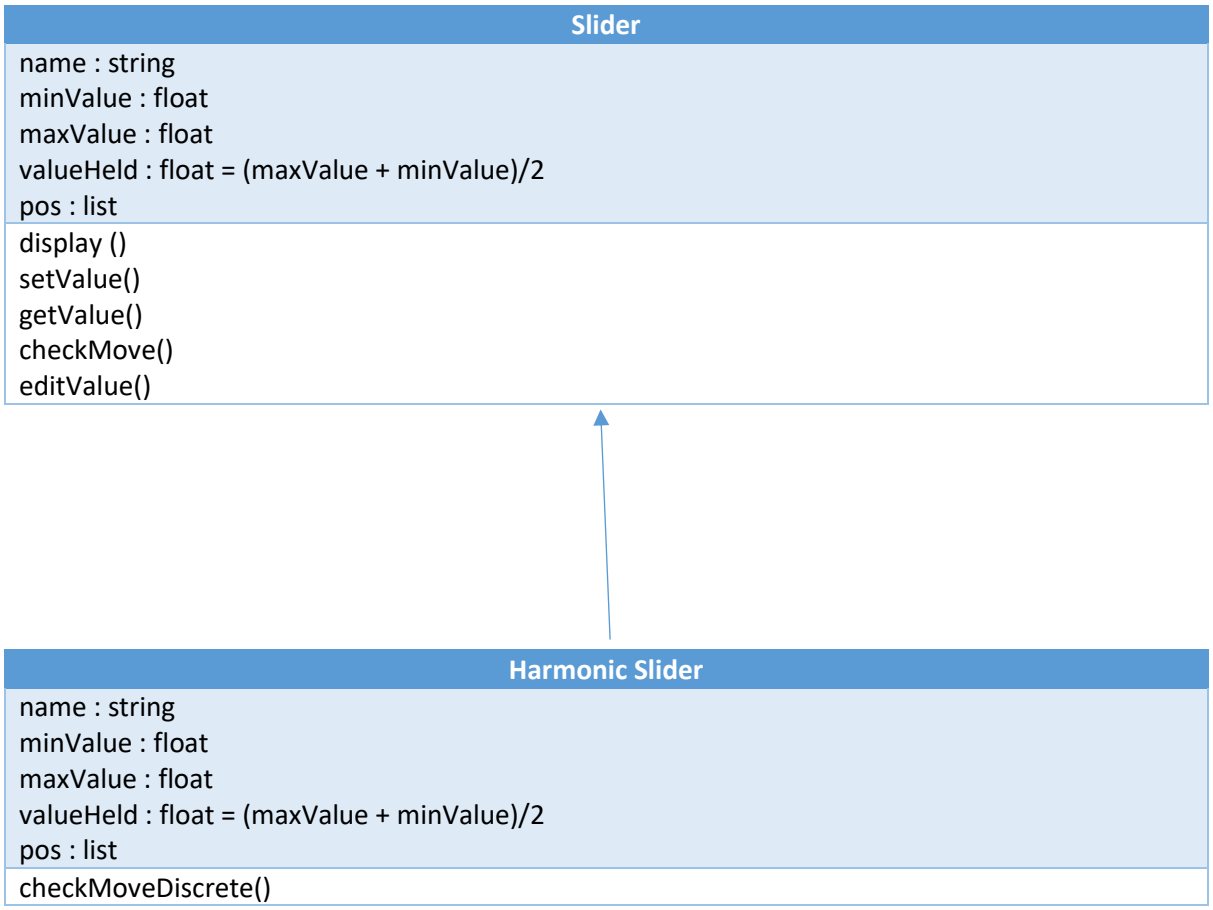
To test the new condition that checks if the line is clicked, I will click on points on the line and expect the pointer to move to that point and the value in the box to change to the relevant value.

### Discrete Slider – Class Diagram

After developing the first prototype which successfully displayed the stationary waves, I needed to create a new slider to display the abstract view. This needs a new slider that allows the user to choose a certain harmonic. This, unlike the previous sliders will only hold discrete values so must jump to the nearest whole number.

I could create a new standalone class for this but this would mean re-writing a lot of code already present in the Slider class. Instead, I am going to use inheritance to create a subclass for this harmonic slider.

This is the new class diagram:



## Discrete Slider – Algorithm Design

This is the pseudocode for the discrete value slider to adjust the number of the harmonic for the abstract display of the stationary waves:

```
class discSlider inherits Slider
      private spacing
      public procedure new(givenPos, givenLength, givenMinValue,
      givenMaxValue)
            super.new(givenPos,givenLength,givenMinValue,
givenMaxValue)
            spacing = length//(givenMaxValue-givenMinValue)
            //finds the distance between smaller lines
      endProcedure

      private procedure setValueHeld()
            //changes the value held to the one indicated by the
slider
            Super.setValueHeld()
            valueHeld = round(valueHeld)
            //rounds it to the nearest integer as it is discrete
            Super.adjustValue()
      endProcedure

      private procedure display()
            //displays the slider on the screen.
            super.display()
            display lines spaced out with spacing between them
      endProcedure

endClass
```

The constructor method in this uses the superclass's constructor method because many of the initial values have to be set to the same thing. I just added the attribute spacing so I know the distance in pixels between each discrete value.

The editValue method is exactly the same so is not included in the pseudocode again. I could have added extra validation to check that the value entered is exactly the same as one of the discrete values the slider can hold but instead, I kept it the same but will round it to the correct integer value in the setValueHeld method. This means that if the user enters erroneous data like 1.2, it will be rounded to the nearest integer (1).

The move method is exactly the same so is not included again. For this, I could have made it so the user had to drag the pointer to exactly one of the integer values but this would be hard to use because they would have to drag it to the exact pixel. Instead, I allowed the user to drag the pointer to anywhere they want on the line and then use the method setValueHeld to round it to the correct position.

### Discrete Slider – Validation required

The same validation as the normal slider is required for the discrete slider but it also needs to be an integer entered and not a float.

| Module | Validation Required | Response | Justification |
|---|---|---|---|
| **discSlider – editValue** | Type – value entered must be an integer | Box flashes red and returns to previous value | I could make it round to the nearest integer but I think this response makes it clear to the user that only integer values of the harmonic are accepted. |

### Discrete Slider - Test Data

The test will be run on a discrete slider with minimum value 1 and maximum value 5 because this is the range that will be used for the harmonic slider.

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **discSlider – editValue** | 2 | Normal | Pointer to move to the second vertical line on the slider. | Check that the pointer will move when a valid value is entered. |
| **discSlider - editValue** | 2.7 | Erroneous – Wrong Type | The box should flash red and return to its previous value. | This will check that the validation is working for the check for the correct type. |
| **discSlider - move** | Move the slider from left to right | Normal | Should jump from value to value. | This tests that the user can successfully adjust the value by moving the pointer. |

## Equipment – Mass Hook and Masses

This is being designed and developed now because the only variable that cannot be changed by a slider in this simulator is the tension in the string which needs to be adjusted by masses on a hook to fully replicate the lab experience.

I am designing and developing the hook and mass together because they would not be able to function independently: There is no point having a hook if you can't add masses and vice versa. They also need to be tested together.

### Class Diagram

As stated previously, I need to make the equipment able to be drag and dropped from a menu into its correct position. The equipment needed for RP 01 – Stationary Waves is the following:

- Signal generator,
- Vibration generator,
- Hook for weights
- 100g masses
- Wooden bridge

I will start by designing and developing the hook for the weights and the 100g masses themselves because this is integral to the solution as the user needs to add and remove masses to increase and decrease the tension.

I will use an object oriented approach for the 100g masses so that a class can be created for the masses and many mass objects can be instantiated from this, allowing the user to add multiple masses to the hook. I could have used a procedural approach but this would be challenging to create multiple masses.

I will also use an object oriented approach for the mass hook as this will give a more modular structure to my code as all code relating to the hook will be contained within the class.

This is the class diagram for the mass hook:

| **MassHook** |
| --- |
| Pos : list<br>numOfMasses : int |
| addMass()<br>removeMass()<br>move()<br>getTension()<br>display() |

This is the class diagram for the 100g masses:

| **Mass** |
| --- |
| pos : list |
| Display()<br>Move()<br>checkCorrectPos() |

## Attributes and Methods Used

*Mass Hook:*

| Identifier | Data Type/ structure/ method type | Description | Justification |
|---|---|---|---|
| **pos** | list | Holds the position of the centre of the hook. (xCoordinate, yCoordinate). | I have used a list because the hook is able to move so the position of the hook should be able to change. Therefore, a mutable data structure is required rather than an immutable one like a tuple. |
| **numOfMasses** | Integer | Holds the number of masses that have been placed on the hook including the one it starts with. | The program needs to know the number of masses on the hook so it can calculate the tension in the string and therefore display the stationary waves. |
| | | | |
| **addMass()** | Procedure | Adds a mass to the top of the stack. | I need a procedure for this rather than just saying numOfMasses increments because it needs to check if the hook is full. |
| **removeMass()** | Procedure | Removes a mass to the top of the stack. | Same as above but it needs to check if the hook is empty. |
| **move()** | Procedure | Allows the user to move the hook across the screen. | The hook needs to be moved if the user wants to setup the equipment themselves. |
| **getTension** | Function | Calculates the tension in the string represented by the mass hook. | This is needed so that the tension can be fetched and used for the dispReal and dispAbstract procedures. Instead of using this function, I could just get the number of masses and multiply it by 0.98 but this is confusing so I made it a public method. |
| **Display()** | Procedure | Displays the hook on the screen. | I could have included the code for this within the move() method but separated it to maintain modularity. |

*Mass*

| Identifier | Data Type/ structure/ method type | Description | Justification |
|---|---|---|---|
| **pos** | Tuple | Holds the position of the centre of the hook. (xCoordinate, yCoordinate). | I have used a list because the masses are able to move so the position of the masses should be able to change. Therefore, a mutable data structure is required rather than an immutable one like a tuple. |
| | | | |
| **move()** | Procedure | Allows the user to move the mass across the screen. | The mass needs to be able to be moved so it can be moved onto the hook. |
| **Display()** | Procedure | Displays the mass on the screen. | I could have included the code for this within the move() method but separated it to maintain modularity. |
| **checkCorrectPos()** | Procedure | Checks if the mass is placed over the mass hook and calls hook.addMass if it is. | This is needed to allow users to add masses to the hook. This is included within the masses class rather than the mass hook class because there are multiple masses and only one hook. This means that the mass only needs to check if it is touching the one hook rather than the hook checking if it is touching any of the many masses. |

## Validation Required

There is user input in this module so validation is required.

| Module | Validation Required | Response if not met | Justification |
|---|---|---|---|
| **Mass Hook and Masses** | Range – The object must be moved within the window | The objects stop at the limit of the window. | I could make an error message pop up but I think that it is neater and more usable if the program just doesn't allow it to be moved past the limits |
| **Mass Hook** | Range – a mass cannot be removed when there is only one mass on the hook. | Nothing changes. | Again, I could make an error message appear but it would be smoother without |
| **Mass Hook** | Range – a mass cannot be added to the hook when it is full. | The mass does not get added to the hook. | Same as above. |

## Algorithm Design

The following is the pseudocode for the mass hook class:

```
class MassHook
      private pos
      //list containing coordinates of the position of the mass hook
      Private numOfMasses
      // integer value of number of masses attached to the hook
      Private interact
      // Boolean value indicating if the hook should be shown
      Private autoSetup
      // Boolean value that is true if the hook should initially be
      in its correct position
      Private maxPlates
      //Holds an integer value of the maximum number of plates the
      hook can hold
      Private topOfStack
      // holds the y value for the top of the stack where masses can
      be added
      Private length
      // The length of the hook
      Private massesAdded
      // list containing the mass objects that are added – a stack

      public procedure new(givenAutoSetup, givenPos)
            pos = givenPos
            numOfMasses = 0
            autoSetup = givenAutoSetup
            maxPlates = 8
            length = 100
            topOfStack = pos[1] – length/2 + 5
      endProcedure


      private procedure move()
            // allows the hook to be drag and dropped
            if mouseClicked and clickedHook then
                  // allows the hook to be moved if clicked
                  moving = True
            else if NOT mouseClicked then
                  //stops the hook from being able to be moved if
                  mouse is not held down.
                  moving = False
            else if mouseRightClicked then
                  removeMass()
            else if mouseMoving and moving then
                  //changes the position of the hook if the mouse is
                  moving and held down
                  pos = positionOfMouse
            endif
      endProcedure
```

```
private procedure checkCorrectPos():
      // checks if hook is in correct position
      // and changes autoSetup to True if it is so it cannot be
removed
      if abs(pos[0]-HOOK_POS[0])<10 and abs(pos[1] -
      HOOK_POS[1])<10 then
            autoSetup = True
            pos = HOOK_POS


public Procedure addMass()
      // adds a mass to the top of the stack


      if numOfMasses < maxPlates then //checks if stack is full
            numOfMasses += 1
            newMass = new Mass((pos[0], topOfStack))
            topOfStack += 10
            massesAdded.append(newMass)

      else
            colour = RED
      endif
endProcedure


public Procedure removeMass()
      //removes a mass from the top of the stack
      if numOfMasses > 0 then
            numOfMasses -=1
            massesAdded.pop()
            topOfStack -=10
      else
            colour = RED
      endif
endProcedure

public Procedure display()
      //displays the mass hook and masses on it
      drawline((pos[0],pos[1]+length/2)(pos[0],pos[1]-
      length/2), color = colour)
      drawArc from top of line round ¾ of a circle
      if not autoSetup then
            move()
            checkCorrectPos()
      endif
      for mass in massesAdded
            mass.display()
      next i
endProcedure
```

```
      public procedure getTension()
            //returns the tension caused by the mass hook
            // 1 is added because it starts off with one mass added
            return (numOfMasses+1)*0.98
      endProcedure
endclass
```

This is the class that once instantiated, a mass hook can be displayed and masses can be added to it. From the main loop of the code, the display method will be called which would draw the hook onto the screen and if it is required to be setup, the move method will be called, which would allow the user to move the hook into the correct position. If the hook has been moved to the correct position, it will no longer be able to be moved.

Masses are removed from the hook (for now) by right clicking the mouse as shown in the pseudocode for the move() procedure. I could have made a separate method for this but I put it in as part of the move method because this is already dealing with user input. In the future, a new method will be created that opens up a menu that gives the user options on what they want to do with the hook but for now, this will do.

Masses can be added to the hook by dragging and dropping them onto the hook. This is why addMass() has been made a public procedure so that it can be accessed by the masses class. All other methods except for the constructor method have been made private to ensure encapsulation.

The following is the pseudocode for the Masses class. This will allow mass objects to be instantiated from it and allow them to be moved.

```
Class Mass
      Private pos
      Private width
      Private Height
      Private rectangle
      //holds the rectangle pygame object

      Public Procedure new(givenPos)
            Pos = givenPos
            Rectangle = new Rect(topLeft = (pos[0]-
            width/2,pos[1]+width/2), width = width, height = height)
      endProcedure

      private procedure move()
                  private procedure move()
            // allows the masses to be drag and dropped
            if mouseClicked and clickedHook then
                  // allows the hook to be moved if clicked
                  moving = True
            else if NOT mouseClicked then
                  //stops the hook from being able to be moved if
                  mouse is not held down.
                  moving = False
            else if mouseMoving and moving then
                  //changes the position of the hook if the mouse is
                  moving and held down
                  pos = positionOfMouse
            endif
      endProcedure


      private procedure checkCorrectPos()
            if mass collide with hook then
                  hook.addMass()
            endif
      endProcedure

      public procedure display()
            draw mass in correct position
            move()
            checkCorrectPos()
      endProcedure
endclass
```

I could have used inheritance from the Mass Hook class to create the move() method for the Mass class but I decide to make them separate due to slight differences.

## In-development Test Data

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **MassHook - move** | Move the hook around the screen | Normal | The hook should follow where the cursor of the mouse is. | I need to test that the move method is successfully allowing the hook to be moved around the screen |
| | Move the hook off the screen | Erroneous | The hook should stop at the edge of the window. | I need to test if the validation is working and the hook isn't allowed to be moved off of the screen. |
| **MassHook - removeMass** | Right click the hook with a mass on it | Normal | The hook should have one less mass on it than before | This is needed to test that masses can be successfully removed |
| | Right click the hook with no masses on it | Erroneous | The hook should be left unchanged. | This tests the validation that checks if there are any masses left before removing one. |
| **Mass - move** | This will be the same as the mass hook test data for the move method with the same expected output and justification. | | | |
| **Mass – checkCorrect Pos** | Mass moved over the hook when it is empty | Normal | A mass should be added to the hook. The mass changes colour from red to blue. | I need to test if masses are able to be added to the hook. |
| **Mass - checkCorrectPos** | Mass moved over the hook when it is full. | Erroneous | The mass should remain red and be able to be moved away from the hook again. | This tests if the validation that checks if the hook is full before adding a mass is working. |

## Equipment box

A box needs to be created where equipment can be dragged out of it. This is because I have now created masses and a hook but do not have a place for them to go and I do not have a way of instantiating new masses once one has been dragged out as multiple masses are needed.

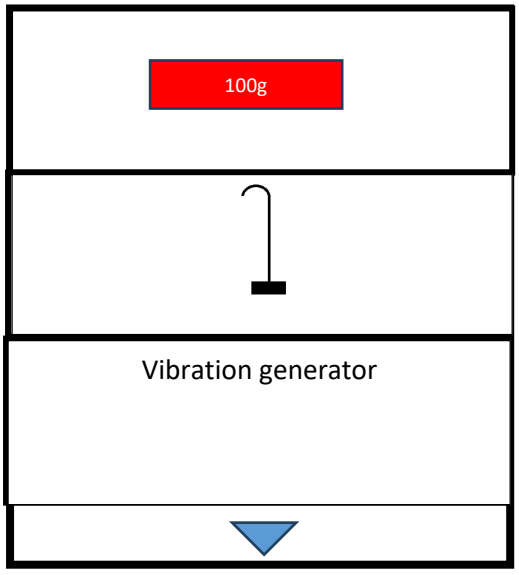
Within this module, instances of the relevant equipment classes need to be created and placed within the box.

I want this to look similar to the equipment box I analysed in the PhET:




As stated previously the arrows make it intuitive to find other equipment and names have been given with each piece of equipment to reduce confusion.

I would want mine to look something like this:




Up to 8 masses can be dragged out of this top box

1 hook dragged into position from 2nd box

Arrow pointing to next slide.

My initial thoughts was to create a Rect object for the outer box and have further Rect objects for the 3 inner boxes. I could then use the centre attribute of each of these boxes to get the coordinates to create a new object. I could do this as a procedure for each of the smaller boxes as each would need slightly different code because more masses can be generated than hooks for example.

However, I think it would make more sense to create a class for each smaller box and these would be controlled by another class for the entire equipment box. This would allow me to make the code more modular as all of the code for the equipment box will be contained within the equipment box class. It would also allow me to easily create many smaller boxes when extending the solution to having more types of equipment. The class diagram for these is here.

One problem that will need to be addressed is that all 8 masses cannot be instantiated at once in the same position because the way that section has been coded means that if one mass is clicked and dragged, all of the other masses would move with it. One way to solve this would be to create 8 different boxes, each for a different mass. This would be less intuitive to use than having them all in one box because the user will think that each mass would do different things. Another way of solving this problem would be to instantiate a new mass in that position if there is no other mass there.

This could be done by creating a list of all of the masses and check if the location of the last mass is equal to the centre of the box it is dragged from. If it is not, a new mass is instantiated in the centre and appended to the list.

In summary, two classes will be created. One will be for a smaller DragBox which holds one type of equipment and the other will be the whole EquipmentBox which holds multiple DragBoxes, each with its own item

In the future, as more practical activity simulators are created, inheritance will be used for the class EquipmentBox so the equipment types can be changed within it. However, for now one class will be created solely for the required practical 1 – stationary waves.

## Class Diagram

This is the class diagram for the DragBox class. I have decided to create a class diagram instead of relying solely on pseudocode to give me a visualisation of the methods and attributes of the class.

| DragBox |
| --- |
| Pos: tuple<br>itemType : class<br>items : list<br>width : integer<br>height : integer<br>maxItems : integer |
| checkMore() |

This is the class diagram for the EquipmentBox:

| EquipmentBox |
| --- |
| Pos: tuple<br>width : integer<br>height : integer<br>massesBox : DragBox object<br>massHookBox : DragBox object<br>boxes: list |
| Display()<br>checkInput() |

## Attributes and Methods Used

### *DragBox*

| Identifier | Data Type/ structure/ method type | Description | Justification |
| --- | --- | --- | --- |
| **pos** | tuple | Holds the position of the centre of the box. (xCoordinate, yCoordinate). | The position of the box should not change so a tuple has been used due to its immutable nature. |
| **itemType** | Class identifier | The class identifier of the object held within that box. | New objects like masses need to be instantiated if the user wants a new one so the class type must be passed in. I could make a separate DragBox class for each piece of equipment but to increase usability, I found that this was a better way of doing it. |
| **items** | list | A list of all of the objects instantiated from that DragBox | All of the equipment created from that DragBox needs to be displayed so within the display method, it will iterate through this list. A list is used because it is constantly added to when new objects are instantiated so it requires the dynamic nature of a list. |
| **Width,height** | integers | The dimensions of the box. | This is needed in order to create a Rect object and display the box. |
| **maxItems** | integer | The maximum number of objects that can be instantiated in that box. | This is needed as some equipment only requires one piece (e.g. the hook) while other equipment needs multiple pieces (e.g. the masses) |
| | | | |
| **checkMore()** | function | Checks if new objects need to be instantiated. | I will create a function that returns all of the equipment currently in use rather than updating an in-built list so it can be accessed from the EquipmentBox class to easily display the objects. |

## Algorithm Design

The following is the pseudocode for the smaller drag box which holds the Rect object of the box and a list of all of the items it holds as attributes and has a method which checks if a new piece of equipment should be instantiated.

```
class DragBox
      // a class for each individual box within the larger equipment
box
      private pos //position of centre
      private width
      private height
      private maxItems
      private items
      private box
      private itemtype

      public procedure new(givenPos,givenMaxItems, givenItemType)
            pos = givenPos
            itemType = givenItemType
            maxItems = givenMaxItems
            width = 100
            height = 30
            box = pygame.Rect(givenPos, width,height)
            items = []
            item = itemType
            items.append(new item)
      endfunction

      public function checkMore()
            //checks if more items need to be added and return a list
of
            //All present masses
            if items[-1].pos != pos AND items.length < maxItems then
                  items.append(new ItemType)
            endif

            return items
      endfunction
endClass
```

The checkMore function checks if the position of the last item in the list of the items is in the position where new items are created. The last item in the list is the most recent item to be created. At the start of the program, there is only one item in the list and it is in the original position and would be at the end of the list. If that item is moved, a new item is created in the original position and would be appended to the end of the list. Therefore, if I want new items to spawn in as soon as one is dragged away from the original position, I need to check if the last item in the list is in its original position.

```
class EquipmentBox
      private pos
      private width
      private height
      private boxes
      private boxes

      public procedure new()
            width = 100
            height = 100
            pos = EQUIPMENT_BOX_CENTRE
            massesBox = dragBox((pos[0], pos[1]+height/3), 8, Masses)
            massHookBox = dragBox((pos[0], pos[1]), 1, MassHook)
            //creates drag boxes for the masses and the mass hook

            boxes = [massesBox, massHookBox]
      endProcedure

      public procedure display()
            //displays each of the smaller boxes
            for box in boxes:
                  items = box.checkMore()
                  for item in items:
                        item.display()
                  next item
                  rectangle = box.getBox()
                  pygame.draw(rectangle)

            next box

      endProcedure

endClass
```

The display procedure iterates through the list of items in each box. This is why a list was needed in the DragBox class instead of creating new variables each time, so that it is easy to iterate through.

### Validation Required

There is no user input in this module so no validation is required.

### In Development Test Data

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **EquipmentBox** | Create an equipment box and drag masses out until no more are created. | Normal | 8 masses should be dragged out and then the box should be empty. | This tests that the creation of masses is limited by the maxItems attribute, that masses are able to be dragged out and able to be moved. |

## Main Menu

### Algorithm Design

I now need to design the pseudocode for the main menu which will look like the image I designed in Usability Features. There will be one procedure that is called to display this menu and within this procedure, sub procedures will be called to display each option.

```
Function realViewMenu(pos)
      //displays the box option for realistic view of stationary
waves

      height = 300
      width = 400

      Box = Rect(pos[0]-width/2, pos[1] - height/2, width,height)
      //creates a box object in desired location
      displayText("RP 01 Stationary Waves - Realistic View)

      displayImage(realView.png)

      If checkClicked(box) == True then
            dispRealView()
            return True
      else:
            return False
      endIf
endFunction

function abstractViewMenu(pos)
      //displays the box option for abstract view of stationary
waves

      height = 300
      width = 400

      Box = Rect(pos[0]-width/2, pos[1] - height/2, width,height)
      //creates a box object in desired location

      displayText("RP 01 Stationary Waves - Abstract View)

      displayImage(abstractView.png)

      If checkClicked(box) == True then
            dispAbstractView()
            return True
      else
            return False
      endIf
endFunction
```

```
function Menu(screen)
      //displays the main menu with two options
      leftCentre = (screenSize[0]/4,screenSize[1]/2)
      rightcentre = (3*screenSize[0]/4,screenSize[1]/2)

       if realViewMenu(leftCentre)     or
abstractViewMenu(rightCentre) then
            return True
      else
            return False
endFunction
```

Menu is called when the program starts up and it then calls the other functions to display the options for what simulator is to be run. Each of the functions return a value. This value is true if one of the boxes has been clicked so that the menu stops being shown and the simulator can be shown.

I could have made realViewMenu() and abstractViewMenu() into one function that takes in a parameter identifying which simulator will be shown. This is because both functions share similar code. However, I have decided not to do this because it would require a lot of if statements and it is neater as two separate functions.

I noticed that there are some local variables that are declared every time the function is called but will not change from frame to frame. I also noticed that the functions realViewMenu and abstractViewMenu display the menu as well as returning a value so could be split into two separate sub modules. This made me think that this module would be better suited to an object oriented approach where attributes like the box are kept and have different methods including one that displays the menu and one that checks that the box is clicked. I will now create the class diagrams for these and then design the pseudocode below.

Here is the pseudocode for this class:

```
class MenuOption
      //one of the options the user can click to go to that
simulator
      private height
      private width
      private box
      private text
      private image
      private pos
```

```
      public procedure new(givenPos, optionType)
            // optionType allows the box to change depending on what
            it //represents
            height = 300
            width = 400
            if optionType == "Abstract" then
                  load image of abstract view
                  generateText("RP 01 : Abstract View")
            else if optionType == "Realistic" then
                  load image of realistic view
                  generateText("RP 01 : Realistic View")
            endif
            pos = givenPos
      endProcedure

      public function checkOptionClicked(mouseMovement)
            //returns True if the option is clicked
            If mouseMovement == box clicked then
                  Return True
            Else
                  Return False
      endfunction

      public procedure display(screen)
            //draws the option to the screen
            Draw box
            Display image
            Display text
      endProcedure
endClass
```

This separates my code into 2 separate sub modules: one that checks if the box has been clicked so that the rest of the code knows to load up the simulator that has been clicked and another that displays the option on the screen. This gives my code more of a modular structure so it is easier to test and spot where errors occur.

There will also be a dispMenu procedure that will display everything in the correct position on the screen:

```
Procedure dispMenu(screen,leftoption,rightOption)
      //displays the main menu
      leftOption.display()
      rightOption.display()
      displayText("Physics Practical simulator")
endProcedure
```

I have decided to make this its own procedure instead of including it within the main menu code to make my code more modular so if something goes wrong with displaying the menu, I know it is a fault within this procedure.

## Class Diagram

Below is the class diagram to help visualise the MenuOption class.

| MenuOption |
| --- |
| Pos : tuple<br>Width : integer<br>Height : integer<br>Text : string |
| checkOptionClicked()<br>display() |

## Attributes and Methods Used

This table shows the attributes and methods in the class, along with a description of what they are/do.

| Identifier | Data Type/ structure/ method type | Description | Justification |
| --- | --- | --- | --- |
| **pos** | tuple | Holds the position of the centre of the option. (xCoordinate, yCoordinate). | The position of the box should not change so a tuple has been used due to its immutable nature. |
| **Width/height** | integers | The dimensions of the image to be displayed. | This is needed so a PyGame Rect object can be created. |
| **text** | string | The name of the option. | This is called text rather than name because it is the only text to be displayed and name can mean other things like the name of a variable. |
| | | | |
| **checkOptionClicked()** | Function | Checks if that option has been clicked. | This is a function because the main loop of code needs to know if an option has been selected and which option has so it must return a value to this loop.<br>It is called checkIconClicked rather than checkClicked so it does not get confused with the checkClicked function. |
| **Display()** | Procedure | Displays the option on the screen. | This is done separately from the checkOptionClicked() function to maintain modularity. |

## In-Development Test Data

For each of the following test data, it will be tested for both an object with optionType “Abstract” and an object with optionType “Realistic” so that I can be sure that it works for both and setting it to one of these values does not cause an error. To test the checkOptionClicked method, I will get the program to print “clicked” if it returns True. The display method will be tested by calling it and the option with a black border around it should appear on the screen. The dispMenu function will be tested after this to test that it successfully displays both options side by side.

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **MenuOption – checkOptionClicked.** | Click the option | Normal | “clicked” | I changed the program to print “clicked” if the option is clicked so I can see if this function alone works. |
| **MenuOption – checkOptionClicked.** | Click elsewhere on the screen | Erroneous | Nothing to be printed. | I also need to test that this function is not returning True very time the mouse is clicked but only when the option itself is clicked. |

## Home Icon

### Class Diagrams

After developing and designing the main menu, it makes sense to also make the home icon a class because it requires multiple methods to display it and also check that it has been clicked.

The home icon lends itself to an object oriented approach because it will have multiple instances (one on each of the simulators). It will also have 2 methods: checkClicked and display which do similar things to the MenuOption class.

| HomeIcon |
| --- |
| Height : int<br>Width : int<br>Box : PyGame Rect object |
| checkClicked()<br>display() |

### Attributes and Methods Used

| Identifier | Data Type/ structure/ method type | Description | Justification |
| --- | --- | --- | --- |
| **Height, width** | integers | Holds the dimensions of the icon that the image is adjusted to. | This is needed in order to re size the image designed under usability features into the proportions I want. I could have used a tuple with the first value being the width and the second the height but they are often accesses separately so I have made them separate variables. |
| **box** | Rect Object | Holds the Rect object of the box around the image. | Having a Rect object means that it is easy to check if the icon has been clicked and easy to overlay the image of the icon on the image. |

### Algorithm Design

This is the pseudocode for the home icon:

```
Function checkClicked(box,mousePosition)
      //returns true if the box is clicked by the mouse
      If mouseButtonDown AND mousePosition touches box then
            Return True
      Else
            Return False
      Endif
endfunction

class HomeIcon
      private width
      private height
      private box
      private image
```

```
    public procedure new(pos)
        width = 60
        height = 60
        image = load image
        box = Rec(pos, width, height)
    endProcedure

    public procedure checkIconClicked(mousePosition)
        //returns true if icon is clicked
        return checkClicked(box,mousePosition)
    endProcedure

    public procedure display()
        //draws the image on the screen
        drawImage()
    endProcedure
endClass
```

I created a new function checkClicked because this Home Icon class needs the same method as MenuOption. Therefore, instead of re-writing code, I created a new function that is called by both methods. I could have made the home icon inherit the MenuOption class but this is the only method that it is needed for so this is not necessary.

### In-Development Test Data

For testing, I will instantiate a HomeIcon object in the middle of the screen and call the checkIconClicked and display sub-routines within the main loop. I will then get the program to print "clicked" if checkIconClicked is True.

| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **HomeIcon - checkIconClicked** | Click the icon | Normal | "clicked" to be printed in the terminal. | It is important that his function works because the icon would have no functionality without it. I could test it by printing what checkIconClicked returns as this would better test the functionality of the function. However this would print a value every frame so the program may slow down and it may be hard to spot the one "True" among all of the "False"s. Therefore it will only print "clicked" when the value returned is True. |
| **HomeIcon – checkIconClicked** | Click elsewhere on the screen | Erroneous | Nothing to be printed in the terminal. | I could have just checked if clicking it functions correctly but I need to make sure that the function does not return True when somewhere other than the icon is clicked. |

## Second Prototype

I now have two separate modules – the abstract view and the realistic view of the first required practical activity – stationary waves. I now need to structure my code so that each of these modules are called at the appropriate time.

### Algorithm Design

This is my initial thoughts for this module in structured English:

```
Run menu first until an option is selected
Run the selected option until program is closed
```

This seems simple but I needed to get my thoughts down to understand what to do.

This is my pseudocode for the menu function.

It returns False if the program has been quit, 1 if option 1 has been clicked and 2 if option 2 has been clicked.

```
Function menu(screen,screenSize)
      //runs while menu is being displayed

      running = True
      leftOptionCentre =  (screenSize[0]/4,screenSize[1]/2)
      leftOption = MenuOption(leftOptionCentre, "Abstract")
      //instantiates an object for the left option

      rightOptionCentre =  (3*screenSize[0]/4,screenSize[1]/2)
      rightOption = MenuOption(rightOptionCentre, "Realistic")
      //instantiates an object for the right option

      while running == True
            events = getEvents()
            //gets the mouse events from the user

            clicked = checkOptions(events,leftOption,rightOption)
            If checkEnded(events) then
                  //checks if user has terminated the program
                  running = False
                  return False
            else if clicked then
                  //checks if an option has been selected
                  return clicked
            else
                  dispMenu(screen,leftOption,rightOption)
            endIf
      endWhile
endFunction
```

This function first instantiates the two options that the user can select from. This is done before the while loop because I only want to instantiate them once. The while loop is then run until an option is selected or the program is quit. While designing this code, I could have checked within the code which option has been clicked but to improve modularity, I have decided to make this its own function. If nothing has been clicked, the menu is displayed.

This is the pseudocode for the checkOptions function which returns 1 if option 1 is clicked and option 2 if option 2 is clicked:

```
Function checkOptions(events,leftOption,rightOption)
      //returns 1 if option 1 is clicked and 2 if option 2 is
clicked
      For i = 0 to events.length()
            If leftOption.checkOptionClicked(events[i]) then
                  Return 1
            Else if rightOption.checkOptionClicked(events[i]) then
                  Return 2
            Endif
      Next i
endfunction
```

As there are only 2 values that can be returned, I could have used a Boolean value to return but I decided not no because the menu function relies on a number being returned.

### *Realistic View*

The following is the pseudocode that puts the realistic view of required practical activity 1 together:

```
Function createRP1Real(screenSize):
      // instantiates and returns sliders, mass hook and title text
      fSlider = Slider(pos=(screenSize[0]/2,700), length =
      300,minValue = 0, maxValue = 100, name = "frequency (hz)")

      muSlider = Slider(pos = (screenSize[0]-160,700), length = 300,
      minValue = 0.0001, maxValue = 0.001, name = "mass per unit
      length")

      lSlider = Slider(pos = (160,700) , length = 300, minValue =
      0.5, maxValue = 2, name = "length (m)")

      eBox = EquipmentBox((screenSize[0]-100,150)

      homeIcon = HomeIcon((80,80))

      text = ("RP 01 – Realistic View")
      Return [mu,l,f], eBox, text, homeIcon
endfunction
```

```
procedure  displayRP1Real(screen,sliders, eBox, text, homeIcon)
      //displays the title and the sliders and the stationary wave
      For i = 0 to sliders.length():
            Sliders[i].display(screen)
      Next i

      eBox.display(screen)
      homeIcon.display(screen)
      T = eBox.getHook.getTension
      dispReal(T,mu,l,f,A = 50)
      Display text
Enprocedure

Function RP1Real(screen,screenSize)
      //the main code that is run when RP 1 realistic is displayed
      //returns False if quit and true if menu option clicked
      sliders, eBox, text, homeIcon = createRPReal(screenSize)

      running = True

      while running == True
            screen.fill(white)
            events = getEvents()
            //gets the mouse events from the user

            If checkEnded(events):
                  Running = False
                  Return False
            Else if homeIcon.checkIconClicked(events):
                  Return True

            Else
                  for i = 0 to sliders.length()
                        sliders[i].move(events)
                        sliders[i].checkInput(events)
                        //iterates through sliders and checks if
                        they've been moved
            Endif
            displayRP1Real(screen ,sliders, eBox, text, homeIcon)

      endWhile
endFunction
```

The function RP1Real is called when the realistic view of the stationary waves is chosen. It calls the createRP1Real function first which instantiates all of the objects needed in this simulator. I have done this as its own function to increase modularity of my code.

RP1Real then enters a while loop which repeats until the program has been quit or the home icon is clicked. It then allows the sliders to be adjusted. The function then calls the displayRP1Real procedure which displays everything to the screen. I have made this its own procedure so that I know that if there is something wrong with the display, this procedure is at fault.

### *Abstract View*

I need to create similar pseudocode to the one above but for the abstract view of the stationary waves.

I could create a new function that only works for the abstract view but because it would be very similar to the RP1Real function, so decided to modify the RP1Real function so it works for both. The abstract view versions of the createRP1Real and displayRP1Real would be quite different so unique submodules for these would have to be created.

```
Function createRP1Abstract (screenSize):
      // instantiates and returns sliders, mass hook and title text
      multiplierSlider = Slider(pos=(screenSize[0]/2,700), length =
      300,minValue = 0, maxValue = 100, name = "frequency (hz)")

      muSlider = Slider(pos = (screenSize[0]-160,700), length = 300,
      minValue = 0.0001, maxValue = 0.001, name = "mass per unit
      length")

      lSlider = Slider(pos = (160,700) , length = 300, minValue =
      0.5, maxValue = 2, name = "length (m)")

      harmonicSlider = discSlider((600,200),300,1,5, "Harmonic
      Number")

      eBox = EquipmentBox((screenSize[0]-100,150)

      homeIcon = HomeIcon((80,80))

      text = ("RP 01 – Abstract View")
      Return [mu,l,harmonic, multiplier], eBox, text, homeIcon
Endfunction

Procedure displayRP1Abstract(screen,sliders, eBox, text, homeIcon)
      //displays the title and sliders and the waves
      For i = 0 to sliders.length():
            Sliders[i].display(screen)
      Next i

      eBox.display(screen)
      homeIcon.display(screen)
      T = eBox.getHook.getTension()
      DisplayAbstract(T,mu,l,harmonic,A = 50, multiplier =
multiplier)
      Display text
endProcedure
```

These two sub modules may look like their realistic view counterparts but when written in python, everything will have to be displayed in different positions for the two different simulators so they require their own sub modules.

This pseudocode shows the design of the algorithm for the updated version of RP1 real which works for both the abstract and realistic view:

```
Function rp1(screen,screenSize, view)
      //the main code that is run when RP 1 realistic is displayed
      //returns False if quit and true if menu option clicked

      If view == "real" then
            sliders, eBox, text, homeIcon = createRPReal(screenSize)

            displayRP1Real(screen ,sliders, eBox, text, homeIcon)

      else if view == "abstract" then
            sliders, eBox, textSurface, homeIcon =
            createRP1Abstract(screenSize)

            displayRP1Abstract(screen, screenSize, sliders, eBox,
            textSurface, homeIcon)

      endIf
      //instantiates the sliders and boxes


      running = True

      while running == True
            screen.fill(white)
            events = getEvents()
            //gets the mouse events from the user

            If checkEnded(events):
                  running = False
                  return False
            Else if homeIcon.checkIconClicked(events):
                  Return True

            Else
                  for i = 0 to sliders.length()
                        sliders[i].move(events)
                        sliders[i].checkInput(events)
                        //iterates through sliders and checks if
                        they've been moved
            Endif
      endWhile
endFunction
```

This is almost identical to the function RP1Real except for the if statement at the beginning which checks which view is supposed to be shown and displays the appropriate one.

## Key Variables and Data Structures

| Module | Variable name | Data type | Description | Justification |
|---|---|---|---|---|
| **Menu, rp1** | running | Boolean | Holds whether the program is supposed to be running or not. | I could have used while True because the function will end when one of the items are returned but it is better practice to use a variable in case a value is not returned and it says in a never ending loop. |
| **Menu** | leftOption, rightOption | MenuOption object | Holds the object of one of the options that can be selected. | I could have used the names realistic and abstract as these are the two options that will be presented but to improve maintainability, I used left and right because the name of the simulators may change but there will always be one on the left and one on the right. |
| **createRP1Real, createRP1Abstract** | muSlider, fSlider, lSlider, multiplierSlider, harmonicSlider | Slider Objects | Holds the objects of sliders. | I could have used mu, f, l etc but this would cause confusion with the values that the sliders hold as this is what mu represents in the visuals module. Therefore, I added slider at the end to reduce confusion |
| **createRP1Real, createRP1Abstract, rp1** | eBox | EquipmentBox object | Holds the object of the equipment box. | I've called this eBox rather than equipment box so I do not get confused with this and the class name EquipmentBox. |
| **rp1** | sliders | list | Holds all of the sliders in the simulator displayed | Having all of the sliders in one list is useful as it makes it easy to iterate through, and display all of the sliders easily. |

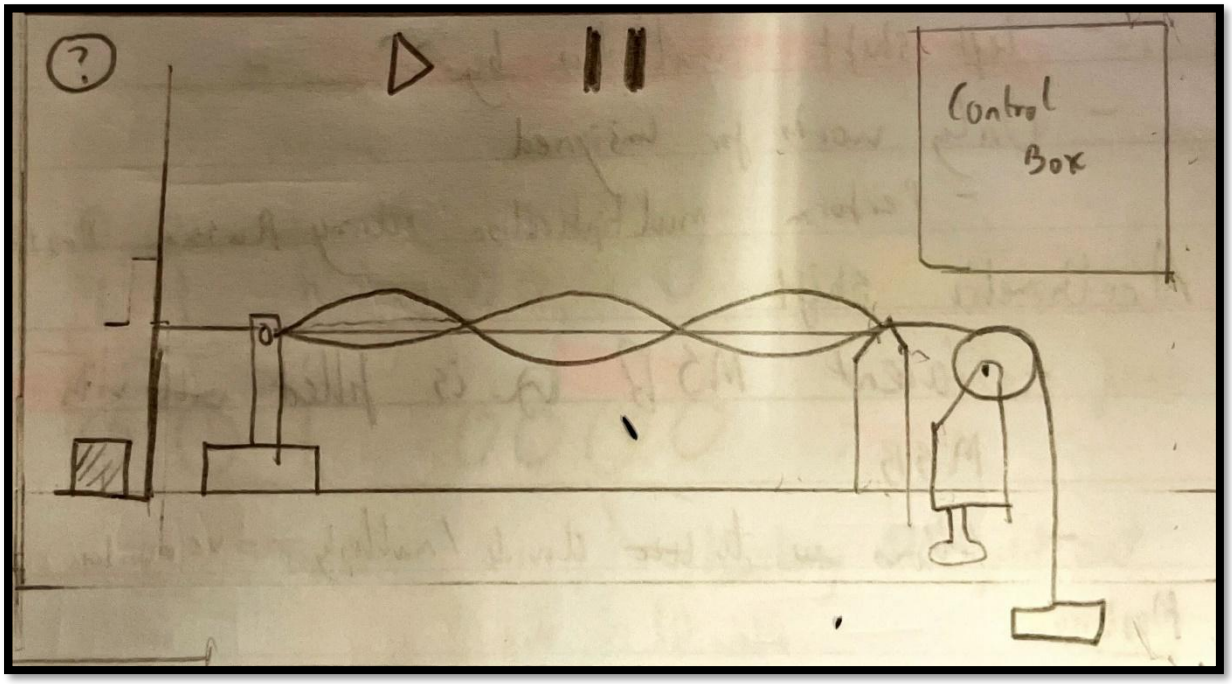
## In-Development Test Data

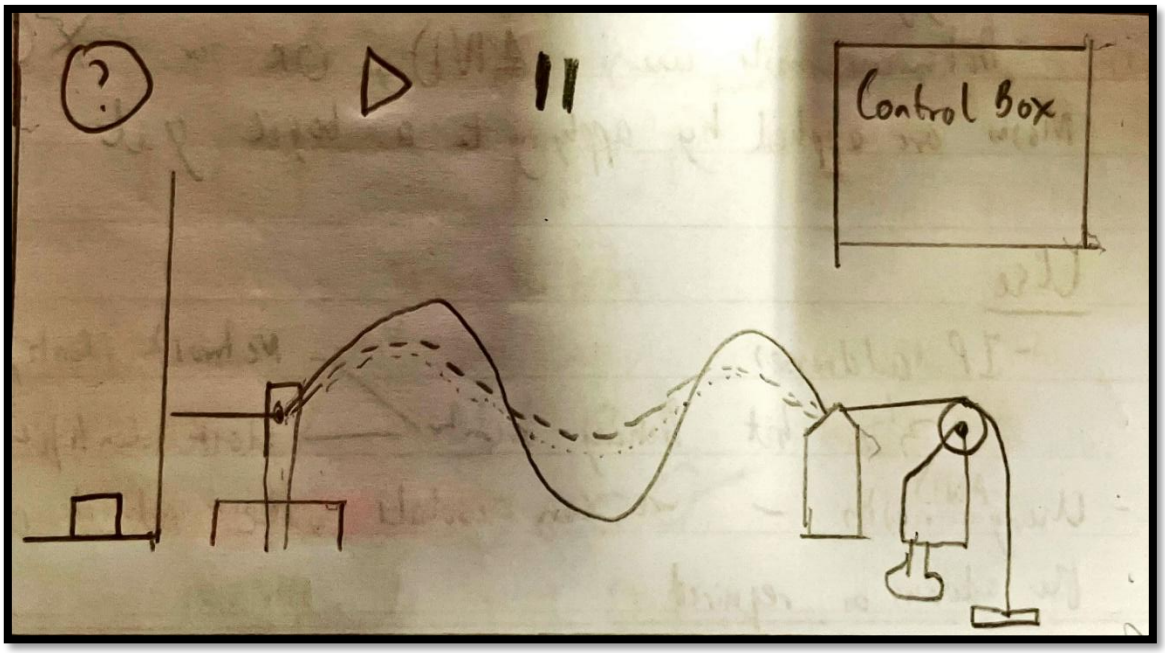
To test the menu function alone, I will get the program to print 1 if the left option is clicked and 2 if the right option is clicked.

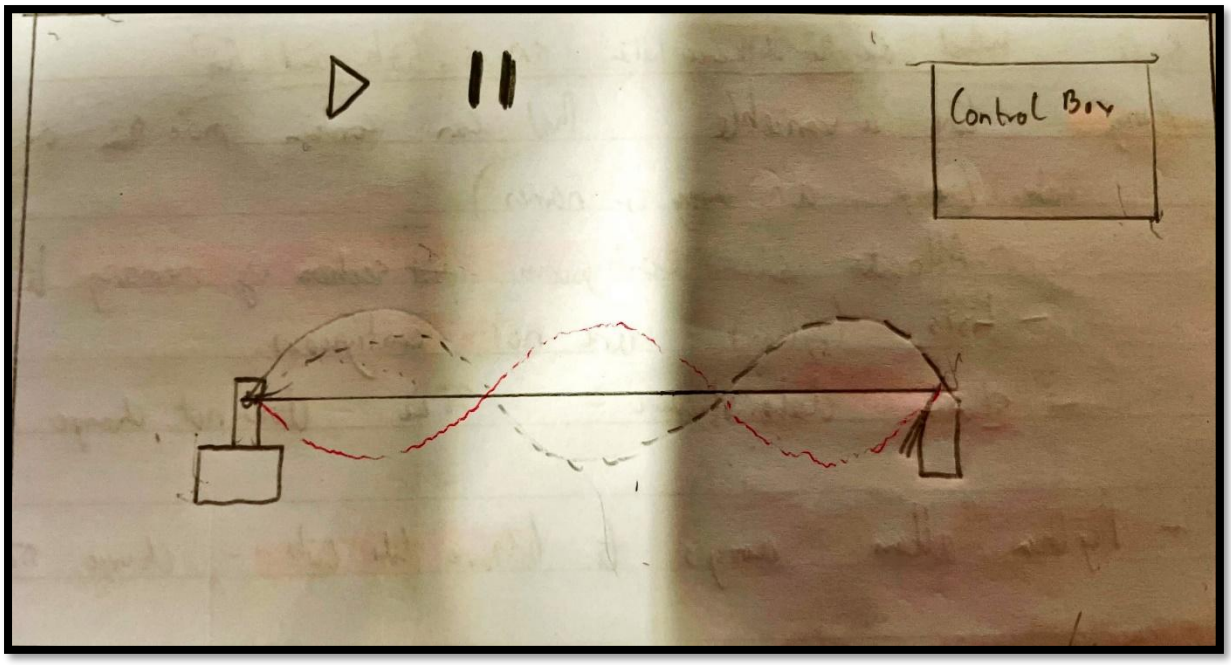
| Module | Test Data | Test Type | Expected Output | Justification |
|---|---|---|---|---|
| **menu** | Click left option<br><br>Click right option | Normal | 1 if left is clicked or 2 if right is clicked to be printed and program to end. | I need to check that the program is able to exit the loop once an option is clicked so in post development, it will stop displaying the menu and display a simulator. |
| **menu** | Click somewhere else on the screen | Erroneous | Program continues to run | I have to check that a random click on the screen does not cause it to think an option has been clicked. |
| **createRP1Real, createRP1Abstract** | screenSize = (1000,500) | Normal | Should return appropriate sliders etc. | I need to make sure that all objects are instantiated correctly. |
| **displayRP1Abstract, dispRP1Real** | N/A | Normal | Should display everything in the correct position but nothing able to move. | I need to make sure that the simulator can be displayed correctly before testing the inputs. |
| **Rp1** | View = “realistic”, view = “abstract” | Normal | The correct simulator displayed and sliders are able to be adjusted. | I need to test that this function displays the simulator as expected and the simulator can be used. |
| **Rp1** | X in top right is clicked | Normal | The simulator should stop running and False printed | I need to test that this function returns the correct value when the program is closed. |
| **Rp1** | Home Icon Clicked | normal | Simulator should stop running and True printed | I need to test that this function returns the correct value when the user attempts to return to the menu so I can then use this function in future development. |

## Usability features

### RP 01 Visuals






These three images are an abstraction of what the simulator screen will look like as I have removed the details like the contents of the control box and options to switch views. The full view will be shown in later iterations.

The top image is closest to what the practical activity looks like in real life. It shows the result of two sine waves interfering and producing a stationary wave that oscillates at a very high frequency. In real life, this results in a blur.

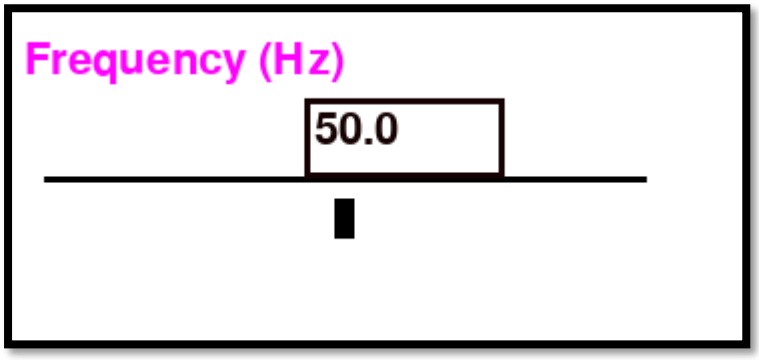
The second image shows what would be a slower moving wave along with the two progressive waves that for the stationary wave.

The third image shows the same as the second but removes some of the equipment.

I could use the second image as the display for my simulation but this could be confusing for the user because there is a mix of reality and the abstraction of the waves and the physics behind the waves. Therefore, I will primarily create a display like the first one for the realistic view of the first required practical activity. However, for the abstract view, I will show the option to display an interface like the third image because this will allow the user to see the physics behind the practical activity without the clutter of equipment you would have in real life.

## Sliders

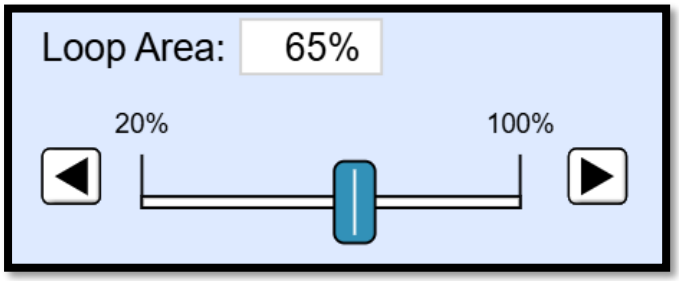
In my first prototype my slider looked like this:




After talking to Ian (one of my stakeholders), he said that it was not clear that this was a slider. As it is part of my success criteria created in the design section that all components must be intuitive to use, I must fix this and make it more usable.

Ian recommended that I place the slider in the middle of the line.

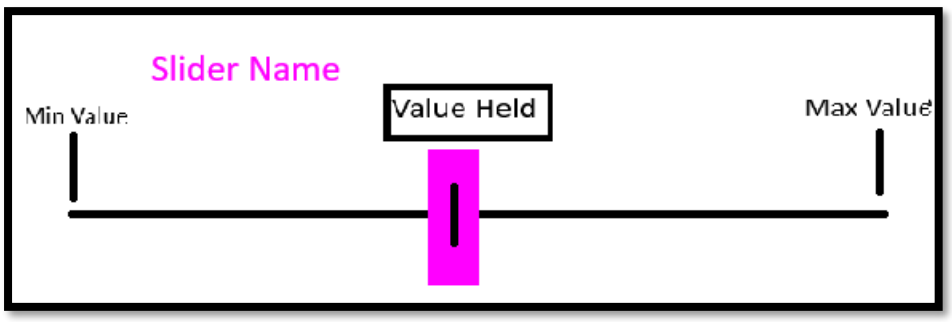
I looked back at the sliders I analysed from PhET. Here is one from their generator simulator (PhET Colorado, 2025):




As you can see the pointer is in the middle of the line and the end points are also shown on the screen.

Arrows allowing you to adjust the slider by small amounts are also shown. I will not develop this as I will not have time.

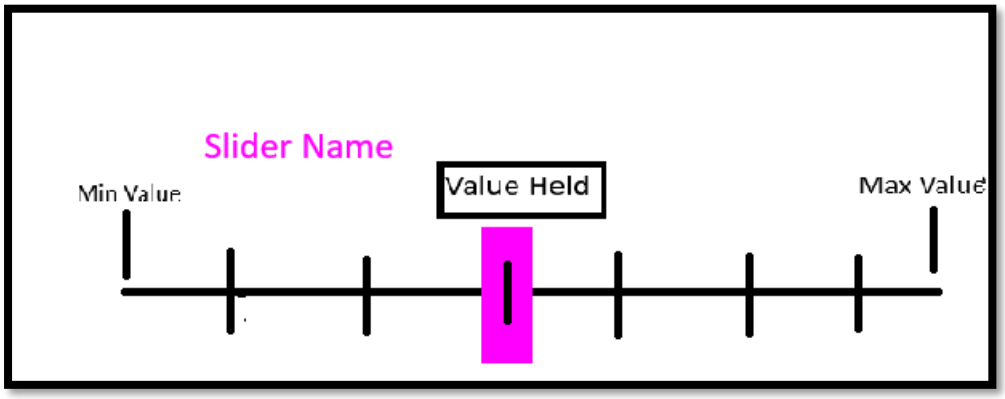
This is the design for my new slider:




I have changed the colour of the pointer from black to magenta so it pops out more and is easier to be seen. Blue would make it look more professional, but it does not pop as much as the magenta does. This colour matches the name of the slider which should also pop.

The pointer has been moved up onto the line so it is more intuitive. I could have left it like that but I instead decided to add a line through the centre of the pointer like they did in the PhET simulation. This line allows the user to see exactly where on the line the slider is at.

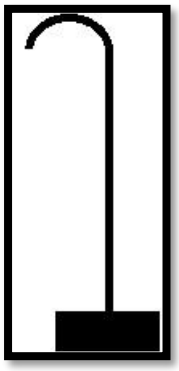
This is the design of the discrete slider which will be used to adjust the number of the harmonic for the abstract display of the stationary waves:




As you can see, I have added lines perpendicular to the main line that show where each of the positions where the slider can be. I could have added values to these lines but I instead chose not to as this would make the slider look cluttered and unappealing. It should still be clear what values each line holds because it will be displayed in the value held box and the minimum and maximum values are still shown.

### Mass Hook

This is what the mass hook will look like in my simulators that I drew using MS Paint:



This is based upon what mass hooks look like in real life and the ones used in these Physics Practical activities.

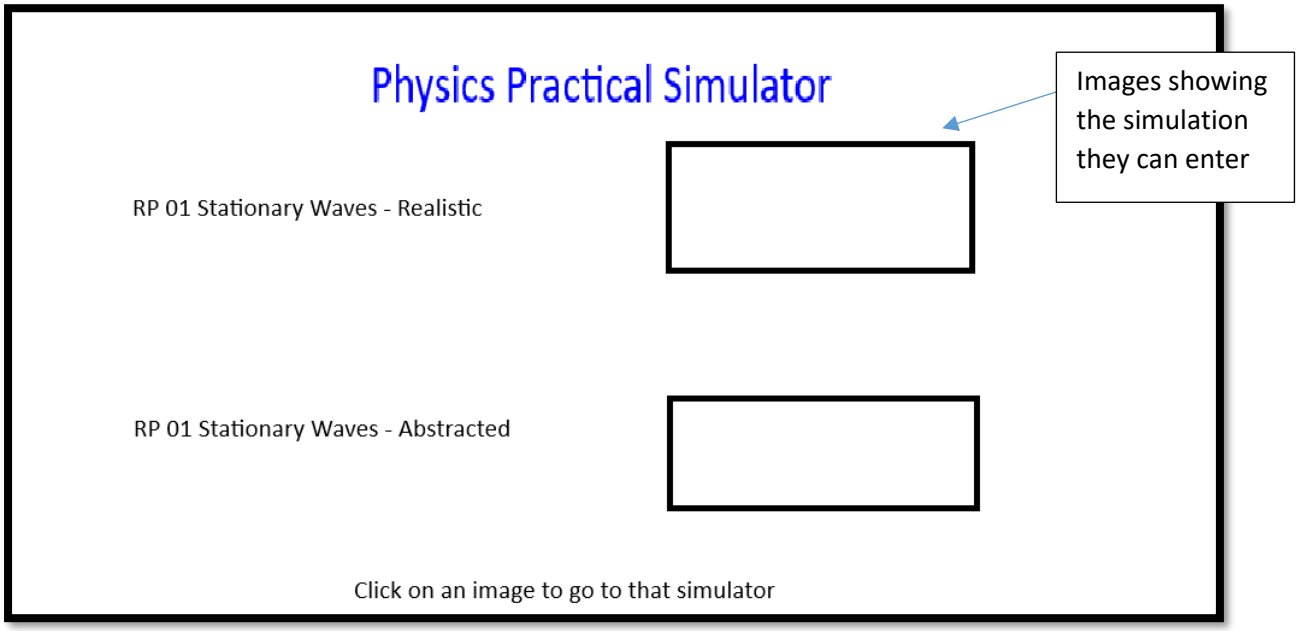
This is what the hook will look like without any masses placed upon it. The hook already has one mass on it which is the initial mass of the hook, replicating what it is like in real life. Therefore, without adding any masses to the hook, the method getTension() will return 0.96 as this is the normal weight of a hook with no masses attached.

Masses will appear as red rectangles which should be a good 2D replication of the cylinders used in real life.

I need to make a distinct difference between masses added to the hook and masses not added to the hook so I will make the masses turn blue once they are added to the hook. This will allow the user to know which masses affect the tension of the string.

## Main Menu

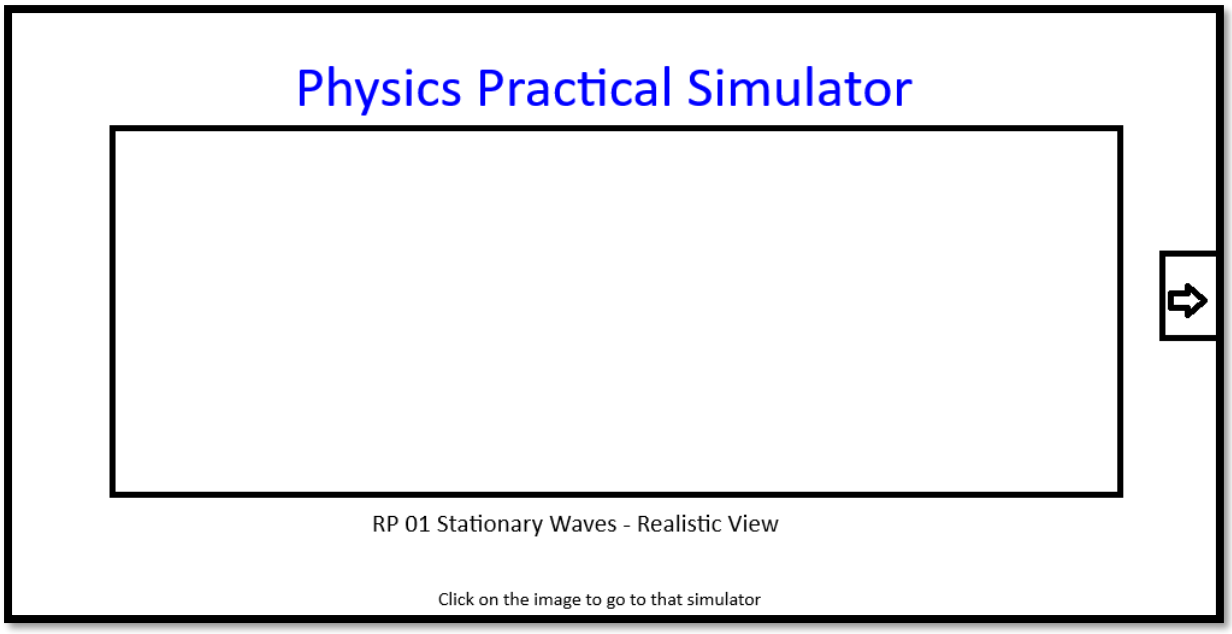
I need to design the display of the main menu so that it is easy for users to switch between views and in the future, practical activities.




This is one option for my main menu. It will display the title of each simulator along with a picture of the simulator so they understand what it means. For example, a first time user may not understand what is meant by realistic or abstracted but by looking at the image, they will understand.

I have made the title in a large font and in blue so that it stands out so the user knows what they are using.

Instructions at the bottom of the screen make the user understand what they are supposed to do.

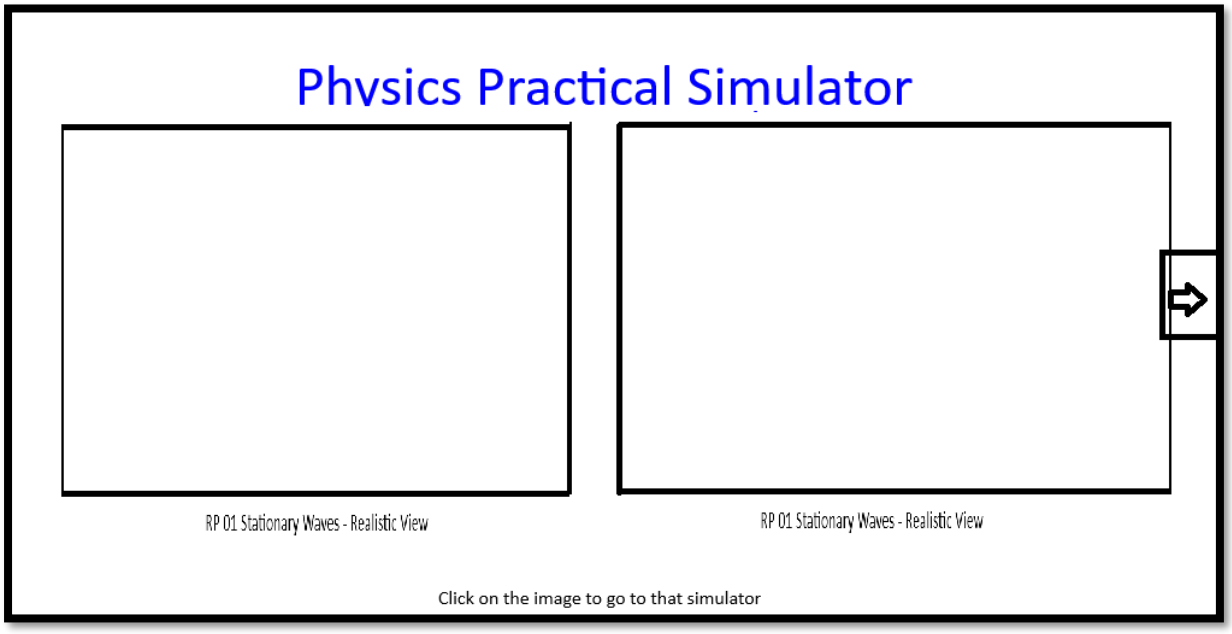



This is a second option for the main menu design. It has one large image in the centre and the name of the practical activity below it. Users can either click on the image to go to the simulator or click the arrow to scroll through to other simulators.

I prefer the second option for 2 reasons:

1. It is easier to add new simulators – a new page can be loaded after the arrow is clicked.
2. It is more obvious as to what each simulation is because the image is larger.

Therefore, this is the option that I will develop

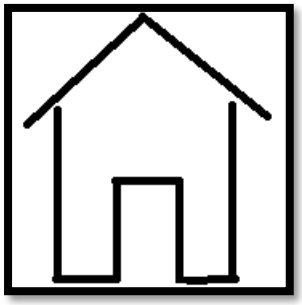



This is a third option which is very similar to the second but shows two at the same time.

I will use the third option when there are two simulators that are very similar to each other (e.g. RP 01 abstract and realistic view).

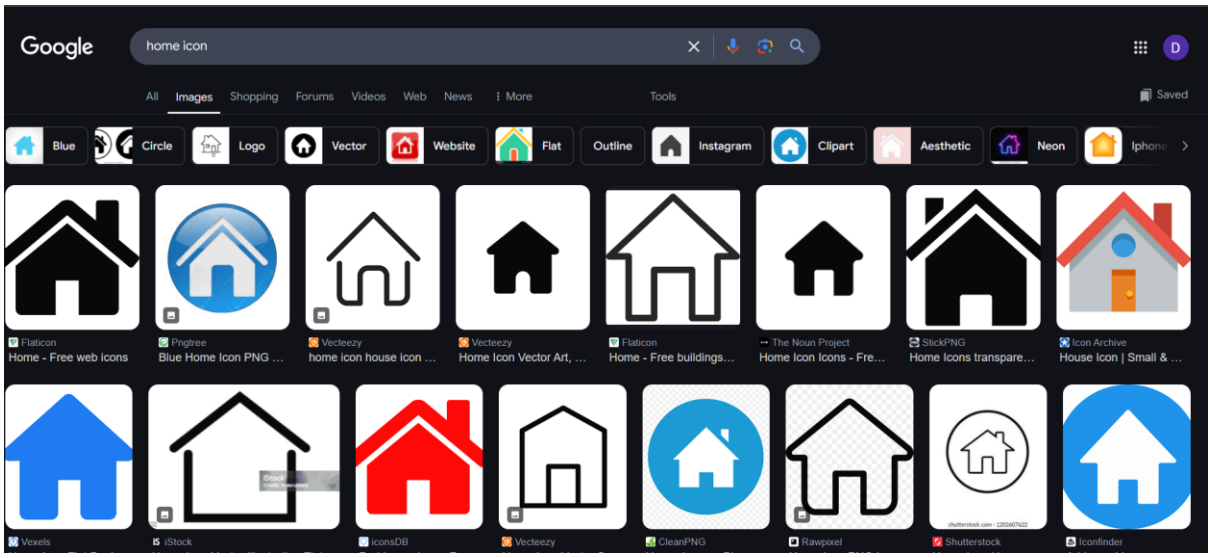
## Home Icon

I need to design a home icon that takes the user back to the main menu from each of the simulators.



This is my initial design for the home icon.

I wanted to make this icon familiar to users so that they can quickly and easily recognise what the home icon should look like. To do this I looked up “home icon” on google to see what users are familiar with.




After seeing this, I created my home icon in MS paint to recreate this design.

I will put the home icon in the top left of the screen because this is where users would normally find one in a language that reads left to right. I could create options to adjust for each language but my solution is targeted at those taking the British A-Level award so will be designed for a British user.

## Font

In this program, the text displayed will be in Calibri. This is because this is a commonly used font that users will be familiar with so it would give them a sense of familiarity. Calibri is also quite a professional font so it is suitable for an educational application.

In Analysis, people in my survey said on average that the solution should not be too informal. I believe that the Calibri font achieves this by being more professional than a font like comic sans.

## Stakeholder Feedback

I showed the above usability features to Dante (one of my stakeholders) to get a second opinion and see if it meets their expectations.

The following are quotes from Dante:

“I agree that the first drawing of the visuals would be well suited to the realistic view and the third well suited to the abstracted view.”

“The home icon is recognisable”

“The text should be bigger for the menu”

In development, I will increase the size of the text for the menu but will not re -design it in this section because it is obvious what bigger text would look like

Ian also gave his views on the new sliders:

“It is much clearer now”

## Post-Development Test Data

After development, I need to test that the program works smoothly as a whole and meets the success criteria set out in the Analysis section.

I will perform 1 run through of the program myself to test that everything I have developed all works as expected together and as smoothly as possible. I will also allow all three of my stakeholders to use the program so I can get a good idea of whether I have met my success criteria.

These are the steps that I will take myself when testing the solution to ensure it is functional and robust. I will screen record this test to evidence it.

| Step number | Step | Expected Output | Justification |
|---|---|---|---|
| **1** | Open program | Home menu to open within 10 seconds of opening. | This tests the success criterion that says the program should run within 10 seconds of opening. |
| **2** | Select Realistic View of RP1 | Realistic view of RP 1 to open within 5 seconds. | This tests the success criterion that sys each simulator should run within 5s on selecting. |
| **3** | Move frequency slider gradually from 0 to 100 and record frequencies where a stationary wave is displayed. | Given that length is 1m, mass per unit length is 0.00055kg/m and the tension is 0.98N as it will be on setup stationary waves should be displayed at: 21.1Hz (first harmonic), 42.2Hz (second harmonic), 63.3Hz (third harmonic), 84.4Hz (fourth harmonic).<br>When adjusting the slider, it should produce an instant change to the wave and amplitudes should decrease to seemingly 0 between waves. | I have already tested that the correct wave is displayed given a certain frequency in the visuals module but I am doing this test to make sure it also works with the slider and there are no errors or latency issues. |
| **4** | Add 8 masses to the hook then remove 7 and repeat the above step. | The masses should change colour as they are added to the hook and the first harmonic should be displayed when the frequency is set to 29.8Hz | I have previously tested that the masses are able to move but I need to test this again to make sure it still works when other code is also running with little latency. I also need to test that this correctly adjusts the wave displayed. |
| **5** | Add 5 more masses and repeat. | The first harmonic should be displayed when the frequency is set to 55.8Hz. This should have smaller amplitude than previous ones. | I could have only tested it for 2 different tension settings but decided to do it for 3 to ensure it was not just a coincidence. I will assume from this that it produces the correct |

| Step number | Step | Expected Output | Justification |
|---|---|---|---|
| | | | output for all other tensions. |
| **6** | Set the frequency to 55.8Hz and then adjust the mass per unit length using the slider, return it to 0.0055 and then adjust the length to 0.5m by entering the value into the text box. | The first harmonic should be displayed at first then as mass per unit length is adjusted, the amplitude should decrease until only a straight line is shown. It should then return to the first harmonic when moved back. Changing the length should also cause a straight line to be shown. | This tests that the other sliders are working and have an effect on the waves shown. It tests that the input box still works while the rest of the program is also running. |
| **7** | Click the Home Icon | The home menu should appear. | This tests that the home icon works as intended in the final solution. |
| **8** | Click RP1 abstracted view | The abstracted view of RP1 should open. | This tests that the button works as intended as previously it has not been tested when it is supposed to open a simulator. |
| **9** | Move the harmonic slider from 1 to 5. | The first to the fifth harmonics to be shown and the frequency box should show the relevant values as listed in step 3. | This tests that the discrete slider has the desired effect in the final solution as it has not been tested while the waves are being shown. |
| **10** | Click home icon | The program should return to the main menu. | This tests that the home icon works on the abstracted view simulator as well as the realistic view. |
| **11** | Click x in top right. | The program should close within 3 seconds. | The user must be able to close the program. |

I will also send a copy of the program to my stakeholders so that I know whether I have met usability requirements and so they can find errors that I may have overlooked. This will be part of the beta testing phase.

Mr Moran will look at both simulators and check that the Physics is correct. Mr Moran will be doing this as he is an experience Physics teacher and has performed this practical activity in real life multiple times.

Ian will open the abstracted view of RP 1, comment on the usability and say whether it has benefitted his understanding of the Physics behind the practical activity. Ian will do this because he is looking to revise the theory behind the practical activity for his upcoming exam.

Dante will open the realistic view of RP 1, comment on its usability and say whether it has helped him remember how to perform the practical activity. Dante will do this because it has been a long time since he performed the activity in class so needs to revise the setup and display of the practical activity.

# 3. Development

## Libraries Used

I could use a library like matplotlib to quickly create a graph for these stationary waves but this will appear with a grid and features that I do not want the user to see.

I decided instead to use PyGame because it is easier to move objects like sine waves than in other libraries like Tkinter. To help me learn PyGame I used a PowerPoint on mastering Python and PyGame by teach yourself python (teachyourselfpython.com, 2025).

To create the points to be displayed for the graph, I could have iterated through each x coordinate and calculated its corresponding y coordinate but instead I chose to use the library NumPy which allows me to execute calculations on large amounts of data at once. I learnt how to use NumPy by using their NumPy QuickStart guide .

## Naming Conventions

camelCase will be used for all variable names.

ALL_CAPS will be used for the names of constants.

PascalCase will be used for the names of classes.

camelCase will be used for the names of functions.

## Stationary Waves Results

### Calculate Frequency

```
import math
def calcFreq(T,mu,l):
    # calculates the frequency of the first harmonic
    h1Freq = (1/(2*l))*math.sqrt(T/mu)
    return h1Freq
```

This function returns the frequency of the first harmonic using the appropriate equation from the tension, mass per unit length and length of the string. This has been developed first because it is needed in order to display the waves. The python code is unchanged from the pseudocode that has been designed.

The library “math” has been used because it includes all of the basic maths function like square rooting a number I need. I could have used another maths library like NumPy but math is more lightweight than NumPy but it still contains all the functions I need.

### Calculate Speed

```
def calcSpeed(T,mu):
    # calculates the speed of the stationary wave
    return math.sqrt(T/mu)
```

### Testing

The following is the in development test data for this module created in the design section. For each row I have stated the output and whether that means the test passed or failed.

| Module tested | Data | Test data type | Expected Output | Output | Passed or failed |
|---|---|---|---|---|---|
| **calcFreq** | T = 1.96<br>Mu = 0.0007<br>l = 1 | Normal | 26.458 | 26.458 | Passed |
| **calcFreq** | T = 0.98<br>Mu = 0.001<br>l = 1.5 | Normal | 20.870 | 20.870 | Passed |
| **calcSpeed** | T = 2.94<br>Mu = 0.0002 | Normal | 121.2 | 121.2 | Passed |

### Review

This section was a first time success but this was expected as it was supposed to be an easy win to start off my development journey. I now need to create the visuals to display the appropriate waves.

## Stationary Waves Visuals:

### Closest Harmonic

This is the developed code for the closest harmonic function.

As stated in my design section, the closest harmonic to the frequency the user has set needs to be calculated as this is the wave that is shown on the screen. This is calculated by rounding the frequency divided by the frequency of the first harmonic to the nearest integer.

While developing, I noticed that if the frequency input is less than half of the frequency of the first harmonic, 0 will be returned. A stationary wave of harmonic 0 cannot be displayed as it does not exist. To fix this, I wrote the algorithm to check if harmDisp equals 0 and if it does, set it to 1.

```
def closestHarmonic(T,mu,l,f):
    # This function returns the number of the harmonic
    # closest to the frequency the user has set the frequency generator to
    # f = frequency input
    # T = tension in string
    # mu = mass per unit length of string
    # l = length of string
    harm1Freq = calcFreq(T,mu,l)
    harmDisp = round(f/harm1Freq)
    # rounds the frequency divided by the frequency of the 1st harmonic
    # to the nearest whole number which is the number of the harmonic to be displayed.
    if harmDisp == 0:
        harmDisp = 1
        # As the 0th harmonic does not exist, this is changed to the 1st.
    #endif
    return closestHarmonic
#endfunction
```

*Testing*

| Module tested | Data | Test data type | Expected Output | Output | Passed or failed |
|---|---|---|---|---|---|
| **closestHarmonic** | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 26 | Normal | 1 | 1 | Passed |
| | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 1 | Boundary | 1 | 1 | Passed |
| | T = 0.98<br>Mu = 0.0007<br>L = 1<br>f = 99 | Normal | 5 | 5 | Passed |

Since this has all passed first time, I can move onto the next part of the module.

## dispReal

This is my initial code for the dispReal procedure and it closely follows the pseudocode created in the design section. This procedure is supposed to draw 1 frame of the wave onto the screen.

```
def dispReal(T,mu,l,f,A,flip,screenSize):
    #This procedure draws the wave for the realistic view on the screen
    harmonic = closestHarmonic(T,mu,l,f) # finds number of harmonic to be
displayed
    firstHarmonic = calcFreq(T,mu,l) #finds frequency of first harmonic
    harmFreq = harmonic* firstHarmonic #finds frequency of harmonic to be
shown
    b = 0.1 #factor that reduces amplitude if f is high
    x = np.arange(1,screenSize[0]+1)
    #creates a NumPy array where each element is the x coordinate

    y =
np.round((flip*A*np.sin(harmFreq*x*np.pi/screenSize[0]))/np.sqrt((f**2 -
harmFreq**2)**2+(b*f)**2))
    #calculates the y coordinates
    coordinates = list(zip(x,y))
    pygame.draw.lines(surface, YELLOW,False,coordinates,width = 3 )
    #draws the line to the screen

#endProcedure
```

```
y = np.round((flip*A*np.sin(harmFreq*x*np.pi/screenSize[0]))/np.sqrt((f**2
- harmFreq**2)**2+(b*f)**2))
```

This line is the most important part of the above code. Using the NumPy array of numbers from 1 to the length of the screen (x), it calculates the corresponding y coordinates. This was explained in the Design section. This uses a NumPy array because NumPy uses concurrency to perform the same calculations on a large array of data efficiently

This is the dampening equation designed earlier:

$$\frac{\text{flip}}{\sqrt{(f^2 - \text{harmFreq}^2)^2 + (bf)^2}} * A * (\sin(\text{harmFreq} * x * \pi))$$

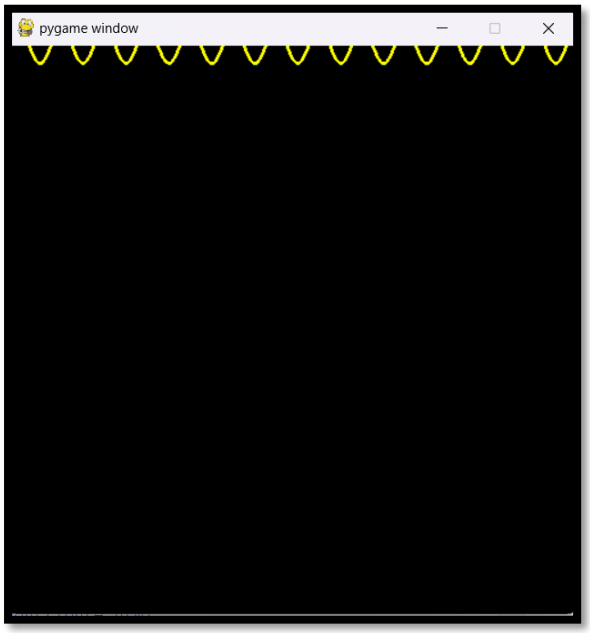
The only difference between this and the developed line is that in the code, the variables inside the sin function are divided by the width of the screen. This means that the length of a wave at the first harmonic will take up the whole screen instead of taking up 1 pixel.

I could have used the number 500 for the width of the screen as this is the width of the screen I was testing with but to improve flexibility, I used the variable "`screenSize`" as this would allow me to apply the same code to different screen sizes and allow me to allow the user to adjust the screen size.
As stated in design, the flip variable switches between 1 and -1 each frame to continually flip the sine wave to give an illusion of a stationary wave.

*Testing*

| Module tested | Data | Test data type | Expected Output |
|---|---|---|---|
| dispReal | T = 1.96<br>Mu = 0.0007<br>L = 1<br>f = 26.5, 53, 79.5<br>A = 50<br>screenSize = (500,500) | Normal | A graph showing the first harmonic stationary wave with the wave flipping up and down for f = 26.5 in the centre of the screen. It should then show the 2nd harmonic when f = 53 and the 3rd when f = 79.5 |

When tested with T = 1.96, mu = 0.0007, l = 1, f = 26.5, A = 50 and screenSize = (500,500), this was the output:




It would flicker between this and a reflection of the above wave .

This does not meet the expected output so has failed the test as it is not in the centre of the screen and does not show the first harmonic but it shows the 25th.

### *Remedial Action*

To centre the graph, I added the screen height divided by 2 to all y values.

After counting the number of peaks and realising that it is close to the frequency, I noticed that the x values should be multiplied by the number of the harmonic rather than "harmFreq"

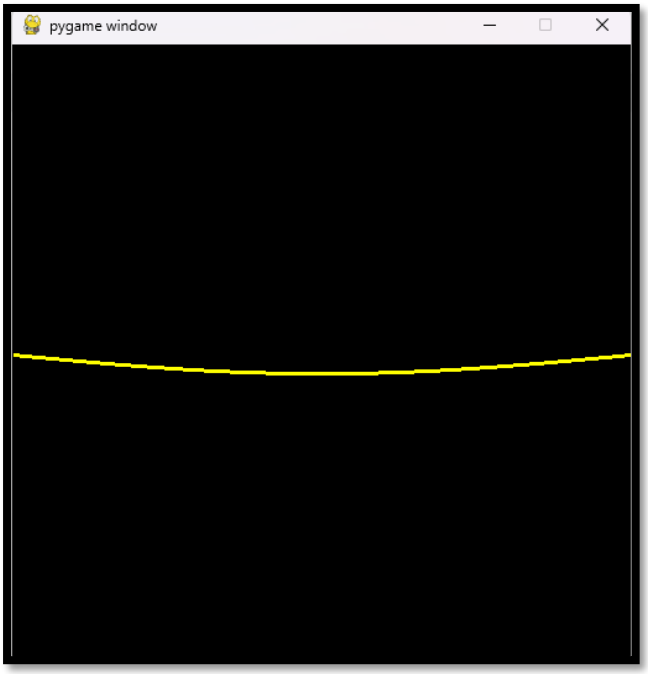
This is the updated code with corrections in yellow (text copied from Notepad++):

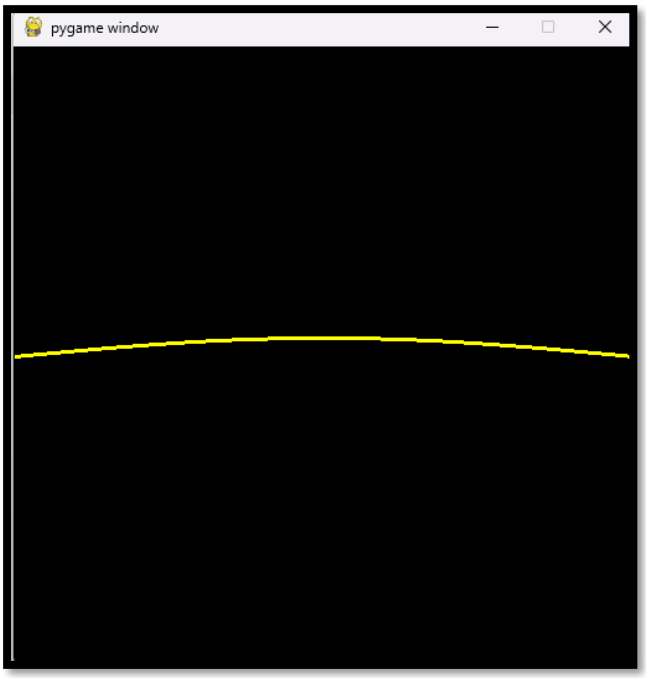
```
def dispReal(T,mu,l,f,A,flip,screenSize):
    #This function returns a list containing coordinates
    # for the points on the realistic wave to be displayed
    harmonic = closestHarmonic(T,mu,l,f) # finds number of harmonic to be
displayed
    firstHarmonic = calcFreq(T,mu,l) #finds frequency of first harmonic
    harmFreq = harmonic* firstHarmonic #finds frequency of harmonic to be
shown
    b = 0.1 #factor that reduces amplitude if f is high
    x = np.arange(1,screenSize[0]+1)
    #creates a numpy array where each element is the x coordinate

    y =
np.round((flip*A*np.sin(harmonic*x*np.pi/screenSize[0]))/np.sqrt((f**2 -
harmFreq**2)**2+(b*f)**2))+(screenSize[1]//2)
    #calculates the y coordinates
    coordinates = list(zip(x,y))
    pygame.draw.lines(surface, YELLOW,False, coordinates,width = 3 )
    #draws the line to the screen

#endFunction
```

When tested with the same data as above the output alternates between the two images below.



This is the expected output for this frequency. It also produces the expected output for the 2nd, 3rd, 4th and 5th harmonic frequencies.

It also produced the expected output for the other test data designed in the design section.

However, when you change the length of the string, it does not change the length of the string on the screen. Additionally, after talking to my stakeholder Ian, he agrees that it still does not look realistic when it is flickering between these 2 different waves.

After developing dispAbstract, I found that the stationary wave formed in dispAbstract at full speed was much more realistic and can be used solely (without the 2 progressive waves). I did not do this at first because I thought that it would be too slow for the position of the wave to be calculated every time but after setting dispAbtract to real speed, I realised that it is possible.

```
def dispReal(T,mu,l,f,A,screenSize):
    #displays the resultant stationary wave in real time

    SCREEN_LENGTH = 2 #metres  How many metres the screen width represents

    metresPerSecond = calcSpeed(T,mu)
    v = metresPerSecond * screenSize[1] * SCREEN_LENGTH# calculates speed
in pixels/second

    pixelL = (l * screenSize[0])/SCREEN_LENGTH #finds length of string in
pixels
    start = screenSize[0]/5
    end = screenSize[0]/5 + pixelL
    # graph is left adjusted


    heightRatio = screenSize[1] / 500  #multiplier so amplitude is adjusted
with screen height
    ratioL = l/SCREEN_LENGTH  #makes the string the correct length


    firstHarmonic = calcFreq(T,mu,l) #finds frequency of first harmonic
    harmonic = closestHarmonic(T,mu,l,f) # finds number of harmonict to be
displayed
    harmFreq = harmonic* firstHarmonic #finds frequency of harmonic to be
shown

    x = np.arange(1,pixelL) # creates np array with all x values

    t = time.time()
    b = 0.1

    l2rWaveY = np.round(heightRatio*A*np.sin((harmonic*np.pi*(x)-
v*t)/(screenSize[0]*ratioL))/(np.sqrt((f**2 - harmFreq**2)**2+(b*f)**2))
+(screenSize[1]/2))
    # Calculates y values for wave moving left to right
    r2lwaveY = np.round(heightRatio*-A*np.sin((harmonic*np.pi*(-x)-
v*t)/(screenSize[0]*ratioL))/(np.sqrt((f**2 - harmFreq**2)**2+(b*f)**2))
+(screenSize[1]/2))                                    # np.round(-
A*np.sin((harmonic*np.pi*(-x)-v*t)/screenSize[1]) + screenSize[0]/2)
    # Calculates y values for wave moving right to left
    resultantY = l2rWaveY + r2lwaveY - screenSize[1]//2
    # Calculates y values for resultant stationary wave
```

```
    x = np.arange(start,end+1)

    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph


    pygame.draw.aalines(surface, GREEN, False, resultant)
    #draws lines on screen
#endFunction
```

Unlike dispAbstract, I have decided to start the graph from the left hand side rather than centre the graph. This is because in the realistic view there will be stationary equipment on the left hand side that cannot move but there is no equipment in dispAbstract.

The other difference between this and dispAbstract is that this divides the y values by `np.sqrt((f**2 - harmFreq**2)**2+(b*f)**2))` because for dispReal, the frequency selected can be between harmonics and this `(f**2 - harmFreq**2)` reduces the amplitude of the wave if it is not at the harmonic frequency. `+(b*f)**2))` reduces the amplitude of the wave as the frequency gets larger. This is a realistic effect that does not need to be included in the abstracted view.

### *Testing*

I have not provided screenshots for this testing phase as a still looks very similar to the previous screenshot but when played as a video, it looks much more realistic.

| Module Name | Test Data | Test Type | Expected Output | Passed or Failed |
|---|---|---|---|---|
| **dispReal** | T = 1.96<br>Mu = 0.0007<br>L = 1<br>f = 26.5, 53, 79.5<br>A = 50<br>screenSize = (500,500) | Normal | A graph showing the first harmonic stationary wave with the wave flipping up and down for f = 26.5. It should then show the 2nd harmonic when f = 53 and the 3rd when f = 79.5 | Passed |
| **dispReal** | T = 0.98<br>Mu = 0.0007<br>L = 1<br>f = 18.7<br>A = 50<br>screenSize = (500,500) | Boundary | A graph showing the first harmonic, constantly flipping. It should have a greater amplitude than the one when T = 1.96 | Passed |
| **dispReal** | T = 1.96<br>Mu = 0.0007<br>l = 1<br>f = 35<br>A = 50<br>screenSize= (500,500) | Normal | A sine wave with little amplitude or none at all constantly flipping. | Passed |

### *Review*

I have created a way of displaying the realistic view of the stationary waves. This is a vital part of the solution so it is good that this is completed early on.

If I were to do this again, I would spend more time designing the module. This is because I spent unnecessary time developing the waves that flip up and down before swapping it with another algorithm. If I had designed it well, I would have realised that it would have worked at full speed if a stationary wave was formed by the addition of two progressive waves rather than one sine wave being flipped up and down.

I now need to create an abstracted view for this same practical activity.

### dispAbstract

This is my initial code which is based largely upon the pseudocode created in the design section:

```
def dispAbtract(T,mu,l,harmonic,A,screenSize,surface):
    # Displays 3 waves. One resultant stationary  wave and two progressive
waves
    # Takes harmonic as parameter because frequency has to be at a harmonic
    v = calcSpeed(T,mu)
    f = calcFreq(T,mu,l) * harmonic

    x = np.arange(1,screenSize[0]+1) # creates np array with all x values
    t = time.time()
    #gets time so moves by same amount each second

    l2rWaveY = np.round(A*np.sin(f*x*np.pi-v*t)) + screenSize[1]//2
    # Calculates y values for wave moving left to right
    r2lwaveY = np.round(-A*np.sin(f*np.pi*(-x)-v*t)) + screenSize[1]//2
    # Calculates y values for wave moving right to left

    resultantY = l2rWaveY + r2lwaveY
    # Calculates y values for resulatnt stationary wave

    l2rWave = list(zip(x,l2rWaveY))
    r2lWave = list(zip(x,r2lwaveY))
    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph

    pygame.draw.aalines(surface, RED, False, l2rWave)
    pygame.draw.aalines(surface, BLUE, False, r2lWave)
    pygame.draw.aalines(surface, GREEN, False, resultant)
    #draws lines on screen

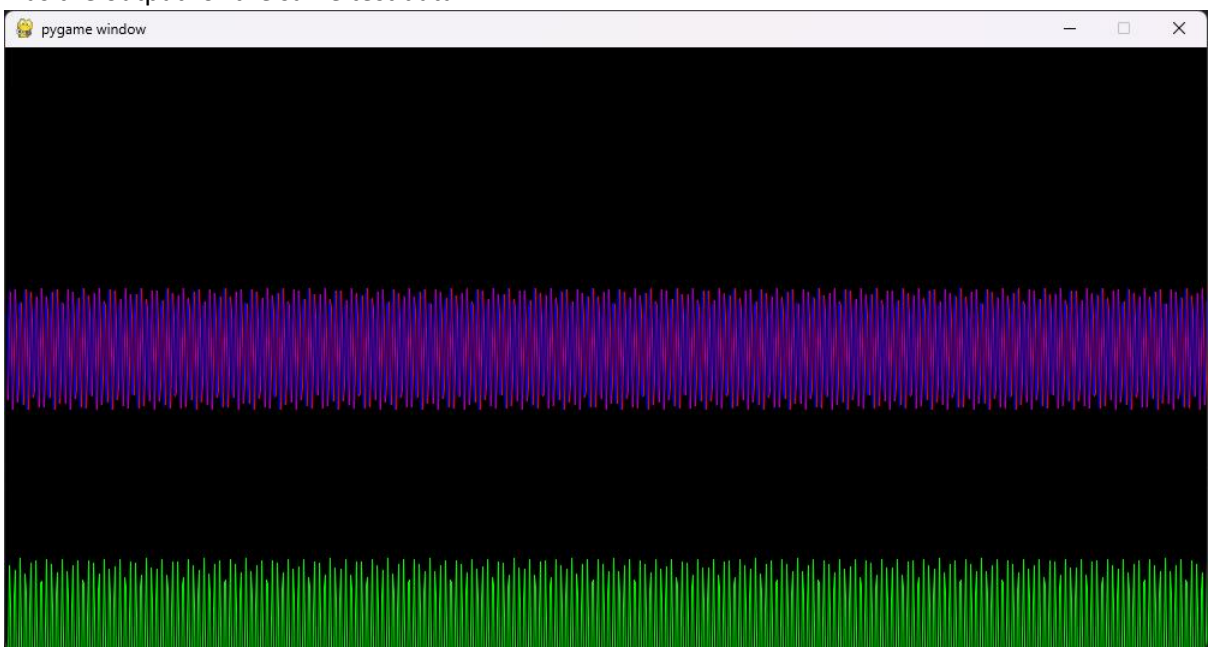
    surface.fill(BLACK)
    #resets screen
#endProcedure
```

#### *Testing*

This resulted in a black screen when tested with values: T = 1.96, mu = 0.007, l = 1, harmonic = 1, A = 50, screenSize = (1000,500). This is because the surface was reset to black at the end of the procedure so would clear anything else on the screen.

### *Remedial Action*

After removing that line and instead adding it to the main loop before this procedure is called, this was the output for the same test data:




You can now see the waves but the frequency is far too high and the green resultant wave should be in the middle not at the bottom of the screen.

The reason why the green stationary wave is at the bottom of the screen rather than the top is because as resultantY is the the sum of l2rWaveY and r2lWaveY, half the screen height is added twice, resulting in it being at the bottom of the screen. To correct this, I will subtract half the screen height from resultantY so it is in the centre. I did this because this is the simplest way of solving the problem.

As found in the dispReal module, there are many waves appearing because x is being multiplied by f rather than the number of the harmonic and x needs to be divided by the screen width because one phase of the wave is currently only one pixel wide and should be the same width as the screen.

This made me notice that the speed of the waves should also be suitable for the screen width in pixels because it is currently being calculated in metres per second but should be calculated in pixels per second.

This is the code with the stated corrections:

```
def dispAbtract(T,mu,l,harmonic,A,screenSize,surface,multiplier):
    # Displays 3 waves. One resultant sationary  wave and two progressive
waves
    # Takes harmonic as parameter because frequency has to be at a harmonic
    metresPerSecond = calcSpeed(T,mu)
    v = metresPerSecond * screenSize[1] * multiplier
    # This assumes the width of the screen is 1m - need scale
    # It adjusts the speed to pixels per second

    x = np.arange(1,screenSize[0]+1) # creates np array with all x values
    t = time.time()

    l2rWaveY = np.round(A*np.sin((harmonic*x*np.pi-v*t)/screenSize[0]) +
screenSize[1]/2)
    # Calculates y values for wave moving left to right
    r2lwaveY = np.round(-A*np.sin((harmonic*np.pi*(-x)-v*t)/screenSize[0])
+ screenSize[1]/2)
    # Calculates y values for wave moving right to left
    resultantY = l2rWaveY + r2lwaveY - screenSize[1]//2
    # Calculates y values for resultant stationary wave

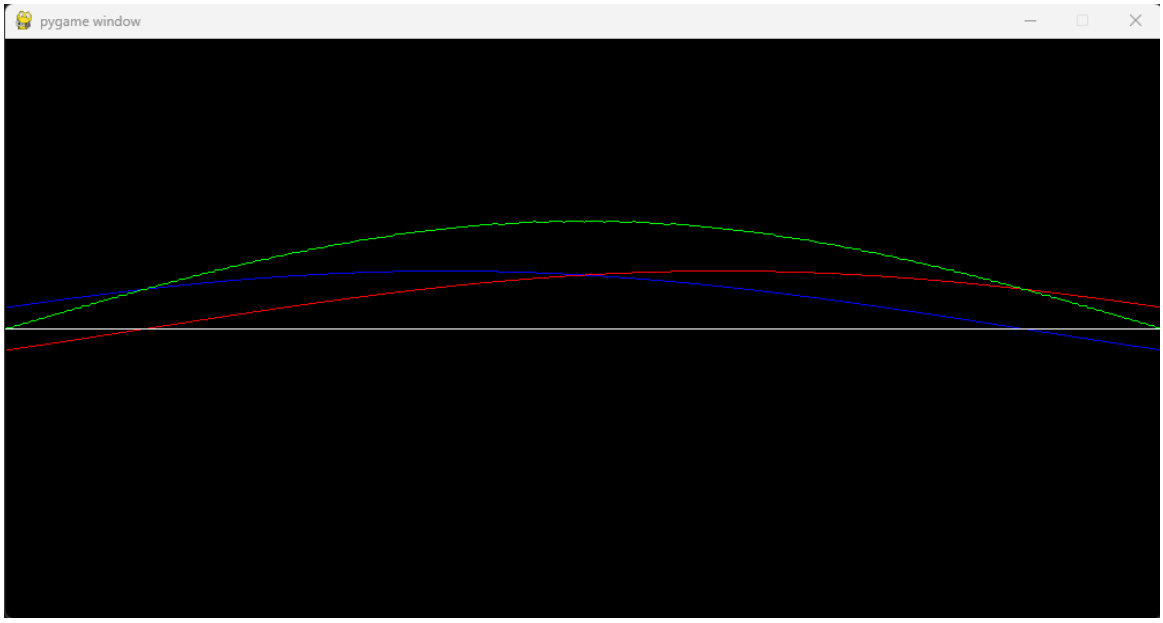
    l2rWave = list(zip(x,l2rWaveY))
    r2lWave = list(zip(x,r2lwaveY))
    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph

    pygame.draw.aalines(surface, BLUE, False, l2rWave)
    pygame.draw.aalines(surface, RED, False, r2lWave)
    pygame.draw.aalines(surface, GREEN, False, resultant)
    #draws lines on screen


#endProcedure
```

I also added the parameter “multiplier” because without this it runs in real time which is too fast for a user to gain a conceptual understanding. “multiplier” allows the wave to be simulated at a fraction of the realistic speed.

Seeing the green wave run in real speed made a much more realistic looking wave than the one previously produced in dispReal because it was moving through different amplitudes rather than just an alternation between 2 sine waves. This made me change dispReal to be the same as dispAbstract but without this multiplier and without the 2 progressive waves being shown.




This corrected dispAbstract code results in the expected output for the same test data. It also delivers the expected output for harmonic = 2,3,4,5. However, adjusting the length of the string has no effect.

To fix this problem I defined the start and end of the graph so that it was centred. I then used the same x values as above in the equations so they produce the correct y values then after the y values are calculated, adjusted the x values so the graph appears in the centre. I also added the variable ratioL which means that the correct harmonic is always displayed.

```
def
dispAbtract(T,mu,l,harmonic,A,screenSize,surface,multiplier,progressive):
    # Displays 3 waves. One resultant sationary  wave and two progressive
    waves
    # Takes harmonic as parameter because frequency has to be at a harmonic

    SCREEN_LENGTH = 2 #metres  How many metres the screen width represents

    metresPerSecond = calcSpeed(T,mu) # speed in metres per second
    v = metresPerSecond * screenSize[1] * multiplier * SCREEN_LENGTH
    #multiplied by screenSixe to get speed in pixels per second
    #multiplier is the fraction of the speed it moves

    pixelL = (l * screenSize[0])/SCREEN_LENGTH
    start = (screenSize[0]-pixelL)/2
    end = (screenSize[0]+pixelL)/2
    #centres the graph

    heightRatio = screenSize[1] / 500 #multiplier so amplitude is adjusted
    with screen height
    ratioL = l/SCREEN_LENGTH #makes the string the correct length

    x = np.arange(1,pixelL) # creates np array with all x values

    t = time.time()

    l2rWaveY = np.round(heightRatio*A*np.sin((harmonic*x*np.pi-
v*t)/(screenSize[0]*ratioL)) + screenSize[1]/2)
    # Calculates y values for wave moving left to right
```

```
    r2lwaveY = np.round(heightRatio*-A*np.sin((harmonic*np.pi*(-x)-
v*t)/(screenSize[0]*ratioL)) + screenSize[1]/2)
    # Calculates y values for wave moving right to left
    resultantY = l2rWaveY + r2lwaveY - screenSize[1]//2
    # Calculates y values for resultant stationary wave

    x = np.arange(start,end+1)

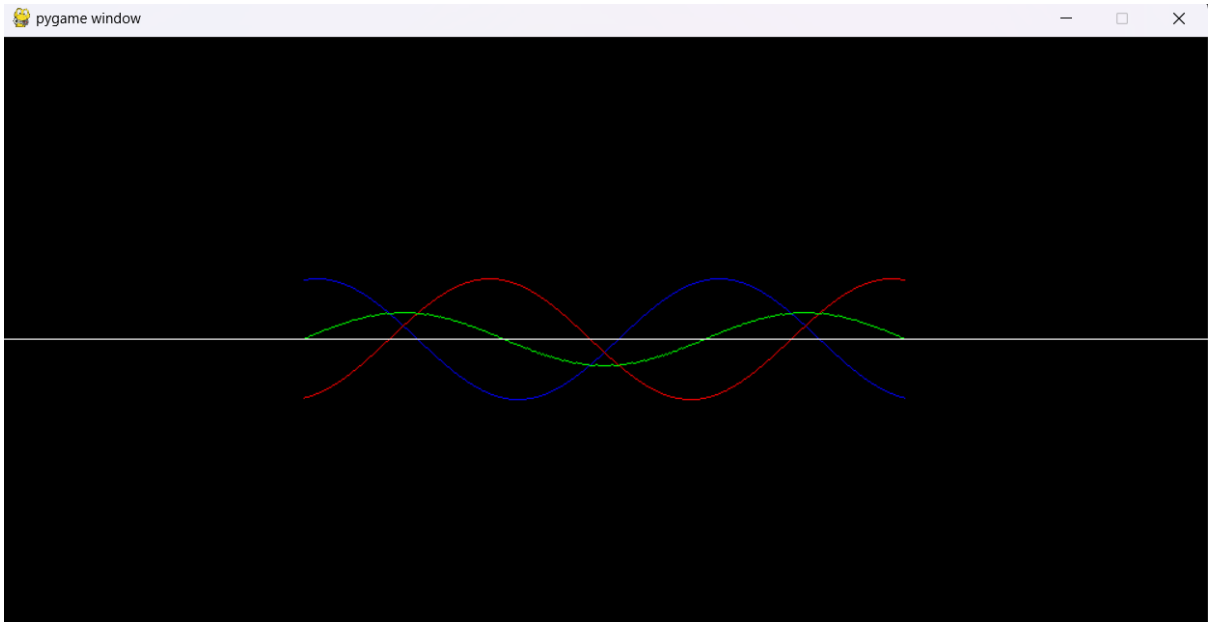
    l2rWave = list(zip(x,l2rWaveY))
    r2lWave = list(zip(x,r2lwaveY))
    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph

    if progressive: #only shows 2 progressive waves if selected by user
        pygame.draw.aalines(surface, BLUE, False, l2rWave)
        pygame.draw.aalines(surface, RED, False, r2lWave)
    #endif

    pygame.draw.aalines(surface, GREEN, False, resultant)
    #draws lines on screen
#endFunction
```

### *Testing*

This is the output when T = 1.96, mu = 0.0007, l = 1, harmonic = 3, A = 50.




This meets the expected output that was stated in the design section.

It also gave the expected output for the 1st, 2nd, 4th, 5th harmonics.

Passing a multiplier with a lower value made it move slower and passing a larger l (length) made the string longer but still displayed the correct harmonic. This is all the expected output.

### *Review*

This was a very successful module as I got stationary waves to be displayed in an abstracted view. It also helped me understand the Physics behind this practical activity myself as I could see how they are formed. If I were to do this again, I would make sure in design that I have put the correct variables in the correct place in equations so that I would not get the errors I got. I now need to create controls that allow the user to adjust the properties of this wave.

## Control Panel - RP 01 Sliders

I have decided to develop the sliders for RP 01 after developing the visuals for it because the user needs to be able to easily adjust variables so that they can see their effects on the display. I could have chosen to develop the equipment after the visuals because this is also a key module of the final solution but I chose to develop the sliders because the solution cannot be used without them.

To develop this module, I will follow the pseudocode for the sliders class I created in Design. I also used a Geeks for Geeks webpage (Geeks For Geeks, 2022) to help me understand how to move objects in PyGame.

At first I decided to leave out the procedure enterValue so I can test the slider alone and then develop that at a later date.

This was my initial code:

```
class Slider:
    #This class allows sliders to be instatiated and moved
    def __init__(self,givenPos, givenLength, givenMinValue,givenMaxValue):
        self.pos = givenPos
        self.length = givenLength
        self.minValue = givenMinValue
        self.maxValue = givenMaxValue
        self.valueHeld = (givenMaxValue + givenMinValue)/2
        self.posPointer = list(givenPos)
        self.boxColour = BLACK
        self.pointer = pygame.Rect(self.posPointer[0]-2,
self.posPointer[1]+4,4,8) #creates a Rect object for the pointer
    #endfunction

       def move(self):
        # This procedure changes the pointer location to the position
selected by the user
        for event in pygame.event.get():
            if event.type == MOUSEBUTTONDOWN and
self.pointer.collidepoint(event.pos):
                #checks if pointer is clicked
                moving = True

            elif event.type ==  MOUSEBUTTONUP:
                #stops moving pointer if finger off mouse
                moving = False
            elif event.type == MOUSEMOTION and moving:
                if abs(event.pos[0]-self.pos[0]) < self.length/2:
                    #if the mouse is within the range
                    #the mouse x coordinate = pointer x coordinate
                    self.posPointer[0] = event.pos[0]
                elif event.pos[0] <= self.pos[0] -self.length/2:
                    #sets x value of pointer to min if mouse off to left
                    self.posPointer[0] = self.pos[0] -self.length/2
                else:
                    #sets x value of pointer to max if mouse off to left
                    self.posPointer[0] = self.pos[0] + self.length/2
                #endif
            #endif
            self.pointer.move_ip(tuple(self.posPointer)) # moves the
pointer to the correct position
    #endProcedure
```

```
    def setValueHeld(self):
        #This procedure changes the value held to the one indicated to by
the pointer
        range = self.maxValue - self.minValue
        relPosPointer = self.posPointer[0] - (self.pos[0] - self.length/2)
        #distance of pointer from left end
        unitsPerPixel = range/ self.length
        self.valueHeld = self.minValue + relPosPointer * unitsPerPixel

    def display(self,surface):
        #displays the slider on the screen
        startPos = (self.pos[0]-self.length/2, self.pos[1])
        endPos = (self.pos[0]+self.length/2, self.pos[1])
        pygame.draw.line(surface, BLACK,startPos, endPos, width = 2)
        #draws the black slider line

        pygame.draw.rect(surface, BLACK, self.pointer)
        #draws the pointer as a rectangle

        self.move()
        self.setValueHeld()
    #endprocedure
#endclass
```

The move method allows the user to adjust the slider by clicking and dragging the pointer. It first checks if the pointer is clicked and sets the moving variable to True if it is so that the pointer can be moved in the next frames. If the mouse is lifted up, the moving variable is set to False so that the pointer cannot be moved in subsequent frames. If the mouse is moving and the moving variable is True, the position is then validated using the validation designed in the design section:

*Validation:*

| Module | Validation Type | Response if not met |
|---|---|---|
| Sliders - Move | Range – the movement of the mouse must be within the length of the slider. | If the slider moves past the end of the line, the pointer will move to that end of the line |

This is met by the if statements in the move procedure when moving is True and the mouse is moving.

I have developed this move procedure in this way because it is a simple way of checking if the mouse is being held down and the mouse is moving.

*Testing*

After creating a slider in the middle of the screen with length 300, minValue 0 and maxValue 100, I got this error message:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.7.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
Traceback (most recent call last):
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Sliders.py", line 87, in <module>
    slider.display(surface)
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Sliders.py", line 51, in display
    self.move()
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Sliders.py", line 25, in move
    elif event.type == MOUSEMOTION and moving:
UnboundLocalError: local variable 'moving' referenced before assignment
PS C:\Users\domin\OneDrive - Wren Academy\Year 12\Computer Science\NEA>
```

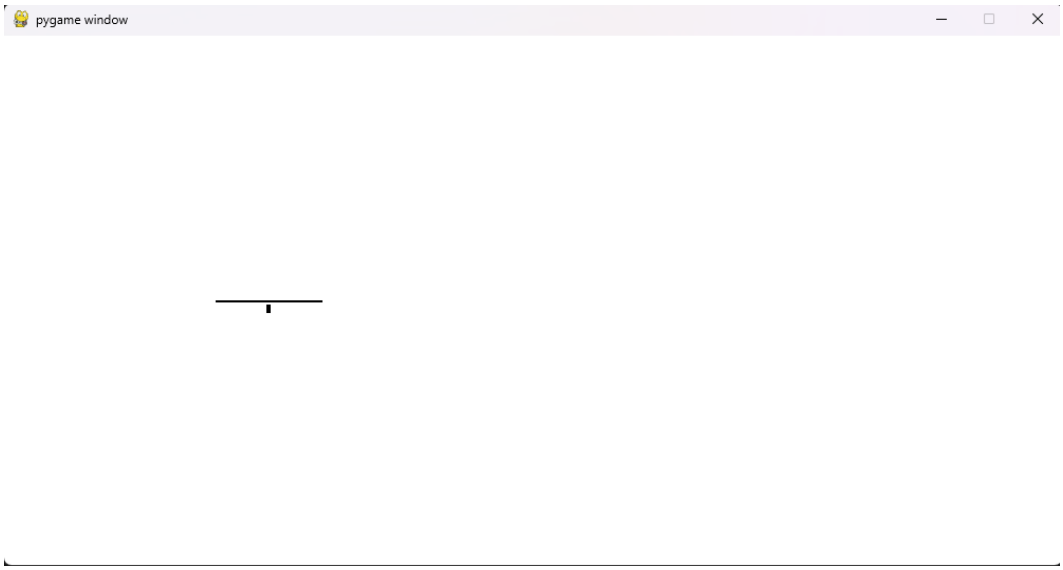
### *Remedial Action*

This is because moving was not set to False before being referenced. If I set it to False at the start of the move procedure, the program would not work because moving would be reset every frame.

To fix this I decided to make moving an attribute so it is declared beforehand and not reset every time. I could have instead made it a global variable but I decided to make it an attribute to use the benefits of encapsulation so I do not change it elsewhere in the program.

### *Testing*

After fixing this problem, this is what was output when length was set to 100 and pos set to (250,250)




At first this seemed successful, but when I moved my mouse onto the screen, the pointer disappeared.

### *Remedial Action*

The only sub routine that affects the position of the pointer is the move() procedure so the error must be in this section of the code. The only line in this procedure that occurs when the mouse is not moving is this line:

```
self.pointer.move_ip(tuple(self.posPointer))
```

After looking at the PyGame documentation (PyGame, 2025) for the method move_ip, I found that it moves the rectangle x pixels to the right and y pixels down if the tuple (x,y) is passed, rather than moving it to the position (x,y) which is what I expected.

I fixed this by changing this line to:

```
self.pointer.topleft = (self.posPointer[0] - 5, self.posPointer[1] + 10)
```

This successfully moves the pointer to the correct position identified by posPointer. It is adjusted by -5 and +10 so that the top left is in the correct location as posPointer holds the coordinates of the centre of the pointer.

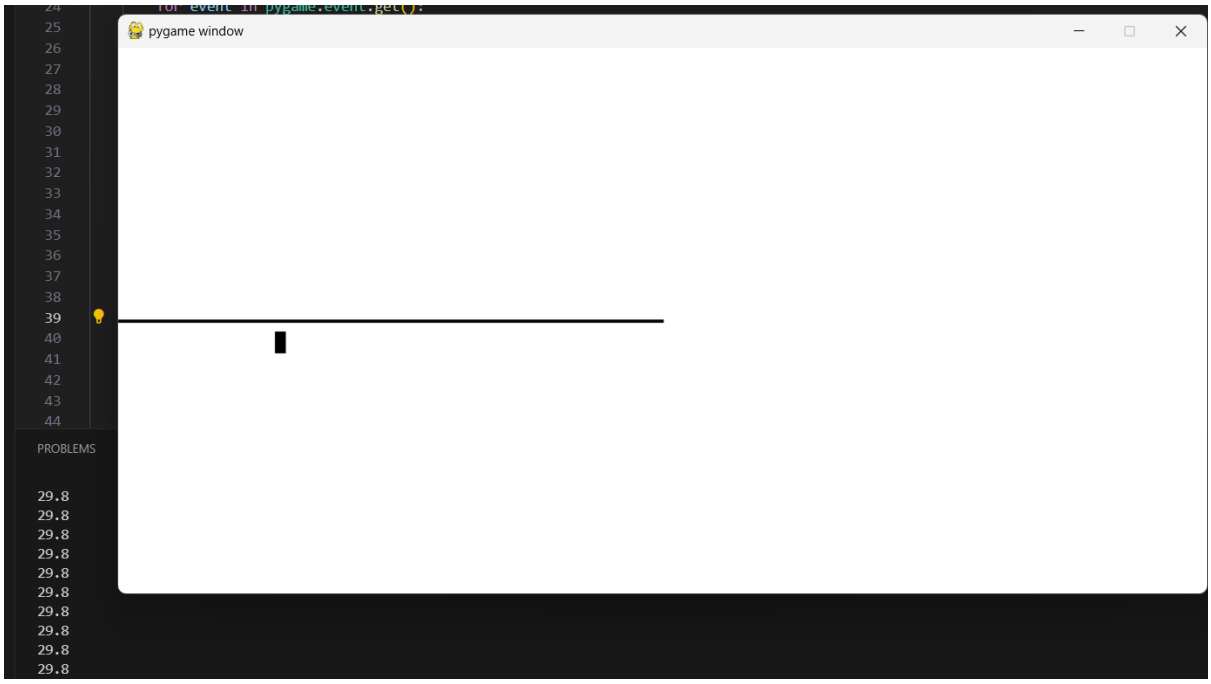
To test that this works successfully, I made a new method :

```
    def getValueHeld(self):
        return self.valueHeld
```

I then put print(self.getValueHeld()) within the display() method.

This prints the value the slider represents every loop.

### *Testing*




After setting the minValue to 0 and the maxValue as 100, this was the output when the pointer was in line with the cursor. This is correct because the pointer is approximately 30% across the slider and it is printing out 29.8.

When the cursor is dragged to the far right, the pointer moves to the right end of the line and it prints out 100 which is the correct output.

The program also meets the test data specified in the Design section because when the pointer is dragged past the right limit, the pointer stops at that end of the line and when it is moved far left, it stops at the left end of the line, meeting the validation set in the design section.

### Input Box

I now need to make an input box within the slider that allows the user to enter a value and the slider moves to this value. I read "How to create a text input box using PyGame" by Geeks for Geeks (Geeks for Geeks, 2021) to help me learn how to accept user input.

This was my initial code for this module:

```
def editValue(self):
      #allows the user to enter a value
        for event in pygame.event.get():
            if event.type == pygame.MOUSEBUTTONDOWN : #checks if mouse is
clicked
                if self.inputBox.collidepoint(event.pos):
                    self.active = True #if box is clicked; it is set to
active
                else:
                  self.active = False #if somewhere else is clicked, it
                  is no longer active

            elif self.active and event.type == pygame.KEYDOWN:
                if event.key == pygame.K_BACKSPACE:
                    self.userText = self.userText[:-1]
                    #deletes text if backspace hit
                elif event.key == pygame.K_RETURN:
                    # Sets the value correctly if enter button is clicked
                    self.active = False
                    if self.userText : # presence check
                        pass #TODO - further validation to be performed
                    else:
                        self.boxColour = (255,0,0)


                else:
                    self.userText += event.unicode
                    # adds the key clicked to the string
                #endif
            #endif

            if self.active:
                self.boxColour = (255,2,25)
                #box is red if clicked
            else:
                self.boxColour = (22,2,2)
                #if the box is clicked, it is black
            #endif

            self.textSurface =
pygame.font.Font(None,32).render(self.userText,True, self.boxColour)
            # creates a surface that has the text in it



            self.inputBox.width = max(100,self.textSurface.get_width()+10)
            # adjusts the width of the inputBox Rect so that it increases
            as more text is added
    #endfunction
```

This code does not change the value held in the object if a new value is entered as this will be developed next but I have done this first to check that the input box allows you to type in it. I have only written this code to see if it allows the user to type a value in.

This code meant that a few more attributes had to be created to allow it to function :

```
self.inputBox = pygame.Rect(self.pos[0]-20,self.pos[1]-40,40,40) # creates
A Rect object for the input box
self.userText = str(self.valueHeld) #the string to be displayes in the
input box
self.active = False #whether the input box is active
self.textSurface = pygame.font.Font(None,32).render(self.userText,True,
(255,255,255))
#The surface the text is written onto
```

The display procedure must also be updated so that editValue() is called and the input box is displayed:

```
def display(self,surface):
        #This procedure displays the slider and innput box

        startPos = (self.pos[0]-self.length/2, self.pos[1])
        endPos = (self.pos[0]+self.length/2, self.pos[1])
        pygame.draw.line(surface, BLACK,startPos, endPos, width = 3)
        # Draws the line for the slider

        pygame.draw.rect(surface, BLACK, self.pointer)
        #drwas the pointer

        pygame.draw.rect(surface, self.boxColour, self.inputBox,width = 3 )
        #draws the input box
        surface.blit(self.textSurface, (self.inputBox.x, self.inputBox.y))
        # places the text in the box

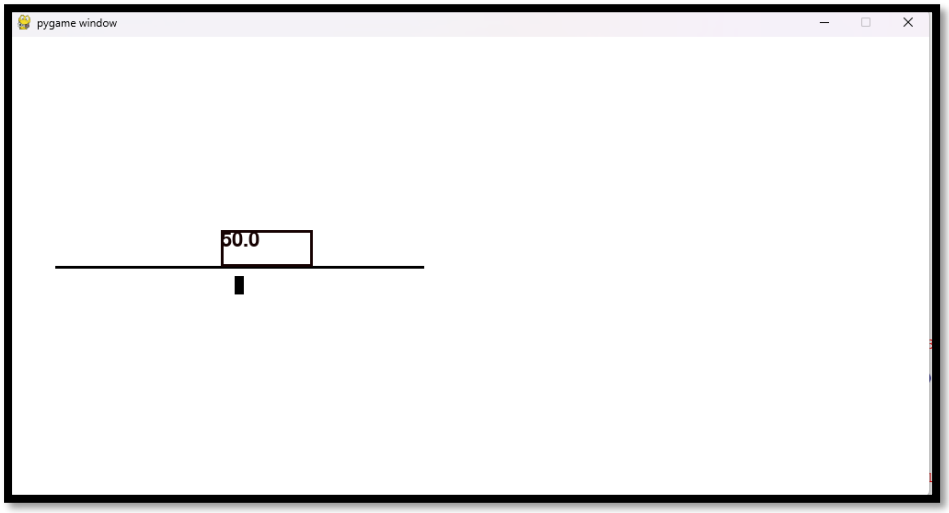
        self.editValue()

        self.move()
        self.pointer.topleft = (self.posPointer[0] - 5, self.posPointer[1]
+ 10)
        #adjusts the pointer to the new position

        self.setValueHeld()

    #endprocedure
#endclass
```

When run, this was the output:




I expected the number to be in the middle of the box so adjusted this line:

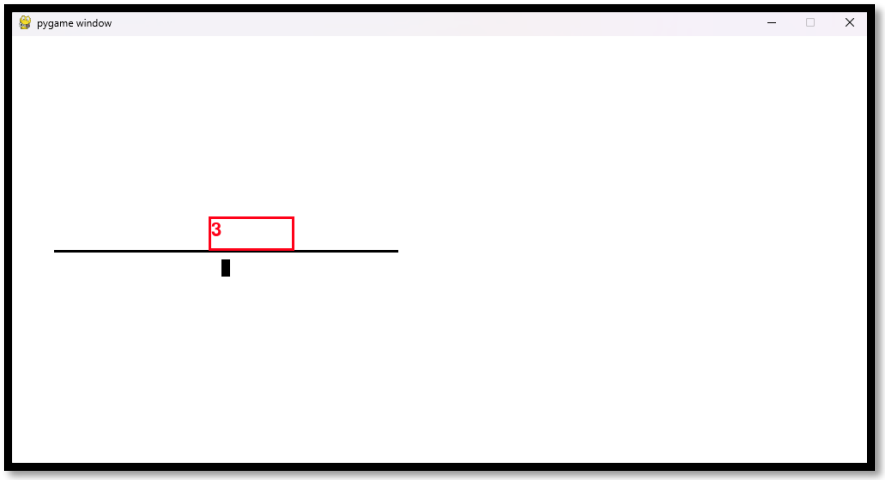
```
surface.blit(self.textSurface, (self.inputBox.x, self.inputBox.y))
```

to this:

```
surface.blit(self.textSurface, (self.inputBox.x +3, self.inputBox.y +5))
```

This fixed this issue.

As expected, the program allowed me to click the box and enter a value but clicking enter would only turn the box black and it would return to its previous value:



*Validation*

I then needed to allow a value to be entered and save that value. As a string is being input by the user, validation is required. Here is a small table showing the input and validation required:

| Input | Validation Required | If validation is not met |
|---|---|---|
| **Keyboard** | Must be a float or integer.<br>It must be greater than or equal to the minimum value<br>It must be less than or equal to the maximum value<br>A value must be entered | The value shown returns to the previous value held. |

The following is the new code for the editValue procedure with validation added in yellow and the code that moves the slider to the correct position highlighted in blue.

```
def editValue(self):
      #allows the user to enter a value
        for event in pygame.event.get():
            if event.type == pygame.MOUSEBUTTONDOWN : #checks if mouse is
clicked
                if self.inputBox.collidepoint(event.pos):
                    self.active = True #if box is clicked; it is set to
active
                else:
                    self.active = False #if somewhere else is clicked, it
is no longer active

            elif self.active and event.type == pygame.KEYDOWN:
                if event.key == pygame.K_BACKSPACE:
                    self.userText = self.userText[:-1]
                    #deletes text if backspace hit
                elif event.key == pygame.K_RETURN:
                    # Sets the value correctly if enter button is clicked
                    self.active = False
                     if self.userText and (self.userText.isnumeric()) and
float(self.userText) <= self.maxValue and float(self.userText) >=
self.minValue):
                        #presence and range check
                        self.valueHeld = float(self.userText)
                        #updates value held

                        # Move slider to correct position:
                        range = self.maxValue - self.minValue
                        pixelsPerUnit = self.length/range
                        relPosPointer = pixelsPerUnit * (self.valueHeld -
self.minValue)
                        self.posPointer[0] = relPosPointer + self.pos[0] -
self.length/2
                    else:
                        #sets box colour to red if validation is not met
                        self.boxColour = (255,0,0)



                else:
                    self.userText += event.unicode
                    # adds the key clicked to the string
```

```
                #endif
            #endif

            if self.active:
                self.boxColour = (255,2,25)
                #box is red if clicked
            else:
                self.boxColour = (22,2,2)
                #if the box is clicked, it is black
            #endif

            self.textSurface =
pygame.font.Font(None,32).render(self.userText,True, self.boxColour)
            # creates a surface that has the text in it

            self.inputBox.width = max(100,self.textSurface.get_width()+10)
            # adjusts the width of the inputBox Rect so that it increases
as more text is added
    #endfunction
```

### *Testing*

I entered into the box: 25, “h”, nothing, 100, 101 as designed in the test data section of my design and the output was as expected.

However, when a float was entered into the box like 55.55, the box would reset to the previous value held. This is incorrect because a float should be a valid input, not just integers.

### *Remedial action*

The only thing checking the type of the input is the isnumeric() function. After testing the function on different strings like “353” and “2.4” I found that the function only returns True if the data can be casted to an integer so does not return true for numbers that are not whole.

To fix this, I created my own function to check if a string can be casted to a float for user validation:

```
def isNumber(stringInput):
    #returns true if the string can be casted to a float
    try:
        float(stringInput)
        return True
    except ValueError:
        return False
#endFunction
```

After replacing self.userText.isNumeric() with isNumber, the program ran as expected when all any of the test data was entered.

I could have included the code within this function within the class definition but decided instead to define a function as this would increase readability and reusability.

### *Review*

I have succeeded in creating a slider that is able to be moved by clicking and dragging the pointer as well as being able to enter a value which is a key section of my final solution.

Looking back at the code for this module, I think that it can be made more modular as the editValue method is currently performing multiple functions.

I presented this slider to one of my stakeholders, Ian and he said that it was not obvious at first that the slider looked like a slider but once he knew it was a slider, it functioned as expected. He also said that it would be useful to be able to click a point on the line the slider is on and the pointer would move to that position.

To make my solution more usable, I went back to design for a 2nd iteration of the sliders.

## Sliders – 2nd Iteration

I now need to develop the adjustment to the move method which allows the line to be clicked and the pointer moves to the correct position.

I changed the move method to this with the new code highlighted in yellow:

```
def move(self,event):
        # This procedure changes the pointer location to the position
selected by the user

        if event.type == MOUSEBUTTONDOWN and
self.pointer.collidepoint(event.pos):
            self.moving = True
        elif event.type ==  MOUSEBUTTONUP:
            self.moving = False
        elif event.type == MOUSEMOTION and self.moving:
            if abs(event.pos[0]-self.pos[0]) < self.length/2:
                self.posPointer[0] = event.pos[0]
            elif event.pos[0] <= self.pos[0] -self.length/2:
                self.posPointer[0] = self.pos[0] -self.length/2
            else:
                self.posPointer[0] = self.pos[0] + self.length/2
        elif event.type == MOUSEBUTTONDOWN and
self.lineRect.collidepoint(event.pos):
            self.pos[0] = event.pos[0]

            #endif
        #endif
```

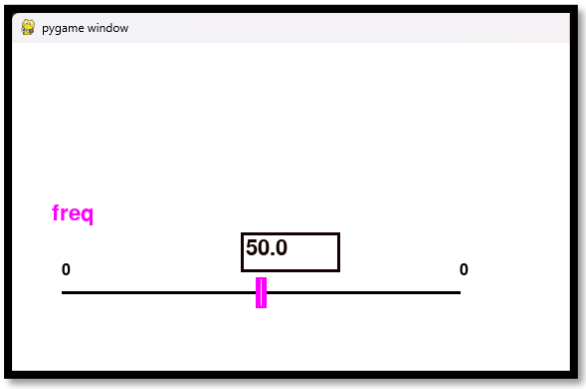
When running this code this error message appeared:

```
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Sliders.py", line 212, in <module>
    slider.move(event)
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Sliders.py", line 76, in move
    self.pos[0] = event.pos[0]
TypeError: 'tuple' object does not support item assignment
PS C:\Users\domin\OneDrive - Wren Academy\Year 12\Computer Science\NEA>
```

This is because this code was attempting to move the position of the entire slider, not just the pointer. It shows that my code has been well designed because if I had made pos a list instead of a tuple, it would allow me to move the whole slider which could cause errors.

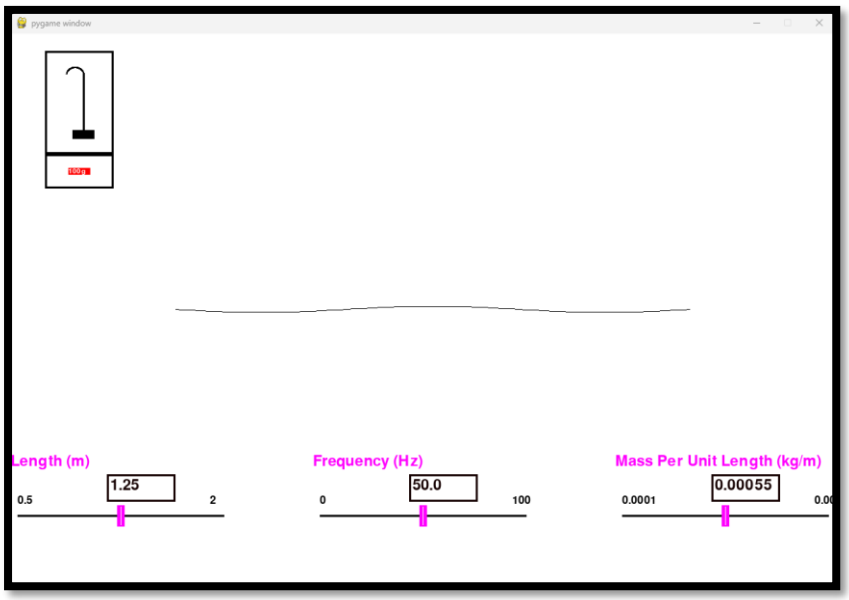
I changed self.pos to self.posPointer and the code ran as expected.

To meet my stakeholder's usability requirements, I changed the sliders to look like this:



as specified in the usability features section of my design.

The numbers on each end of the slider are supposed to show the maximum and minimum values that the slider can hold. The one on the left only should be 0 while the one on the right should be 100.




I realised I was passing the minimum value into where maximum value should be.

Now the maximum value is not always right aligned. It is in the correct position for the frequency slider but is too far left for the length slider that it may not be clear that it is the maximum value and it is too far right in the mass per unit length slider that it is going off of the screen.

I fixed this by getting the length of the string of the max value and subtracting the width of the surface object from it.

```
surface.blit(self.maxValSurface,(self.pos[0]+self.length/2 -
self.maxValSurface.get_width(), self.pos[1] - 30))
```

As stated in the design section I split the editValue method into 2 smaller methods to improve modularity:

```
def adjustValue(self):
    # move slider to correct position:
    range = self.maxValue - self.minValue
    pixelsPerUnit = self.length/range
    relPosPointer = pixelsPerUnit * (self.valueHeld - self.minValue)
    self.posPointer[0] = relPosPointer + self.pos[0] - self.length/2


def editValue(self,event):

    if event.type == pygame.MOUSEBUTTONDOWN :
        if self.inputBox.collidepoint(event.pos):
            self.active = True
        else:
            self.active = False

    elif self.active and event.type == pygame.KEYDOWN:
        if event.key == pygame.K_BACKSPACE:
            self.userText = self.userText[:-1]
            #deletes text if backspace hit
        elif event.key == pygame.K_RETURN:
            self.active = False
            if self.userText and (isNumber(self.userText) and
float(self.userText) <= self.maxValue and float(self.userText) >=
self.minValue):
                #presence and range check
                self.valueHeld = float(self.userText)
                #updates value held
                self.adjustValue()

            else:
                self.boxColour = (255,0,0)

        else:
            self.userText += event.unicode


        #endif
    #endif

    if self.active:
        #sets colour to red if currently typing
        self.boxColour = RED
    else:
        self.boxColour = BLACK
        self.userText = str(round(self.valueHeld,5))
    #endif

    self.valueSurface =
pygame.font.Font(None,32).render(self.userText,True, self.boxColour)
    #creates a surface of with text on it size 32 font



    self.inputBox.width = max(100,self.valueSurface.get_width()+10)
#endfunction
```

*Review*

This second iteration of the development of the sliders was a success: I had made the sliders more usable and I also made my code more modular by splitting the editValue method into 2 separate methods.

This module as a whole has allowed me to learn more about object oriented programming and gave me experience in implementing it.

I showed this new slider to Ian and he said it was “straightforward and clear”.

I need to create a new discrete slider that inherits from slider ad

## Discrete Slider

This is my code for the discrete slider which allows the user to adjust the number of the harmonic when displaying the abstract view of the stationary waves.

```
class discSlider(Slider):

    def setValueHeld(self):
        #changes the value held to the one indicated by the slider
        super().setValueHeld()
        self.valueHeld = round(self.valueHeld)
        #Round the value to nearest integer as its discrete
        super().adjustValue()
    #endProcedure

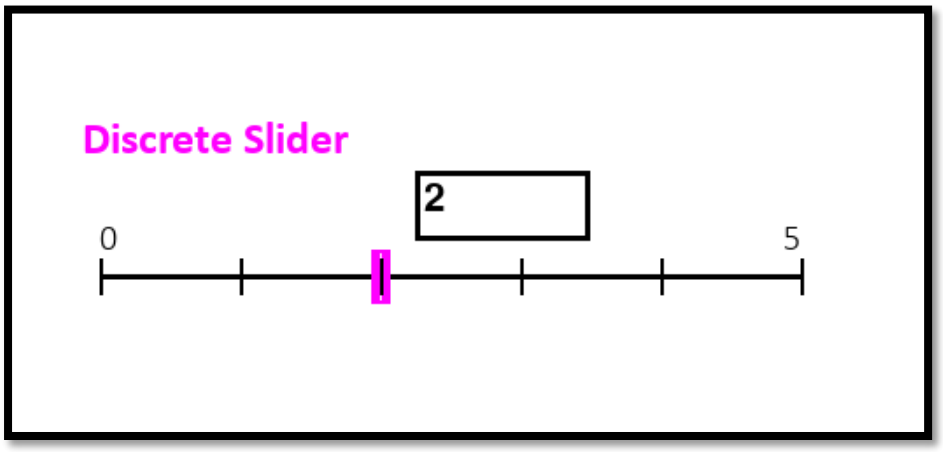
    def display(self,surface):
        #displays the slider on the screen
        super().display(surface)

        farLeft = self.pos[0]-self.length/2
        bottom = self.pos[1]-10
        top = self.pos[1] +10
        spacing = self.length /(self.maxValue-self.minValue)
        for i in range(self.maxValue+1):
            xPos = farLeft + i * spacing
            pygame.draw.line(surface, BLACK, (xPos,bottom),(xPos,top),
width= 2)
    #endProcedure
#endClass
```

The code is very similar to the pseudocode I designed earlier. However, when developing I removed the constructor method because there wasn’t any differences between that constructor and the constructor of the parent class except for the attribute spacing. When developing, I noticed that spacing is only used within the display method so can be defined as a local variable within this method.

I used inheritance for this module because the discrete slider shares many similarities with the normal slider so I used inheritance so I do not need to re -write code as I would if it was a stand-alone class.

This is what the slider looks like when the code is run.




It is displayed as expected.

*Validation*

| Module | Validation Required | Designed Response | Developed response |
|---|---|---|---|
| **discSlider – editValue** | Type – value entered must be an integer | Box flashes red and returns to previous value | In development, I changed the floats from being rejected to rounding them to the closest integer. This is because it is easy to change this within the setValueHeld method as I can just round the value set by this. Changing the validation within the editValue method would require re-writing the entire method. This change is also more accepting of user input, making it more user-friendly. |

*Testing*

This is the table showing the different modules in this discreate slider being tested

| Module | Test Data | Test Type | Expected Output | Output as expected |
|---|---|---|---|---|
| **discSlider – editValue** | 2 | Normal | Pointer to move to the second vertical line on the slider. | Yes |
| **discSlider - editValue** | 2.7 | Erroneous – Wrong Type | The box should flash red and return to its previous value. | No – I have changed it in development to change to the closest integer – see validation. |
| **discSlider - move** | Move the slider from left to right | Normal | Should jump from value to value. | Yes |

### *Review*

This sub-module went well: I created a new class without having to re-write lots of code by using the benefits of abstraction.

If I were to do this again, I would think ahead in design and consider what the best response to validation would be by considering what would be easier to program as well as the usability.

## Equipment – RP 01

### Mass Hook

I will now be developing the mass hook and the option to add masses to the hook.

I will start by developing the mass hook that I designed in the design section. This will be a class that once an object has been instantiated from will be able to be moved around and cylindrical masses added to.

In my design section, I said that the hook will be displayed by drawing a line connected to a PyGame arc which would be a semi-circle. However, this would be tricky to implement as the line and the arc would have to be moved together and treated as one object. Instead, I used MS Paint to draw an image of what the hook would look like and imported this into PyGame. This allowed me to move the hook as one.

```
class MassHook:
    def __init__(self, givenPos:tuple, givenAutoSetup:bool):
        self.pos = givenPos #startingh position
        self.numOfMasses = 0 #starts with no masses on
        self.autoSetup = givenAutoSetup

        self.moving = False #if the hook is currently being dragged
        self.hookImageBig = pygame.image.load("Mass Hook Image.png")
#imports the hook image
        self.hookImage =
pygame.transform.scale(self.hookImageBig,(self.hookImageBig.get_width()/2,s
elf.hookImageBig.get_height()/2))
        #resizes by scale factor 2
    #endProcedure

    def move(self):
        # This procedure changes the mass Hook location to the position
selected by the user
        for event in pygame.event.get():
            if event.type == MOUSEBUTTONDOWN and
self.hookImage.collidepoint(event.pos):
                #checks if image has been clicked
                self.moving = True
            elif event.type ==  MOUSEBUTTONUP:
                # stops drag if mouse button raised
                self.moving = False
            elif event.type == MOUSEMOTION and self.moving:
                #moves the hook to the correc position
                self.pos = pygame.mouse.get_pos()

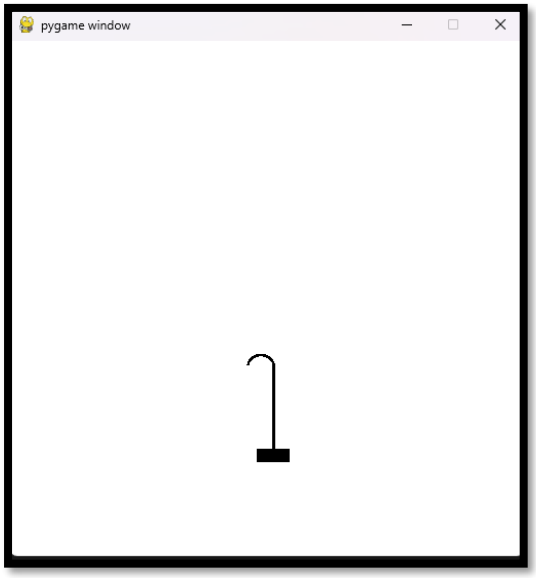

            #endif


    def display(self):
        #displays the hook on the screen
        surface.blit(self.hookImage,(self.pos[0]-
self.hookImage.get_width()/2,self.pos[0]+self.hookImage.get_height()/2))

        if not self.autoSetup:
            self.move()
    #endFunction
```

### *Testing*

When running this code with givenPos set as the centre of the screen and given auto setup set as False, this is what was displayed:




The hook is evidently not in the centre of the screen so I looked at the code where the image was being displayed.

### *Remedial Action*

I noticed this line:

```
surface.blit(self.hookImage,(self.pos[0]-
self.hookImage.get_width()/2,self.pos[0]+self.hookImage.get_height()/2))
```

This is supposed to “blit” the image onto the screen with the top left of the image placed having the coordinates given.

```
self.pos[0]+self.hookImage.get_height()/2
```

is supposed to give the y coordinate of the top left of the image but I have added half the height to the x coordinate of the centre when half the height should be subtracted from the y coordinate.

Therefore this line was replaced with this:

```
surface.blit(self.hookImage,(self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-self.hookImage.get_height()/2))"""
```

When I tried to click and move the hook, this error occurred:

```
  File "c:/Users/domin/OneDrive - Wren Academy/Year 12/Computer Science/NEA/Old Mass hook.py", line 22, in move
    if event.type == MOUSEBUTTONDOWN and self.hookImage.collidepoint(event.pos):
AttributeError: 'pygame.surface.Surface' object has no attribute 'collidepoint'
PS C:\Users\domin\OneDrive - Wren Academy\Year 12\Computer Science\NEA>
```

This meant that I could not check that it collided with the image the way I did with the Rect object in the sliders module. Therefore, I must create a Rect object to move with the hook so I can check if it has been clicked. Therefore, I added this into the constructor method:

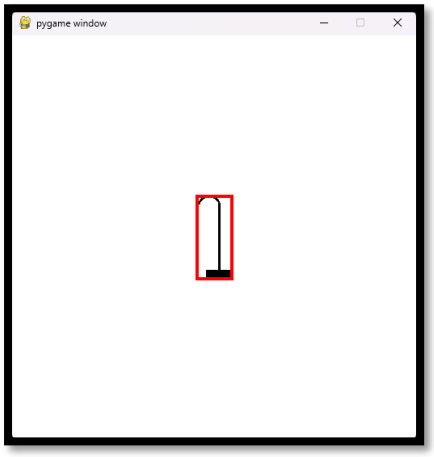
```
self.hookImageRect = pygame.Rect(self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-
self.hookImage.get_height()/2,self.hookImage.get_width(),
self.hookImage.get_height())
```

This creates a PyGame "Rect" object that can be checked with the mouse position to see if they collide. The constructor of the Rect object requires the coordinates of the top left of the rectangle and the width and height of the rectangle. As `pos` is the coordinates of the centre of the hook, half the width and height had to be subtracted.

For the width and height, I could have gone onto MS Paint and read off the height and width of the image in pixels. However, I used the get_width() method instead to improve flexibility so that in case I change the size of the image in the future, my code will still work. It also improves readability as using numbers would be confusing as to where they came from.

### *Testing*

I tried running the code with this change and it was in the correct position:




I now need to test that the move method works using the test data created in the design section:

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **MassHook - move** | Move the hook around the screen | Normal | The hook should follow where the cursor of the mouse is. | It did not move. | Fail |

As you can see, it failed the test to see if it can move. I thought this may be a problem with the position of the Rect object so I got the program to draw it.

Therefore, there must be a problem elsewhere. I noticed that within the main loop, I was creating a new instance of the class for every frame so it was reset to its initial position. I then took the code instantiating the object and it would work.

After reviewing my code, I swapped this:

```
pygame.transform.scale(self.hookImageBig,(self.hookImageBig.get_width()/2,self.hookImageBig.get_height()/2))
```

with this:

```
self.resizeFactor = 2

self.hookImage = pygame.transform.scale(self.hookImageBig,(self.hookImageBig.get_width()/self.resizeFactor,self.hookImageBig.get_height()/self.resizeFactor))
```

to improve flexibility and readability because when looking back at the code, I was unsure what the 2 represented.

After looking at the test data in Design, I realised that I forgot to include validation to make sure the hook does not leave the screen.

I therefore changed the code to include this validation (added code is in yellow):

```
def move(self,events):
    # This procedure changes the mass Hook location to the position selected by the user
    if not self.autoSetup:
        SCREEN_SIZE = (1200,800)
        for event in events:
            if event.type == MOUSEBUTTONDOWN and event.button == 1 and self.hookImageRect.collidepoint(event.pos):
                self.moving = True
            elif event.type ==  MOUSEBUTTONUP:
                self.moving = False
            elif event.type == MOUSEMOTION and self.moving:
                mousePos = pygame.mouse.get_pos()
                if mousePos[0] > 0 and mousePos[1] > 0 and mousePos[1] < SCREEN_SIZE[1] and mousePos[0] < SCREEN_SIZE[0]:
                    #validation to check that it is still within the window
                    self.pos = mousePos
                #endif
            elif event.type == MOUSEBUTTONDOWN and event.button == 3 and self.hookImageRect.collidepoint(event.pos):
                self.removeMass()
            #endif
    #endif
#endProcedure
```

I needed to include this because without it, the user could move the hook off the screen and never find it again. I did this by ensuring that when dragging the hook, the mouse is still on the screen. This means that half of the hook can be moved off of the screen because the position of the mouse corresponds to the middle of the hook. This is fine because the hook can still be easily moved in this situation.

### *Testing*

I will now test this module again, now these corrections have been made.

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **MassHook - move** | Move the hook around the screen | Normal | The hook should follow where the cursor of the mouse is. | The hook follows where the cursor of the mouse is. | Pass |
| | Move the hook off the screen | Erroneous | The hook should stop at the edge of the window. | The hook stops at the edge of the window. | Pass |

### *Review*

This module has been a success so far as I have been able to display and move the mass hook. If I were to do this again, I would have designed my code better so that the editValue method could be split into more subroutines that all only perform one function. I now need to create code that allows masses to be added and removed from the hook so this feature is usable.

### *Adding and Removing Masses*

I now need to create methods that allow masses to be added to the hook.

I decided to add these as rectangles added to the bottom of the hook in a stack. To do this, I needed to create attributes that correspond to the top of the stack so masses can be added in the correct position.

```
self.relStackTopx = 26 #the relative x coordinate to the image dimensions
of the top left of bottom mass
 self.relStackTopy = 187 #the relative y coordinate to the image dimensions
of the top left of bottom mass
 self.stackBase = ((self.topLeftPos[0] +
self.relStackTopx/self.resizeFactor, self.topLeftPos[1]
+self.relStackTopy/self.resizeFactor))
 #fixed coordinates of the top left of the bottom mass
 self.topOfStack = list(self.stackBase)
 #changable cordinates of the top of the stack
```

Despite it reducing the flexibility of the code, I decided to use the coordinates (26,187) of the top of the bottom of the mass from MS Paint. This was necessary because it would be overcomplicated to get the top left of the bottom mass through another method that would be consistent for all images of the mass hook. This does mean that if another image was used, the coordinates would have to be updated.

This was my initial code for the addMass and removeMass methods which are based on the pseudocode created in the design section:

```
def addMass(self):
    #adds masses to top of stack

    if self.numOfMasses < self.MAX_PLATES:
        #checks that stack is not full
        self.numOfMasses += 1
        self.topOfStack[1] += 11

        RECT_HEIGHT = 10
        RECT_WIDTH = 65

        newMass =
pygame.Rect(self.topOfStack[0],self.topOfStack[1],RECT_WIDTH/self.resizeFac
tor, RECT_HEIGHT)
        #creates Rect object for mass to be added

        self.massesAdded.append(newMass)
    else:
        self.colour = RED

    #endif
#endProcedure
```

```
def removeMass(self):
    #removes a mass from the top of the stack
    if self.numOfMasses > 0 :
        #checks if stack is empty
        self.numOfMasses -= 1
        self.massesAdded.pop()
        self.topOfStack[1] -= 11
    else:
        self.colour = RED
    #endif
#endFunction


def display(self):
    #displays hook on the screen

    self.topLeftPos = (self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-self.hookImage.get_height()/2)
    #position of top lefft of hook
    surface.blit(self.hookImage,self.topLeftPos) #puts hook on screen
    self.hookImageRect.topleft = self.topLeftPos #chages op left of Rect
object

    if self.massesAdded :
        self.stackBase = ((self.topLeftPos[0] +
self.relStackTopx/self.resizeFactor, self.topLeftPos[1]
+self.relStackTopy/self.resizeFactor))
        #resests pos of stackbase if it has been moved
        for number, mass in enumerate(self.massesAdded,1):
            #iterates through the masses added, drawing each mass in turn
            mass.topleft = (self.stackBase[0],self.stackBase[1] - number
*11)
            pygame.draw.rect(surface, BLUE, mass)
    #endif



    if not self.autoSetup:
        self.move()
#endFunction
```

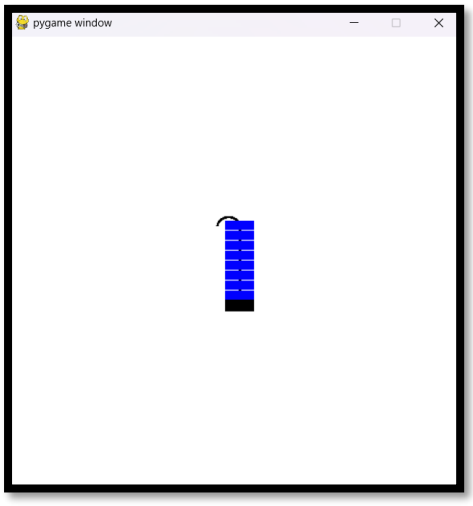
I then added an if statement within the display method that checks if the right mouse button has been clicked on the hook and if it has, it calls the addMass method and if the middle button on the mouse is clicked, it calls the removeMass method. This is not currently very usable but I just want to test for functionality at the moment and usability will be adjusted at a later date.

### *Testing*

In design, I only designed test data for the removeMass module because the addMass module is tested with the Masses module. However, I now want to test the addMass method by itself so have modified the test data:

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **MassHook - removeMass** | Right click the hook with a mass on it. | Normal | The hook should have one less mass on it than before | One mass is removed from the top. | Pass |
| | Right click the hook with no masses on it. | Erroneous | The hook should be left unchanged. | The hook is unchanged. | Pass |
| **MassHook - addMass** | Middle button on mouse is clicked with no masses on it. | Normal | The hook should gain one mass | The hook gained one mass | Pass |
| | Middle button on mouse is clicked when the hook has 8 masses on it. | Erroneous | The hook should be left unchanged. | The hook is unchanged. | Pass |

As you can see the hook is also displayed correctrly when it has 8 masses on it:




After reviewing my code, I noticed that although it functions, it is not space nor time efficient. As I drew the masses and determined their location using the display method and their location changes as the user moves it, there is no point saving Rect objects in a list because the number of masses added is sufficient information to add masses to the hook.

I could have left the code how it was but decided to update it to make it more readable and efficient. This is the updated code:

```
def addMass(self):
    #adds masses to top of stack

    if self.numOfMasses < self.MAX_PLATES:
        #checks that stack is not full
        self.numOfMasses += 1
    #endif
#endProcedure

def removeMass(self):
    #removes a mass from the top of the stack
    if self.numOfMasses > 0 :
        #checks if stack is empty
        self.numOfMasses -= 1

    #endif
#endFunction

def display(self):
    #displays hook on the screen

    self.topLeftPos = (self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-self.hookImage.get_height()/2)
    #position of top lefft of hook
    surface.blit(self.hookImage,self.topLeftPos) #puts hook on screen
    self.hookImageRect.topleft = self.topLeftPos #chages op left of Rect
object

    if self.numOfMasses > 0 :
        self.stackBase = ((self.topLeftPos[0] +
self.relStackTopx/self.resizeFactor, self.topLeftPos[1]
+self.relStackTopy/self.resizeFactor))
        #resests pos of stackbase if it has been moved
        RECT_HEIGHT = 10
        RECT_WIDTH = 65
        for i in range(1,self.numOfMasses+1):
            #iterates through the masses added, drawing each mass in turn
            topLeft = (self.stackBase[0],self.stackBase[1] - i *11)
            mass =
pygame.Rect(topLeft[0],topLeft[1],RECT_WIDTH/self.resizeFactor,
RECT_HEIGHT)
            pygame.draw.rect(surface, BLUE, mass)
    #endif


    if not self.autoSetup:
        self.move()
#endFunction
```

### *Review*

I have successfully been able to display a hook that is instantly recognisable and I have allowed the user to add and remove masses. This is an important feature because it is how the user adjusts the tension in the string for the stationary waves. Howevre, this module is useless without the masses so this must be developed next.

## Masses

This is my initial code for the masses class which will allow multiple 100g masses to be instantiated from it that can be moved and added to the mass hook. From the main loop, a list of hooks will be created that is iterated through, displaying them each in turn.

```
class Mass:
    #The 100g masses that can be added and removed from the hook.
    def __init__(self,givenPos):
        self.pos = givenPos
        self.moving = False
        self.show = True
        RECT_HEIGHT = 10
        RECT_WIDTH = 65/2
        topLeftX = self.pos[0] - RECT_WIDTH/2
        topLeftY = self.pos[1] - RECT_HEIGHT/2
        self.massRect  =
pygame.Rect(topLeftX,topLeftY,RECT_WIDTH,RECT_HEIGHT)


    def move(self,events):

        # Accepts user input and moves the hook to the correct position
        SCREEN_SIZE = (1200,800)

        for event in events:
            if event.type == MOUSEBUTTONDOWN and event.button == 1 and
self.massRect.collidepoint(event.pos):
                self.moving = True
            elif event.type ==  MOUSEBUTTONUP:
                self.moving = False
            elif event.type == MOUSEMOTION and self.moving:
                mousePos = pygame.mouse.get_pos()
                if mousePos[0] > 0 and mousePos[1] > 0 and mousePos[1] <
SCREEN_SIZE[1] and mousePos[0] < SCREEN_SIZE[0]:
                    #validation to check that it is still on
                    self.pos = mousePos
                #endif
            #endif
        #endFor
    #endProcedure

    def checkCorrectPos(self,hook):
        #checks if placed over hook and adds to hook if so
        if hook.hookImageRect.collidepoint(self.pos):
            hook.addMass()
            self.show = False
         #endif
```

```
    def display(self,hook):
        #displays the mass
        if self.show:
            self.move()
            self.massRect.center = self.pos
            pygame.draw.rect(screen,RED, self.massRect)
            self.checkCorrectPos(hook)

            textSurface = pygame.font.Font(None,15).render("100g",True,
WHITE)
            screen.blit(textSurface,self.massRect)
            #writes 100g on the mass
        else:
            pass
    #endProcedure
 #endClass
```

The move method is the same as the move method for the Mass Hook so was simple to replicate.

The Mass Hook module already has a way of drawing the masses onto the screen once they are attached. Therefore, within the Mass class I needed a way of checking that the mass is dragged on top of the hook so that the mass can be added to the hook through the addMass method and then stop beomng displayed itself.

I could have done this by getting the position of the centre mass hook and comparing it to the centre of the mass. However, this would be fiddly as a range would have to be added so the centre of the mass does not need to be held directly above the centre of the hook.

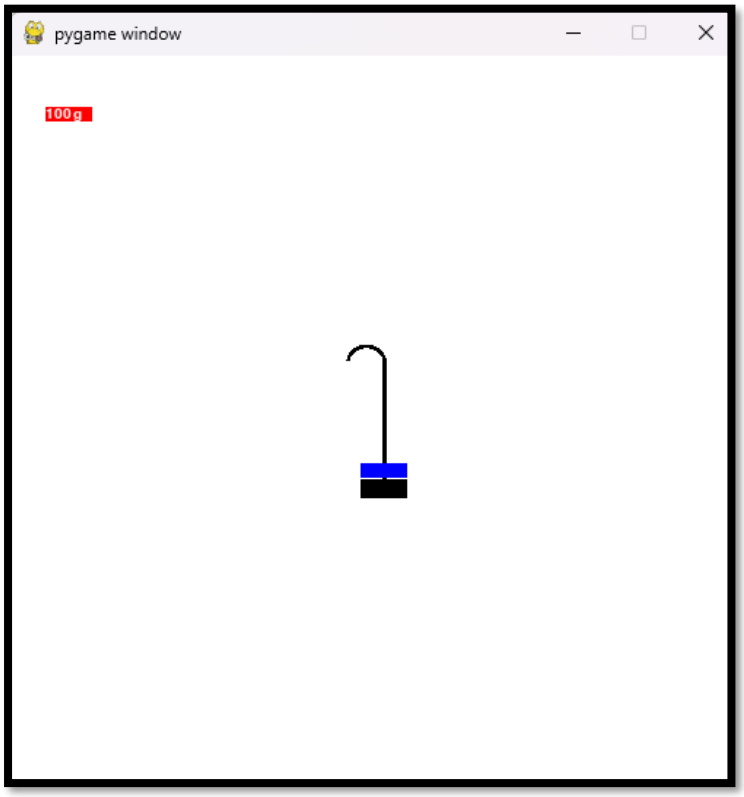
Instead, I got the Rect object from the hook object that defines the position and dimensions of the hook and used the PyGame function collidepoint with the coordinates of the centre of the mass. This would return True if the centre of the mass is over any point of the hook.

If the mass is placed over the hook, the mass no longer needs to be displayed so self.show is set to False which means that when the display method is called, it does not display the hook or allow it to be moved. Instead of doing this, I could have removed the mass from the list when self.show is False but I wanted to make the code for this module self-contained so it is easier to test. Also, when removing items from a list, errors can be caused if the list is emptied.

*Testing*

I tested this by creating 9 instances of the Mass class and creating 1 of the Mass Hook.

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **Mass - move** | Move the mass around the screen | Normal | The hook should follow where the cursor of the mouse is. | The mass follows the cursor. | Pass |
| **Mass – checkCorrect Pos** | Mass moved over the hook when it is empty | Normal | A mass should be added to the hook. The mass changes colour from red to blue. | A mass is added and changes colour from red to blue. | Pass |
| **Mass - checkCorrectPos** | Mass moved over the hook when it is full. | Erroneous | The mass should remain red and be able to be moved away from the hook again. | The mass stays red. | Pass |




As the image shows, the masses are able to be moved and added to the hook and become blue when added.

### *Review*

This module has been a success because I have been able to drag and move masses and move them onto the hook to adjust the tension for the stationary wave.

However, the only way to use multiple masses is to instantiate them all in different positions but it would take up less space on the screen and be more usable if they were all in the same position and one is dragged out at a time. This is not currently possible because if they were all in the same position, they would all be dragged together which would be problematic.

Therefore, I need to create code that allows one mass to be dragged out at a time.

## Equipment Box

As stated in my design section, the way items will be dragged out of the equipment box involves two classes: the DragBox class which is the smaller box that equipment is dragged out of and the EquipmentBox itself which once instantiated, is a larger box that holds multiple DragBoxes.

I have decided to develop this module after developing the masses to allow me to drag multiple masses onto the mass hook and alter the tension as without this module, the masses module is useless.

This was my initial code for the DragBox class which is very similar to the pseudocode designed in the Design section:

```
class DragBox:
    def __init__(self, givenPos, itemType):
        self.pos = givenPos # centre of box
        self.itemType = itemType #the class of the item to be held in the
box
        NewItem = itemType(givenPos)
        self.items = [] # a list of all active items
        self.items.append(NewItem)
        self.height = 50
        self.width = 100
        self.maxItems = NewItem.maxItems

        self.box = pygame.Rect(givenPos[0],givenPos[1], self.width,
self.height)
        self.box.center = (givenPos[0],givenPos[1])

    def checkMore(self):
        #checks if more items need to be added and returns a list of al the
masses
        if self.items[-1].pos != self.pos and  len(self.items) <
self.maxItems:
            self.items.append(self.itemType(self.pos))
        #endif

        return self.items
    #endfunction

#endClass
```

This is slightly different from the pseudocode because instead of passing the maxItems as a parameter, I instead got the max items from the class defines the objects held within the DragBox. I did this to make my code more maintainable because when new DragBoxes are created, I do not have to remember how many of each item there are supposed to be. This does mean that I have to remember to include an attribute called maxItems in each of my equipment classes.

I could test this class alone but it would take extra time and code to display it. I thought that it would be better instead to develop the EquipmentBox class first and test both together.

This is the initial code for the EquipmentBox class:

```
class EquipmentBox:
    def __init__(self,givenPos):
        self.width = 100
        self.height = 200
        self.pos = givenPos
        self.massesBox =
DragBox((self.pos[0],self.pos[1]+self.height/4),Mass)
        self.massHookBox = DragBox((self.pos[0],self.pos[1]-self.height/4),
MassHook)
        self.boxes = [self.massHookBox,self.massesBox] # a list of all
items in box

    #endProcedure

    def display(self,screen):
        #displays each of the smaller boxes
        for box in self.boxes:
            items = box.checkMore()
            #gets a list of all the items
            if box.itemType == MassHook:
                hook = box.items[0]
                hook.display(screen)

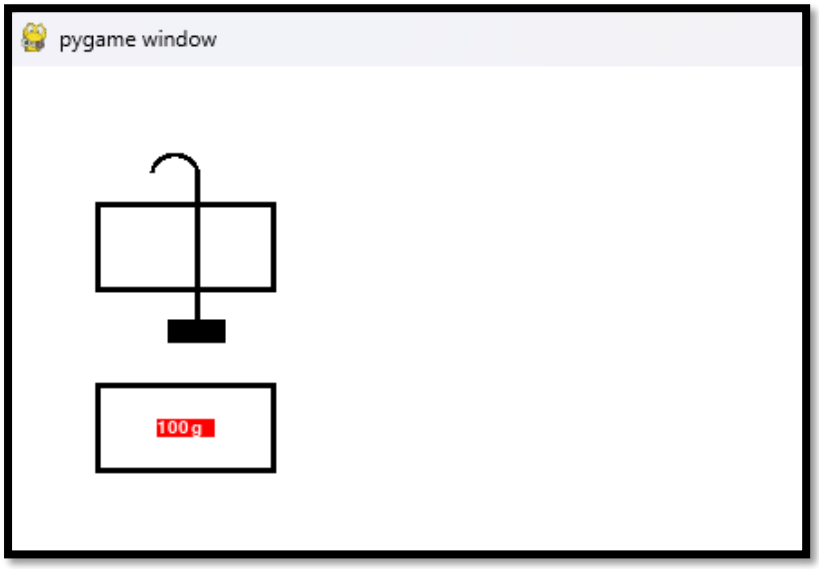
            elif box.itemType == Mass:
                #mass needs an extra parameter
                for item in items:
                   item.display(screen,hook)
            else:
                for item in items:
                    item.display(screen)
            #endif

            pygame.draw.rect(screen, BLACK, box.box, width = 3 )
            #draws each item to the screen

        #endfor
    #endProcedure
#endClass
```

This is very similar to the pseudocode designed in the design section. However, within the display method, I added an if statement that checks if the item type is a Mass. This is because the Mass constructor method takes an extra parameter (hook) so needs to be separated. Due to the Mass constructor needing the parameter hook, a local variable hook is created if the item type is a mass hook, hence the need to check if the item type is MassHook. At this time there is no need for this else condition because the only 2 items being displayed are the masses and the mass hook but I have added it to improve maintainability and flexibility so that in the future when new equipment is added, this method will still work.

### *Testing*



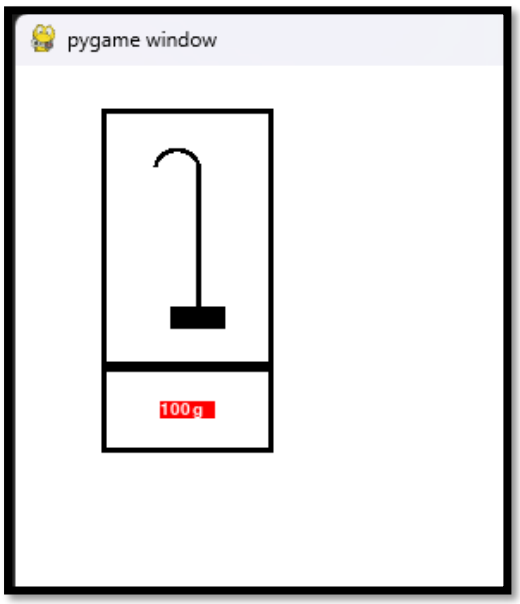

This was the output when an EquipmentBox was instantiated and the display method was called within the main loop. As you can see, the box for the mass hook is too short. However, they are both in the centre of their boxes and up to 8 100g masses can be dragged out and can be placed on the mass hook which can also be moved.

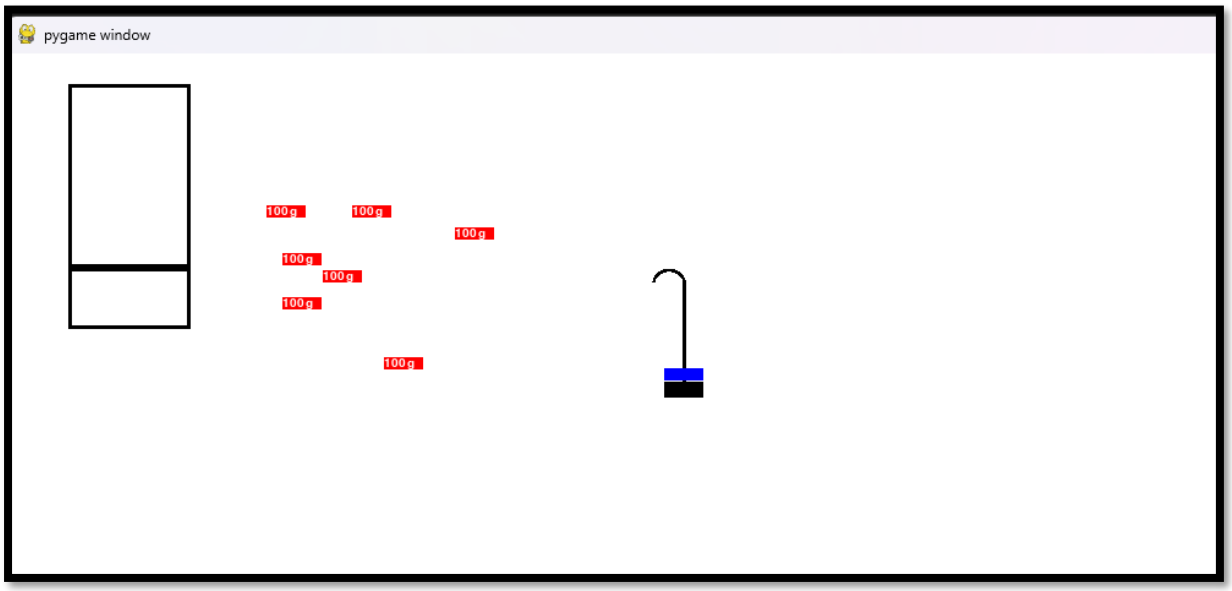
### *Remedial Action*

To adjust the height of the box correctly I added this to the constructor class:

```
if itemType == MassHook:
    #mass hook is a larger item
    self.height = 150
else:
    self.height = 50
```

It now displays correctly:





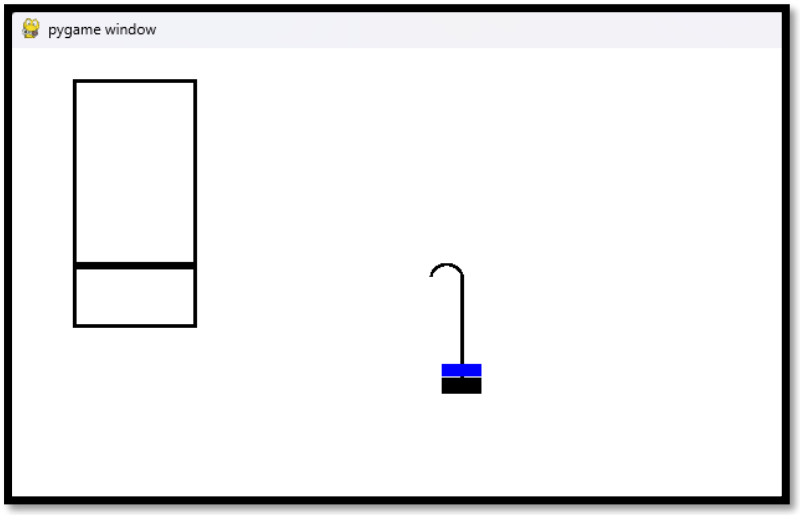

As you can see, masses can be dragged out and added to the hook as expected.

### *Testing*

However, I noticed that with a maximum of 8 masses being added, when a mass is removed, it does not create more masses in the equipment box. This means that if all 8 masses are added to the hook and then all removed, no new masses can be added again.

One way to fix this would be to remove a mass from the list in drag box every time a mass is removed from a hook. This would be very difficult because I would encounter numerous problems with encapsulation as I would not be able to access the DragBox class through the MassHook class unless it is provided as a parameter. Instead, I have chosen to set the maxItems of the masses to be 1000 so that many masses can be created will likely not run out.

Another problem I found was that if the hook was dragged over the position of the masses, no more masses would appear and only one would be added to the hook:



Only one mass was added because the checkMore method in the DragBox class only instantiates new masses if the previous mass is no longer in its original position. As the position of the mass added to the hook does not change when the mass hook is dragged over it (it is only not shown anymore and a rectangle is added to the hook), a new mass is not instantiated. To fix this, I added to the removeMass method within the Mass class this line:

```
self.pos = (-20,-20)
```

so that the mass is no longer located on the screen and allows a new mass to be instantiated.

I also added to the Mass method checkCorrectPos which checks if the mass is over the hook. It has validation that checks if the hook is full before setting show to false. This means that when the hook is dragged over the masses, only a maximum of 8 will leave the box and the other will not disappear.

### *Review*

This module has been a success because I have allowed multiple masses to be instantiated in the same location which fixes the problem I had in the previous masses module.

After talking to my stakeholder Ian, he said that there needs to be a bin or something similar that gets rid of the masses from the screen if the user would like as the screen can get cluttererd. This will be developed in the future if there is time.

## First Prototype

Now the main modules of the solution for the stationary waves simulator have been developed, I wanted to create a prototype to show to my stakeholders and gain feedback as well as testing to see if all modules work together

I added this code to the end of the Masses document to display everything together. At first I set the tension in the dispReal procedure to be 1.96 when in the end product it will be altered by adding masses to the mass hook. This is so I can first test that everything is displaying correctly before figuring out how to get the tension from the hook.

```
import Waves, Sliders
#modules to be used

pygame.init()
#loads the screen

#Define colours:
BLACK=(0,0,0)
WHITE=(255,255,255)
RED=(255,0,0)
GREEN =(0,255,0)
BLUE=(0,0,255)
YELLOW  = (255,0,255)

size = (1200,800)
screen = pygame.display.set_mode(size)

running = True

eBox = equipmemtBox((100,150))
#equipment box

frequency = Sliders.Slider((600,700),300,0,100, "Frequency (Hz)")
mu = Sliders.Slider((1040,700),300,0.0001,0.001, "Mass Per Unit Length
(kg/m)")
l = Sliders.Slider((160,700),300,0.5,2, "Length (m)")
#create sliders for each of the independent variables

clock = pygame.time.Clock()
#needed to set the frame rate

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            #checks if window closed
            running = False
    screen.fill(WHITE)



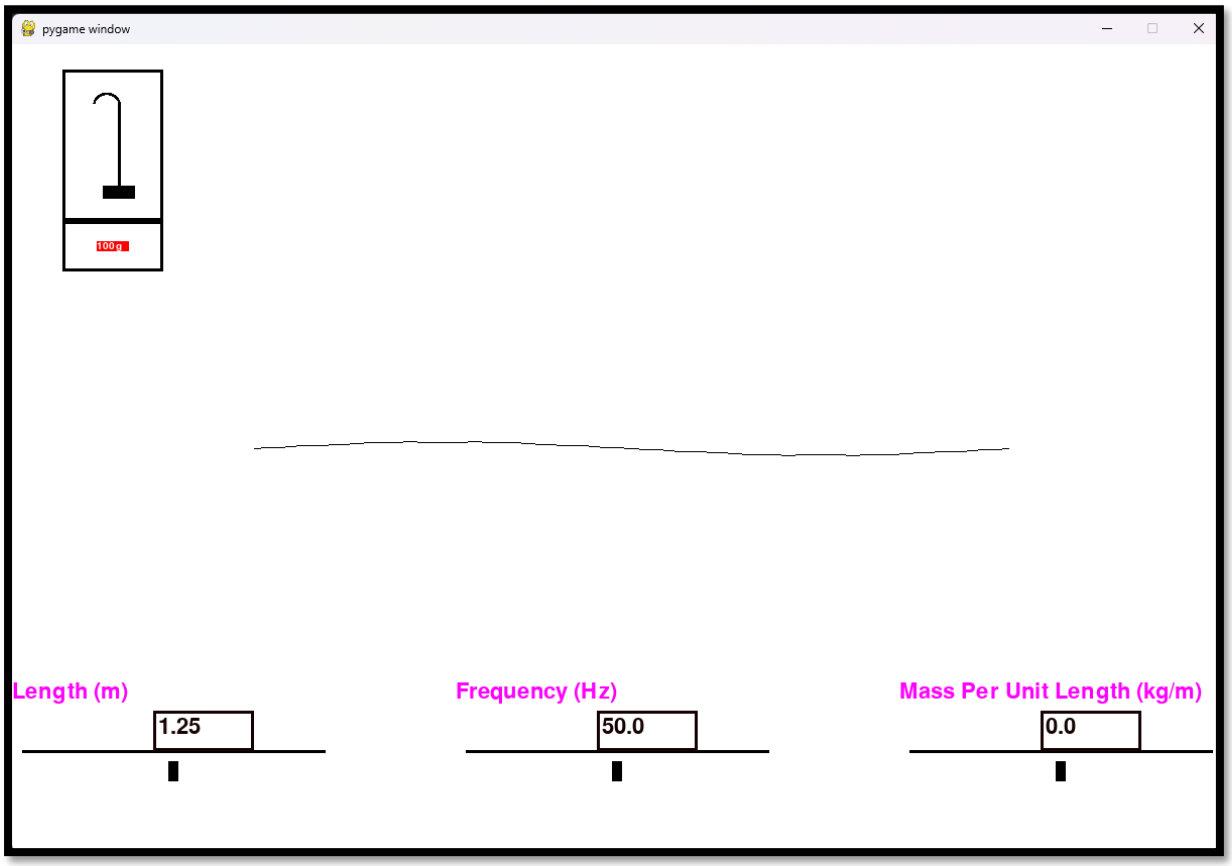
    eBox.display()
    frequency.display(screen)
    mu.display(screen)
    l.display(screen)

    Waves.dispReal(1.96,mu.valueHeld, l.valueHeld, frequency.valueHeld,
500,size,screen)
    clock.tick(60) # sets frame rate to 60 Hz
```

```
    pygame.display.flip()
    #updates screen
pygame.quit()
```

### *Testing*

When this was run, initially it looked good:




One thing to initially note is that the mass per unit length slider is reading 0. This is because I had made this value be rounded to two decimal places when the mass per unit length usually ranges from between 0.0001 and 0.001. To fix this, I could have created a new function that rounds these numbers to 2 significant figures as there is not an in built python function that does this but instead I chose to round all of them to 5 decimal places as this is a much easier fix.

When I tried to move the sliders, they wouldn't move as they previously did when testing the sliders module. The hook also didn't move when dragged and the masses lagged behind where the mouse was.

### *Remedial Action*

This led me to think that there must be a problem with the way each of these objects are moved.

I had developed the code for these classes to include a move method which checks the mouse position and whether the object has been clicked. This means that each frame, for each instance of each class, the events are fetched from PyGame to see where the mouse is and whether it is being clicked.

I looked up the pygame.event.get() function on the PyGame documentation (PyGame, 2025) and it says that the function "will get all the messages and remove them from the queue." Therefore, every time I called this function in one place, there were no events to get in other sections of the code, making most buttons not work except for the ones where the get() method was called first.

To test if this was the problem, I moved all of the move methods out of their classes and into the main loop. This worked and the issue no longer remained.

This left my code not organised in a modular way as lots of processes were occurring within the main section of code. To maintain modularity, I had to rethink previous modules.

I need to make sure that in each frame, the events are only fetched once. Currently, within each move method in each class, the events fetched from user input are iterated through every frame and the move method is called within the display method.

A better way of doing this would be to iterate through the events fetched once in the main loop and check if the input fits the criteria for the items to be moved within the move method in each class.

For example, I changed the move method within the Sliders class from this:

```
def move(self):
    # This procedure changes the pointer location to the position selected
by the user
    for event in pygame.event.get():
        if event.type == MOUSEBUTTONDOWN and
self.pointer.collidepoint(event.pos):
            self.moving = True
        elif event.type ==  MOUSEBUTTONUP:
            self.moving = False
        elif event.type == MOUSEMOTION and self.moving:
            if abs(event.pos[0]-self.pos[0]) < self.length/2:
                self.posPointer[0] = event.pos[0]
            elif event.pos[0] <= self.pos[0] -self.length/2:
                self.posPointer[0] = self.pos[0] -self.length/2
            else:
                self.posPointer[0] = self.pos[0] + self.length/2


            #endif
        #endif
        #self.pointer.move_ip(tuple(self.posPointer))
```

To this:

```
def move(self,event):
    # This procedure changes the pointer location to the position selected
by the user
    if event.type == MOUSEBUTTONDOWN and
self.pointer.collidepoint(event.pos):
        self.moving = True
    elif event.type ==  MOUSEBUTTONUP:
        self.moving = False
    elif event.type == MOUSEMOTION and self.moving:
        if abs(event.pos[0]-self.pos[0]) < self.length/2:
            self.posPointer[0] = event.pos[0]
        elif event.pos[0] <= self.pos[0] -self.length/2:
            self.posPointer[0] = self.pos[0] -self.length/2
        else:
            self.posPointer[0] = self.pos[0] + self.length/2


        #endif
    #endif
    #self.pointer.move_ip(tuple(self.posPointer))
```

As you can see, I removed the for loop and added the parameter event so that it can be accessed within one loop.

I then removed the move call from the display method and placed it in the main loop.

I did the same for the Mass and the MassHook classes.

After doing this, the sliders and other objects worked smoothly and did not have the same problems.

*Review*

This prototype was a partial success bceuase I was able to successfully display the stationary wave and after some bug fixing, use the sliders and the hook all together.

I showed this to Ian, one of my stakeholders and he said that it was not obvious that the sliders were sliders. Therefore, I will go back to the design of usability features to design a new slider.

He also said that he did not understand why the mass hook was in the top left and not at the end of the string. This was already a concern and will be changed in future development.

This is a good start but there is still a lot to be done. This is only a simulator of the realistic view because a new slider needs to be made for choosing a harmonic and a menu needs to be created so the user can select a simulator.

I also showed it to Dante and he said that it looked good and was a good represenatation of the stationary waves practical activity but it

## Main Menu

As stated in the review of the first prototype, a main menu needs to be created to allow users to select what required practical activity and what view of the RP they want.

This is the code for the checkClicked function:

```
def checkClicked(rectObj,event):
    #returns true if rect object is clicked
    if event.type == MOUSEBUTTONDOWN and rectObj.collidepoint(event.pos):
        return True
    else:
        return False
```

As stated in the design section, I have made this into its own function, outside of any class definitions because the same code is being re used in different sections. This increases the modularity of my program.

This is the code for a menu option. A menu option is one of the options that can be clicked in the menu that opens up that simulator.

```
class MenuOption:
    #one of the options on the main menu
    def __init__(self,pos,ID):
        #id is 1 if realistic view is shown
        #id is 2 if abstract view is shown
        height = 300
        width = 400


        self.box = pygame.Rect(pos[0]-width/2,pos[1]-height/2, width,
height)

        if ID == 1:
            imageBig = pygame.image.load("Display Real Example.png")
            self.textSurface =
pygame.font.SysFont("calibri",20).render("RP 01 Stationary Waves -
Realistic View",True, BLACK)
        elif ID == 2:
            imageBig = pygame.image.load("Display Abstract Example.png")
            self.textSurface =
pygame.font.SysFont("calibri",20).render("RP 01 Stationary Waves -
Abstracted View",True, BLACK)


        self.image = pygame.transform.scale(imageBig,(width,height))
    #endProcedure


    def checkOptionClicked(self,event):
        #returns true if box is clicked
        return checkClicked(self.box,event)
    #endFunction
```

```
    def display(self,screen):
        #draws the menu to the screen
        pygame.draw.rect(screen,BLACK,self.box, 3)
        screen.blit(self.image, self.box.topleft)

screen.blit(self.textSurface,(self.box.bottomleft[0],self.box.bottomleft[1]
+20))
    #endProcedure
#endClass
```

In the constructor method, a parameter "ID" is checked to see if it is 1 or 2. This is because the procedure needs to load different images depending on which simulator it represents. I could have created 2 separate classes, one for each simulator that both inherit from one menuOption class but this would not be necessary because there are only two unique attributes for each simulator. It also means that when a new simulator is created, a new class would have to be created.

checkOptionClicked() and display() are almost exactly the same as the pseudocode created in the design section.

### *Testing*

I added this code to test the option classes by themselves. It instantiates an option and displays it until it is clicked. When it is clicked, "clicked" should be output.

```
pygame.init()#initialises pygame screen

size = (600,600)
screen = pygame.display.set_mode(size)

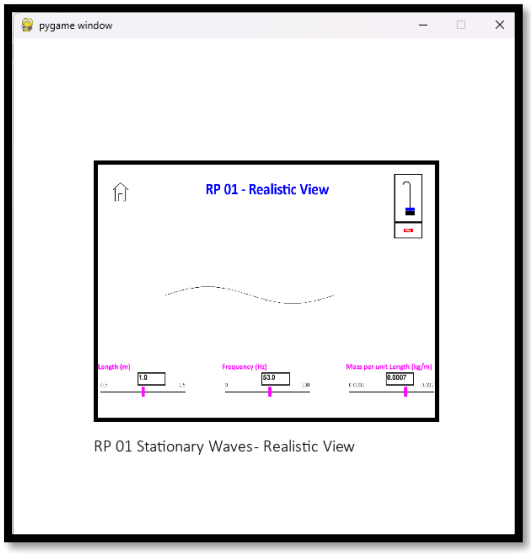
option = MenuOption((300,300),1)
clicked = False #keeps running until the icon is clicked
while not clicked:
    screen.fill((WHITE))
    option.display(screen) #displays option

    events = pygame.event.get()
    for event in events:
        if option.checkOptionClicked(event):
            clicked = True
        #endif
    #next event

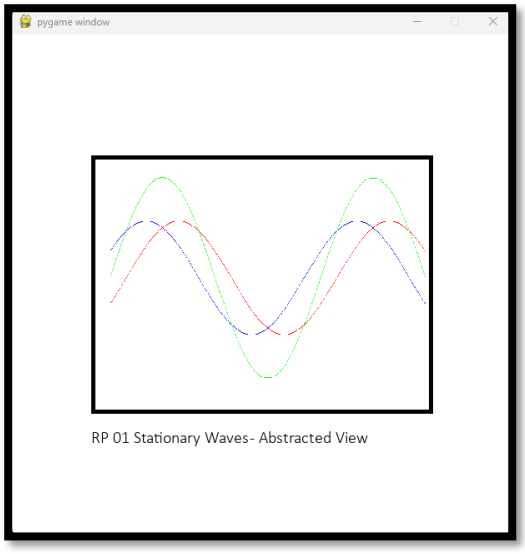
    pygame.display.flip()
print("clicked")
pygame.quit()
```

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **MenuOption – checkOptionClicked.** | Click the option | Normal | "clicked" | "clicked" | Pass |
| **MenuOption – checkOptionClicked.** | Click elsewhere on the screen | Erroneous | Nothing to be printed. | Nothing | Fail |

The image on the left shows the display when the realistic view option is shown and the one on the right shows when the abstracted view option is shown.




RP 01 Stationary Waves- Realistic View




RP 01 Stationary Waves- Abstracted View

*Review*

These options are similar to those designed in the usability features section so should therefore be usable. The testing and images above show that it is functional. Therefore, this module has been a success. Now this is completed, I need to create a home icon that takes you back to the menu page when clicked and I also need to display the menu as a whole, showing two options at the same time which will be developed in the Second Prototype.

## Home Icon

This is the code for the home icon, which again is very similar to the pseudocode created in the design section:

```
class HomeIcon:
    def __init__(self,pos):
        bigImage = pygame.image.load("Home Icon.png")
        width = 60
        height = 60
        self.image = pygame.transform.scale(bigImage,(width,height))
        self.box = pygame.Rect(pos[0]-width/2,pos[1]-height/2, width,
height)
    #endProcedure


    def checkIconClicked(self,event):
        #checks if the icon is clicked
        return checkClicked(self.box,event)
    #endFunction

    def display(self,screen):
        #draws the icon on the screen
        screen.blit(self.image, self.box.topleft)
    #endProcedure
#endClass
```

*Testing*

To test this code, I needed to add code to the main loop. This following code instantiates the home icon, creates a pygame window, displays the icon and continues to run until the icon is clicked. When the icon is clicked, "clicked is output".

```
pygame.init()#initialises pygame screen

size = (1200,800)
screen = pygame.display.set_mode(size)

home = HomeIcon((600,400))
clicked = False #keeps running until the icon is clicked
while not clicked:
    screen.fill((WHITE))
    home.display(screen) #displays icon

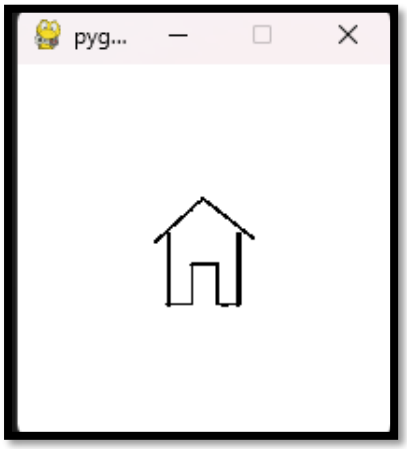
    events = pygame.event.get()

    clicked = home.checkIconClicked(events)

    pygame.display.flip()
print("clicked")
pygame.quit()
```

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **HomeIcon - checkIconClicked** | Click the icon | Normal | "clicked" to be printed in the terminal. | Clicked printed in terminal | Pass |
| **HomeIcon – checkIconClicked** | Click elsewhere on the screen | Erroneous | Nothing to be printed in the terminal. | Nothing happens | Pass |

The following image shows that the icon is displayed correctly.




*Review*

This code functions and the design is copied from usability features so it should be usable. I now need to put all I have done so far together so I can create another prototype.

## Second Prototype

This prototype will include the new sliders and the menu option which were requested after developing the first prototype.

### Menu

I need to find a way of running the correct simulator when the menu is clicked. As designed, I will do this by loading the menu, which would return a 1 or a 2 depending on which option has been clicked and then loading the appropriate simulator from this information.

This is the code that displays the main menu and returns the appropriate value. I did this to fulfil the usability success criteria from the analysis section which says that the users are presented with windows that allow them to choose an RP.

```
def dispMenu(screen, leftOption, rightOption, screenSize):
    #displays the main menu on the screen
    leftOption.display(screen)
    rightOption.display(screen)
    textSurface =  pygame.font.SysFont("calibri",40).render("Physics
Practical Simulator",True, BLUE)
    screen.blit(textSurface,(screenSize[0]/2 -
textSurface.get_width()/2,20) )
#endProcedure


def checkOptions(events,leftOption,rightOption):
    #returns 1 if option 1 is clicked and 2 if option 2 is clicked
    for event in events:
        if leftOption.checkOptionClicked(event):
            return 1
        elif rightOption.checkOptionClicked(event):
            return 2

        #end if
    #next event
#endFunction

def checkEnded(events):
    #returns True if program has been quit
    for event in events:
        if event.type == pygame.QUIT:
            #checks if window is closed

            return True
        else:
            return False
        #endif
    #next event
#endFunction
```

```
def menu(screen,screenSize):
    #displays the main menu and returns 1 if option 1 is clicked and 2 if 2
is clicked
    running = True
    leftOptionCentre = (screenSize[0]/4,screenSize[1]/2)
    leftOption = MenuOption(leftOptionCentre,1)

    rightOptionCentre =  (3*screenSize[0]/4,screenSize[1]/2)
    rightOption = MenuOption(rightOptionCentre, 2)

    clock = pygame.time.Clock()

    while running:
        events = pygame.event.get()

        screen.fill(WHITE)


        clicked = checkOptions(events,leftOption,rightOption)

        if checkEnded(events):
            #checks if program closed
            return False
        elif clicked:
            #checks if an option has been clicked
            return clicked
        else:
            dispMenu(screen,leftOption,rightOption,screenSize)
        #endif
        clock.tick(60)

        pygame.display.flip()

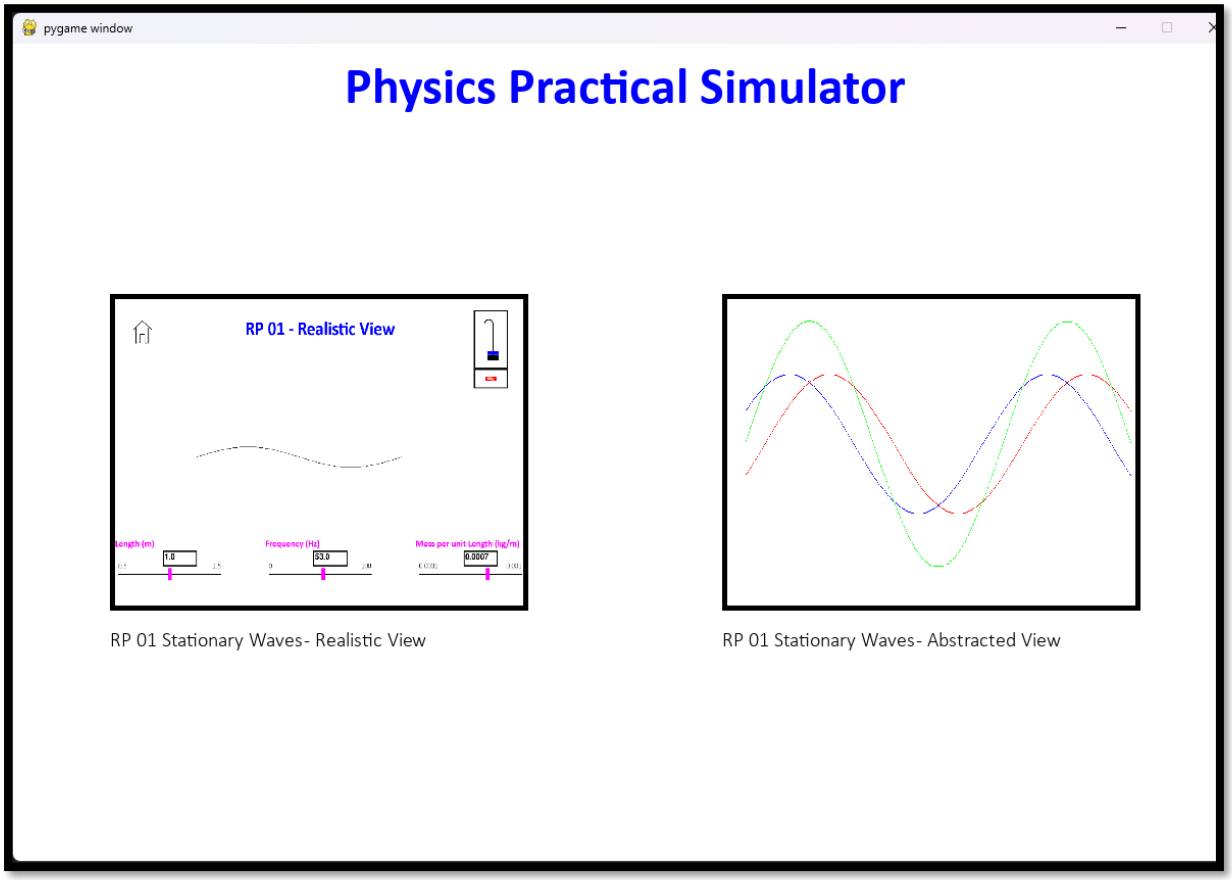
    #endWile
#endFunction

if __name__ == "__main__":
    pygame.init() #initialises pygame screen
    size = (1200,800)
    screen = pygame.display.set_mode(size)
    option = menu(screen,size) #calls the menu to be loaded
    print(option)
    pygame.quit()
```

The procedure “menu” instantiates the option objects, then enters a loop which is run until an option is selected or the program is quit. To check these conditions, it calls the checkEnded and checkOptions functions. I could have included the code to display the menu within this menu function but I decided to make it its own procedure to improve modularity.

*Testing*

When the menu function is called, this is the window that is displayed:




RP 01 Stationary Waves- Realistic View



RP 01 Stationary Waves- Abstracted View

To test this code, I decided to print the option that was selected so I can test that this always returns the correct value before moving on to develop other code.

This is very similar to the design for the main menu created in Usability Features. I decided to use an image of the whole display within the realistic view and just the waves for the abstracted view to emphasise the difference between the two.

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **menu** | Click left option | Normal | 1 printed to terminal | 1 printed to terminal | Pass |
| **menu** | Click right option | Normal | 2 printed to terminal | 2 printed to terminal | Pass |
| **menu** | Click somewhere else on the screen | Erroneous | Program continues to run | Program continues to run | Pass |
| **menu** | Click x in top right | Normal | Program terminated | Program continues to run | Fail |

*Remedial Action*

When the x in the top right was clicked, the program did not stop. This must be a problem within the checkEnded function.

Tracing through the checkEnded function, I found that it would return False if the first event in the events fetched was quit but it can be the second or further down the list. I changed this function so it returned False only after the for loop was finished:

```
def checkEnded(events):
    #returns True if program has been quit
    for event in events:
        if event.type == pygame.QUIT:
            #checks if window is closed

            return True
        #end if

    return False
    #next event
#endFunction
```

When the x is clicked now, the program ends and False is printed. Therefore, this module has been a success.

While the code runs correctly, it can be made more structured and modular. I noticed that the menu function was locating where the options should be and instantiating them as well as checking if they were clicked. This would work better as two separate sub modules. Therefore, I created a new function that instantiates the options in the correct place and returns them:

```
def createMenu(screenSize):
    #displays the main menu with 2 options
    leftCentre = (screenSize[0]/4,screenSize[1]/2)
    rightCentre = (3*screenSize[0]/4,screenSize[1]/2)
    Left = MenuOption(leftCentre,1)
    Right = MenuOption(rightCentre,2)
    return Left,Right
#endFunction
```

After replacing the lines in the menu function that have the same effect with calling this function, it produced the same output.

*Review*

This module was a success: I was able to display the menu. I showed this to Ian and he said although it functions, it looks plain. Therefore, in future development, I will change the background and colours to make it more aesthetically pleasing. I am now going to develop the modules that allow the simulators to be displayed.

## RP 01 – Realistic View

I needed to create code that would put all of the other modules together and display what is appropriate for the realistic view of the stationary waves.

To keep my code well structured, I split it up into 3 modules as defined in the design section.

This is my code:

```
def createRP1Real(screenSize):
    #instantiates and returns sliders, mass hook, ebox and title text

    f = Sliders.Slider((screenSize[0]/2,700),300,0,100,"Frequency (Hz)")
    mu = Sliders.Slider((screenSize[0]-160,700),300,0.0001,0.001, "Mass per
unit Length (kg/m)")
    l = Sliders.Slider((160,700),300,0.5,2,"Length (m)")

    eBox = Masses.EquipmentBox((screenSize[0]-100,150))

    homeIcon = Menu.HomeIcon((80,80))

    text = pygame.font.SysFont("calibri",32,bold = True)
    textSurface = text.render("RP 01 - Realistic View", True, BLUE)

    return [mu,l,f], eBox, textSurface, homeIcon
#endFunction

def displayRP1Real(screen, screenSize,sliders, eBox, textSurface,
homeIcon):
    #displays the Rp 01 Screen
    for slider in sliders:
        slider.display(screen)

    eBox.display(screen)
    homeIcon.display(screen)

    T = eBox.getHook().getTension()

    mu = sliders[0].valueHeld
    l = sliders[1].valueHeld
    f = sliders[2].valueHeld

    Waves.dispReal(T,mu,l,f,100,screenSize, screen)

    screen.blit(textSurface, (screenSize[0]/2-textSurface.get_width()/2,
100))
#endProcedure
```

```
def RP1Real(screen,screenSize):
    #the main code that is run when RP 1 realistic is displayed
    #returns False if quit and true if menu option clicked

    running = True

    sliders, eBox, textSurface, homeIcon = createRP1Real(screenSize)

    clock = pygame.time.Clock()

    while running:
        screen.fill(WHITE)

        events = pygame.event.get()
        #gets mouse events

        if Menu.checkEnded(events):
            running = False
            #checks if program closed
            return False
        elif homeIcon.checkIconClicked(events):
            return True
        else:
            for slider in sliders:
                slider.move(events)
                slider.editValue(events)
                eBox.checkInput(events) #allows masses to be moved

        #endif
        displayRP1Real(screen, screenSize,sliders,eBox, textSurface,
homeIcon)

        clock.tick(60)
        pygame.display.flip()
    #endWhile
#endProcedure
```

Within the RP1Real function, I could have iterated through each of the events and then done slider.move and slider.editValue on these but instead, I changed the slider.move and editValue methods so that they accept the list of all of the events and then from there check if one of them meets the criteria for the pointer to be moved. Not only does this make this function more readable but it also makes it more efficient as the slider.move is only called once every frame and not the number of events per frame.

I also had to add a checkInput method in the EquipmentBox class which allows any of the equipment in the box to be moved.

### RP 01 Abstracted View

This is the code for the abstracted view which runs very similarly to the sub modules just discussed above.

```
def createRP1Abstract(screenSize):
    #instantiates and returns sliders, mass hook, ebox and title text

    multiplier = Sliders.Slider((screenSize[0]/2,700),300,0.0001,0.1,
"Speed Multiplier")
    harmonic = Sliders.discSlider((600,200),300,1,5, "Harmonic Number")
    mu = Sliders.Slider((screenSize[0]-160,700),300,0.0001,0.001, "Mass per
unit Length (kg/m)")
    l = Sliders.Slider((160,700),300,0.5,2,"Length (m)")

    eBox = Masses.EquipmentBox((screenSize[0]-100,150))

    homeIcon = Menu.HomeIcon((80,80))

    text = pygame.font.SysFont("calibri",50,bold = True)
    textSurface = text.render("RP 01 - Abstract View", True, BLUE)

    return [mu,l,harmonic,multiplier], eBox,textSurface, homeIcon
#endFunction

def displayRP1Abstract(screen,screenSize, sliders, eBox, textSurface,
homeIcon):
    #displays the Rp 01 Screen
    for slider in sliders:
        slider.display(screen)
    eBox.display(screen)
    homeIcon.display(screen)

    T = eBox.getHook().getTension()

    mu = sliders[0].valueHeld
    l = sliders[1].valueHeld
    harmonic = sliders[2].valueHeld
    multiplier = sliders[3].valueHeld

    displayFreq(screen,harmonic, l, T, mu)

    Waves.dispAbtract(T,mu,l,harmonic,50,screenSize,screen, multiplier)
    screen.blit(textSurface, (screenSize[0]/2-textSurface.get_width()/2,
50))
#endProcedure
```

```
def rp1(screen,screenSize, view):
    #the main code that is run when RP 1 realistic is displayed
    #returns False if quit and true if menu option clicked

    running = True

    if view == 1:
        sliders, eBox, textSurface, homeIcon = createRP1Real(screenSize)
    elif view == 2:
        sliders, eBox, textSurface, homeIcon =
createRP1Abstract(screenSize)
    #endif


    clock = pygame.time.Clock()

    while running:
        screen.fill(WHITE)

        events = pygame.event.get()
        #gets mouse events

        if Menu.checkEnded(events):
            running = False
            return False
        elif homeIcon.checkIconClicked(events):
            return True
        else:
            for slider in sliders:
                slider.move(events)
                slider.editValue(events)
            eBox.checkInput(events) #allows masses to be moved

        #endif

        if view == 1:
            displayRP1Real(screen, screenSize,sliders,eBox, textSurface,
homeIcon)
        else:
            displayRP1Abstract(screen, screenSize, sliders, eBox,
textSurface, homeIcon)
        #endif


        clock.tick(60)
        pygame.display.flip()
    #endWhile
#endFunction
```

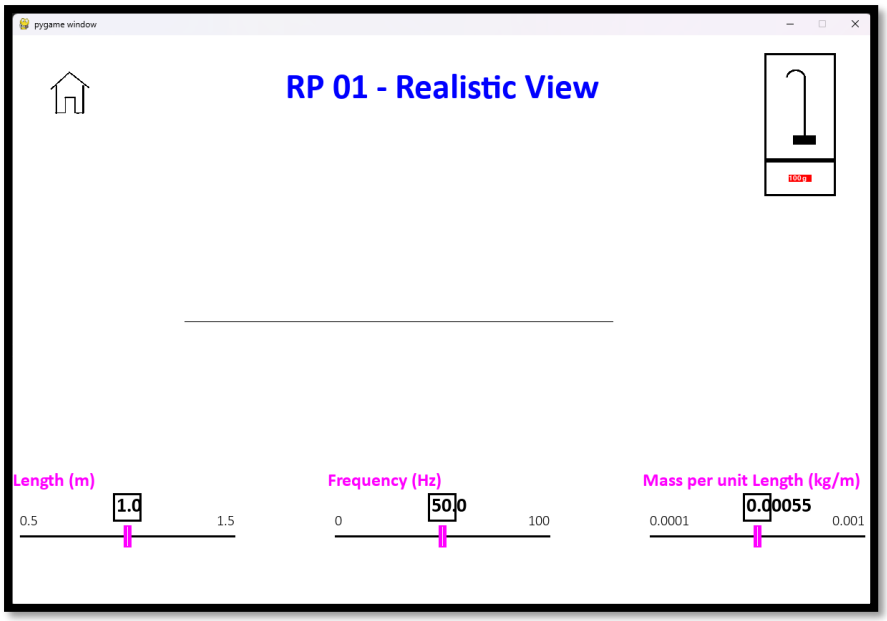
As discussed in Design, I changed RP1Real to just rp1 and edited the code so that it would work for both views of the stationary waves. I did this because I noticed that essentially they performed the same function

*Testing*

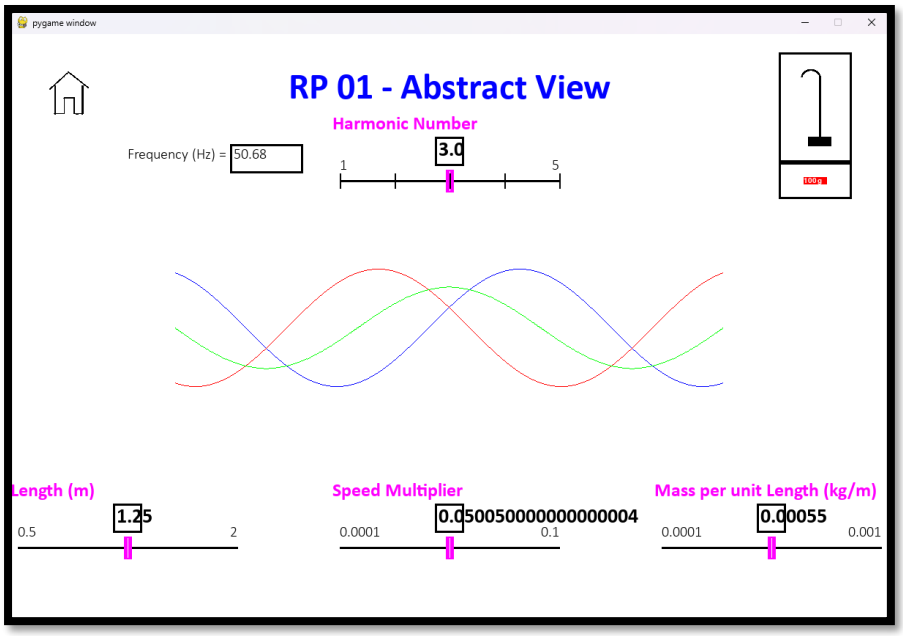
| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **createRP1Real,** | screenSize = (1000,500) | Normal | Should return appropriate sliders etc. | ([<Sliders.Slider object at 0x000001E5DA22F6C8>, <Sliders.Slider object at 0x000001E5DA22F988>, <Sliders.Slider object at 0x000001E5C6F8BF88>], <Masses.EquipmentBox object at 0x000001E5DA22F948>, <Surface(438x50x32 SW)>, <Menu.HomeIcon object at 0x000001E5DA22FCC8>) | Pass – This output shows three sliders in a list followed by an equipment box, a text surface and a home icon. |
| **createRP1Abstract** | screenSize = (1000,500) | Normal | Should return appropriate sliders etc | ([<Sliders.Slider object at 0x000001E2B7CCFA08>, <Sliders.Slider object at 0x000001E2B7CCFB88>, <Sliders.discSlider object at 0x000001E2B7CCF248>, <Sliders.Slider object at 0x000001E2A8F9BFC8>], <Masses.EquipmentBox object at 0x000001E2B7CCFD48>, <Surface(440x50x32 SW)>, <Menu.HomeIcon object at 0x000001E2B7CCFF08>) | Pass – 4 sliders, one equipment box, one text surface and a home icon returned. |
| **displayRP1Real** | *N/A* | *Normal* | Should display everything in the correct position but nothing able to move. | See screen grab on next page.<br>Everything is displayed in the correct position but can't be moved. The boxes on the sliders are small but they should increase in size once they are allowed to be edited. | Pass |

| Module | Test Data | Test Type | Expected Output | Output | Pass or Fail |
|---|---|---|---|---|---|
| **displayRP1Abstract** | N/A | Normal | Should display everything in the correct position but nothing able to move. | See screen grab on next page. Everything is displayed in the correct position but can't be moved. The boxes on the sliders are small but they should increase in size once they are allowed to be edited. | Pass |
| **Rp1** | View = realistic | Normal | The correct simulator displayed and sliders are able to be adjusted. | Everything is displayed as expected and moves as expected. When the x in the top right is clicked, it returns false and when the home icon is clicked, it returns true | Pass |
| **Rp1** | View = abstracted | normal | The correct simulator displayed and sliders are able to be adjusted. | Everything is displayed as expected and moves as expected. When the x in the top right is clicked, it returns false and when the home icon is clicked, it returns true | Pass |

### *displayRP1Real Output*




### *displayRP1Abstract Output*




### *Review*

In this module, I have created multiple submodules that put code together for each of the simulators and the program as a whole.

All there is to do now is add code to the main file that brings all of these submodules together.

This will be my final prototype as I do not have enough time within the given timeframe to produce another so this is the prototype that will undergo post-development testing.

To make all of my code work together, I added this to the end of my python document:

```
pygame.init()
screenSize = (1200,800)
screen = pygame.display.set_mode(screenSize)

option = True
while option:
    option = Menu.menu(screen,screenSize) # returns 1 if option 1 clicked
and 2 if option 2 clicked
    if option:
        option = rp1(screen,screenSize,option) # returns False if quit and
True if menu icon clicked
#endWhile

pygame.quit() #closes pygame tab
```

This loops between the menu and rp1 until quit is hit and option becomes false. I had to add the if statement because if exit is hit in the menu, False would be passed into rp1 and an error would occur.

# 4. Evaluation

## Post Development testing

The program was tested by a run through conducted by myself which was screen recorded (see 3056_Easton_Dominic_PhysicsSimulator_TestingVideo.mp4) and 3 different stakeholder run throughs done by each of my stakeholders.

### Ian run through

Ian used the abstracted view of the required practical activity to give himself a better understanding of the Physics behind the practical activity.

After being allowed to use the abstracted view alone he had the following comments:

- "The 2 smaller waves being shown alongside the stationary wave formed gave me an understanding of how stationary waves work"
- "It needs to be made clearer as to what the blue and red waves are."
- "The sliders are intuitive and easy to adjust"
- "I am not sure why the hook is at the top right"
- "I like the variety of colours used"
- "It is clear what each of the sliders do and I can see their effects. It helped me understand their effects in Physics"

### Dante Run Through

Dante used the realistic view of the required practical activity to remind him of how the practical activity is performed.

He had these comments:

- "It looks very similar to what I remember from doing the practical myself."
- "It is obvious what the sliders do and how to add mass to the hook."
- "It reminded me of what I did when performing the practical in real life."
- "The home icon's function is obvious"
- "Why is the hook in the top right?"

### Mr Moran Run through

Mr Moran is my Physics teacher and I needed him to check that the Physics in the simulator is all correct. He opened both simulators, changed values using the sliders and checked the program with calculations on his calculator. He said that everything ran as he expected to.

## Functionality Testing

From these tests, I can see if my success criteria have been met.

This is the functionality section of my success criteria from the analysis section copied and pasted. There is a comment next to each test showing whether it was a success or a failure. Where relevant, a reference to the evidence used to evaluate whether it was a success or failure is included. Below the table, there will be an explanation of how this evidence means that the success criteria has been met or not

| Test Number | Success criteria - Functionality | Comment – Success or Failure | Evidence if applicable |
|---|---|---|---|
| 1 | Users sign in with username and password. Users have to enter there password twice in set up. | Failure | N/A |
| 2 | A window in the corner of the RP allows users to change speed settings and have two snapshots side by side | Partial Success | Testing video 3:20 |
| 3 | The practical activity can be set up by dragging equipment into its desired location from a window containing all equipment. | Partial Success | Testing video 1:26 |
| 4 | Each user has a profile which saves their progress in questions and results from performing RPs. | Failure | N/A |
| 5 | The simulator should replicate how the practical activity looks in real life. | Partial Success | Dante and Mr Moran interviews. |
| 6 | The simulator should help a student better understand the Physics behind a Physics required practical activity. | Success | Ian interview |
| 7 | The simulator should help a student who has missed a required practical lesson or has forgotten the lesson get experience of performing the practical activity. | Success | Dante interview |
| 8 | Systematic Errors can be added to measuring equipment and can be randomised. | Failure | N/A |
| 9 | The simulator should follow the laws of physics and match the equations that students learn in A-level Physics. | Success | Testing video 0:15 – 2:50, Mr Moran interview |
| 10 | System should load within 5 seconds of opening the program | Success | Not included in video but it would regularly take approximately 3 seconds to load. |
| 11 | Each simulator should load within 10 second of being selected | Success | Testing video – 0:15, 2:54 |
| 12 | Effects of a change in a variable should be shown within 1 second of adjusting it. | Success | Testing video – 3:01, 3:10, 3:20 |
| 13 | When running the simulators, they should move smoothly and not freeze. | Success | Testing video – whole video |
| 14 | The program should be a viable model for future development | Determined in maintainability | Code review |

Test 1 and 4– Failure

I have not included a user sign in because there is no data to save. This will only be developed once I have also developed a feature that allows graphs and tables to be created from results. This success criterion could be met in the future by creating a database that stores the users results alongside

their unique identifier which would be their username. Their password could also be hashed to increase security.

Test 2

The speed of the waves can be adjusted relative to the real speed that the waves move at in the abstracted view of the stationary waves. However, there is no way of getting two screens showing the same simulator simultaneously so this section has failed. This could be done in future development by calling the dispRP1 function twice in the same loop. This idea will be explored further in the limitations section.

Test 3

The testing video shows that the mass hook can be clicked and dragged around the screen but there is nothing checking that it is in the correct position, at the end of the string and it does not stay there. There is also no other equipment in the simulator so no other equipment can be setup. How this will be developed in the future is explored in the limitations section.

Test 5

Dante said that the waves reminded him of how it looked it real life and Mr Moran said the same. However, this is only a partial success because the equipment that is there in real life like the vibration generator are not there in the simulator. This will be improved in the future by creating new classes for more pieces of equipment.

Test 6

Ian said that the abstracted view gave him a better understanding of how stationary waves work. This means that test 6 was a success.

Test 7

Dante said that the realistic view reminded him of what it was like in real life. This means that my solution has solved the problem set out at the beginning of analysis.

Test 8

See limitations .

Test 9

In the test data for post-development testing in my design section, I wrote the expected values for each of the values input in the testing video. The testing video shows that the program met these expected values every time. To check that I wasn't performing the calculations by hand wrong as well as in the program, I showed the simulator to my Physics teacher, Mr Moran and he said that the simulator acted as he expected so the simulator does follow the laws of Physics and this criterion is a success.

Test 11

As shown in the video, the simulators opened almost instantly so this was a success. However, the first simulator does occasionally have a minor delay to open. This can be fixed in the future by introducing pipelining and loading the simulators while the menu is still open.

Test 12

3 minutes and 1 second into the video, the harmonic slider is clicked and the pointer immediately moves to the point where it is clicked and the stationary waves shown immediately change. 3 minutes and 10 seconds into the video, I show that the value of the sliders can be edited by typing in a value and the effect of it is instantaneous. 3 minutes 20 seconds into the video, the multiplier slider is dragged and it immediately changes the speed of the waves. As editing the value, clicking along the line and dragging the pointer all have an immediate effect on the program, this test is a success.

Test 13

There were no latency or freezing issues in the video so this test was a success.

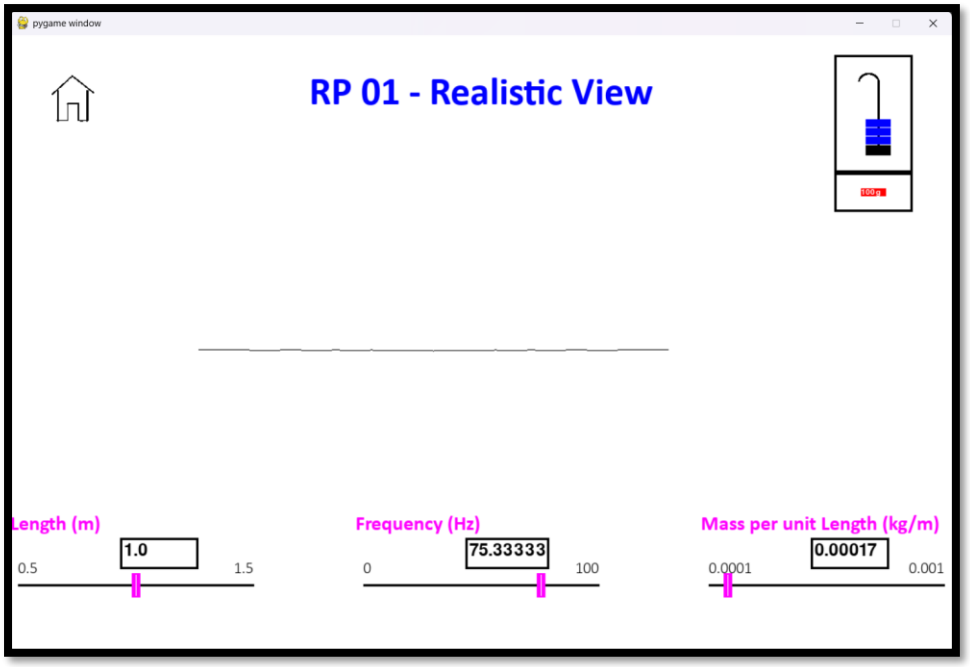
## Robustness Testing

| Test Number | Success Criteria - Robustness | Success or failure | Evidence if applicable |
|---|---|---|---|
| 15 | **The password is checked to be 6 characters long and is covered up by *s.** | Failure | N/A |
| 16 | **A required practical activity simulator should be able to be re-loaded and started from the beginning.** | Success | This was not included in the video but screengrabs show below. |
| 17 | **The program should deliver correct outputs every time.** | Success | Testing video/ User Feedback |
| 18 | **The program should allow for two separate simulations to be run sequentially without error.** | Success | Testing video – 2:55 |

Test 15

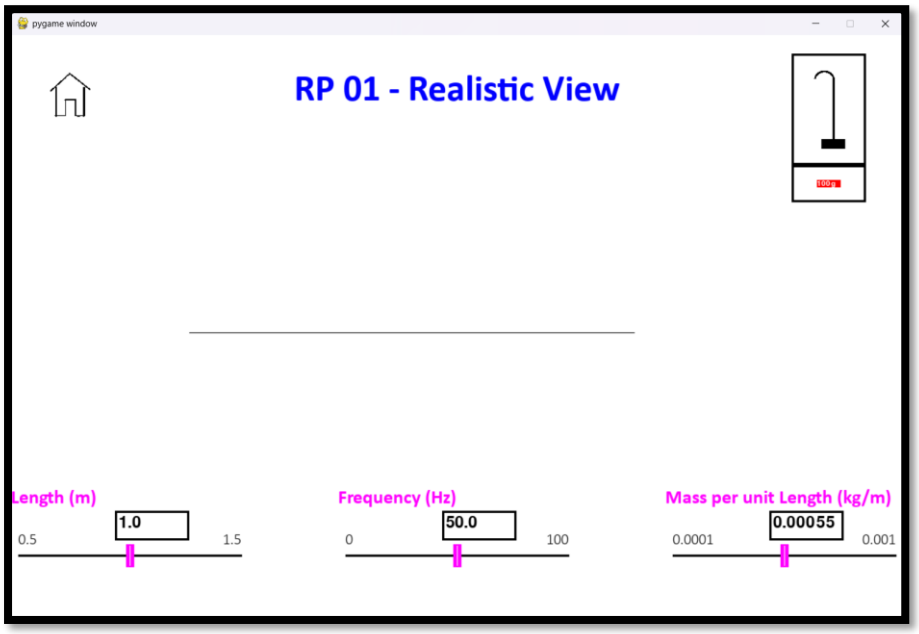
User sign in is not included as stated in the functional testing section.

Test 16:




This is a still of the realistic view open and sliders adjusted.

The home icon was then clicked and the realistic view was opened again. This is what it looked like when reopened:




As you can see, all sliders have returned to the middle and the hook is empty and the program still functions. This means that the simulator is robust because the same simulator can be opened multiple times without failure.

Test 17

For all the values tested in the testing video, the output matched the expected output every time. Additionally, Mr Moran tried multiple values and to him they were correct. Therefore this criterion is a success and the solution is robust because it works every time and not just once.

Test 18

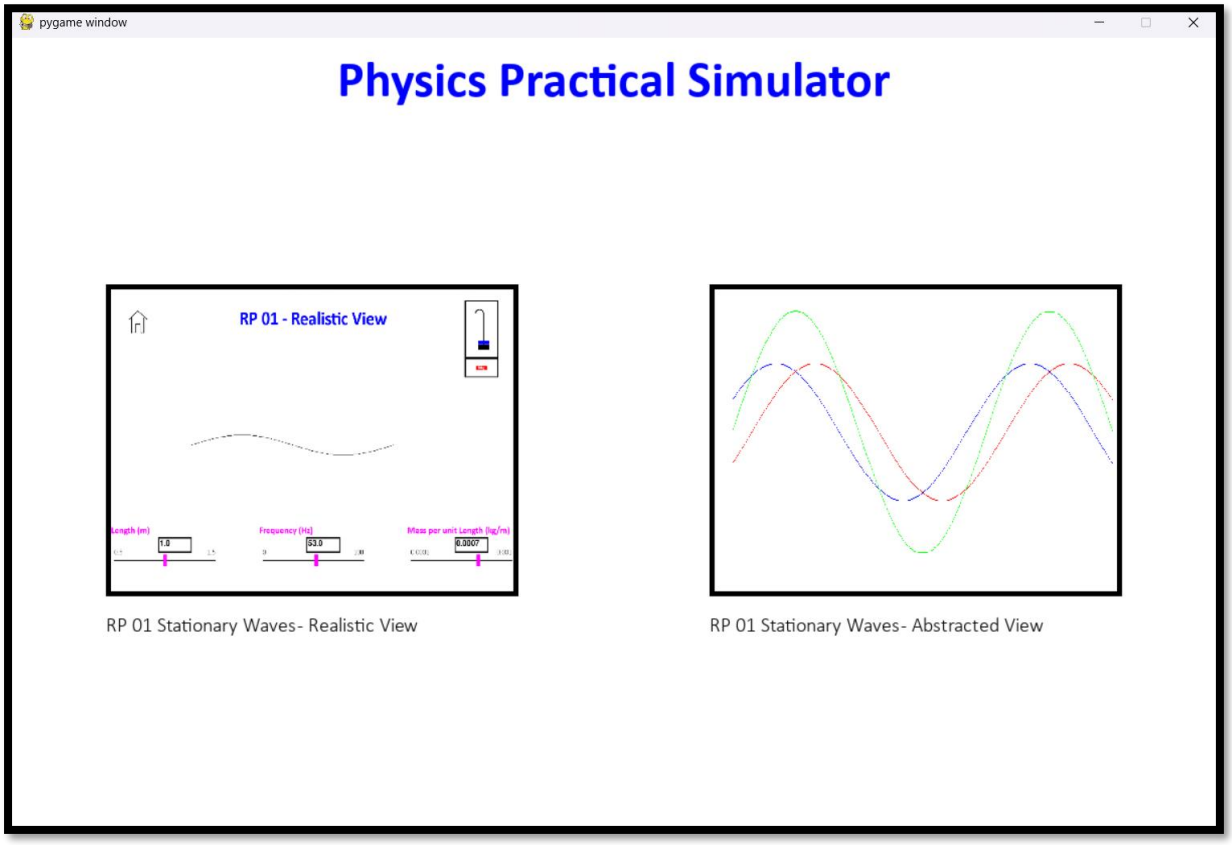
At 2:55 , the abstracted view is opened after the realistic view is opened and closed and it still matches the expected output created in the design section. This means that simulations can be opened sequentially without error so this test has passed.

## Usability Testing /Usability Features

The usability is how intuitive and easy it is to use the program. I have again copied the success criteria and added a comment on whether it passed or failed. In this section

| Test Number | Success Criteria - Usability | Success or Failure | Evidence if applicable |
|---|---|---|---|
| **19** | The user is presented with a menu that allows the user to select a simulator. This should be intuitive and easy to use. | Success | Testing video 0:00-0:15 / User feedback |
| **20** | A home icon is on every page which allows users to return to the home page to select another simulator. The function of the icon should be obvious by the way it looks. | Success | Testing video/ user feedback |
| **21** | An undo button is available on each RP page so that changes to variables can be reversed. It should be obvious from first glance what this button does. | Failure | |
| **22** | The user interface should exaggerate colours of equipment in the RPs. | Success | Testing video/ user feedback. |
| **23** | The user interface should be intuitive and users should understand what certain components do. For example, they should be able to understand what a slider does just from a glance. | Partial Success | User feedback |
| **24** | Equipment should be instantly recognisable | Partial Success | User Feedback |

## Menu – Test 19




RP 01 Stationary Waves- Realistic View



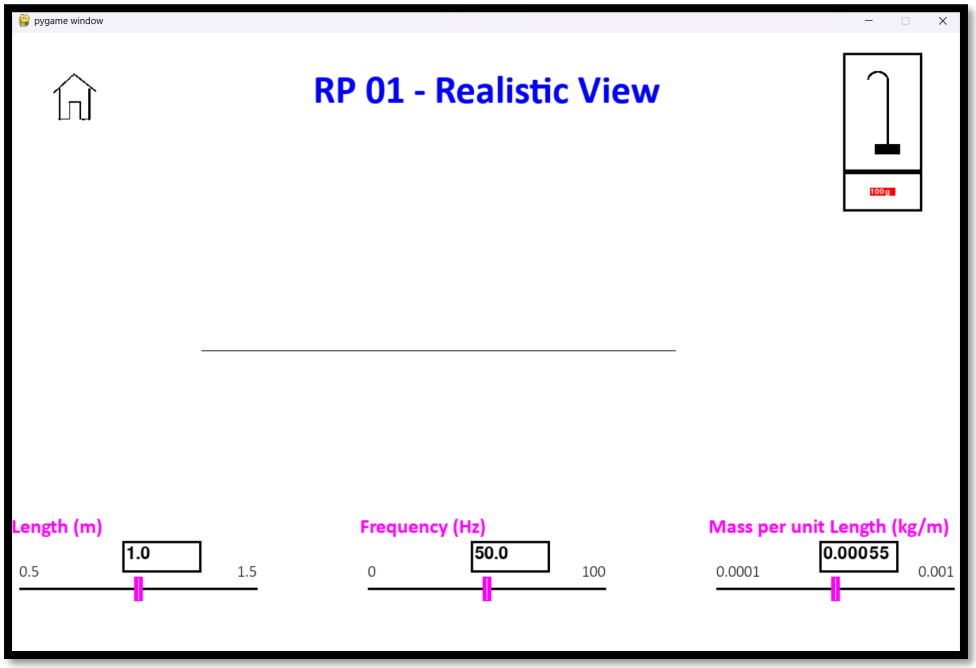
RP 01 Stationary Waves- Abstracted View

This is a screenshot of the main menu in my program. This is very similar to the one designed in the design section. When I asked Ian if this menu was clear he said:

“The pictures help me know which simulator is which and it is clear what I must do to open each simulator”

This usability feature is therefore a success and helps the program meet the usability section of the success criteria. Therefore, test number 19 in my post-development table is a success.

## Test 20




As you can see in this image of the realistic view simulator, there is the home icon in the top left of the screen which is the same image as the one created in the design section. As stated earlier, Dante said that the function of that icon was obvious. This means that Usability test number 20 is a success.

## Test 21

An undo button was not included as it was only a desirable success criterion and I had a limited timeframe to complete this project. How it will be developed in the future will be included in the limitations section of my evaluation.

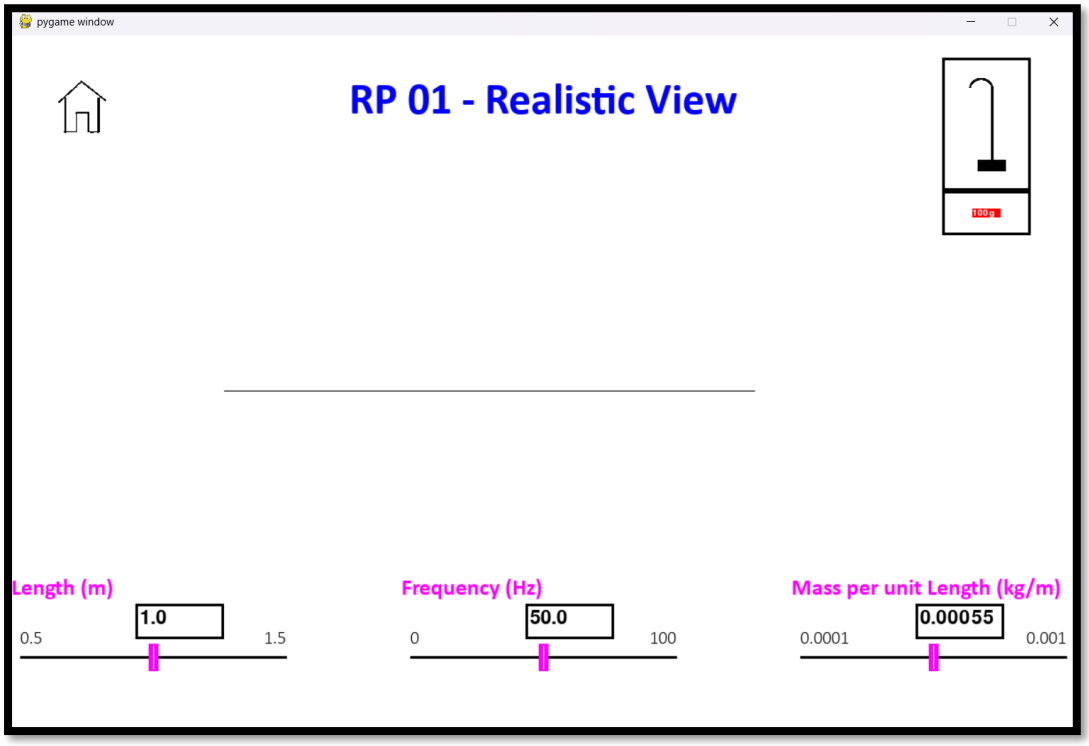
## Test 22

As seen in the testing video at 1:20, the masses added to the hook change colour from blue to red. This is an abstraction from reality as they do not change colour in real life but was included to exaggerate colours in order to make it clearer to the user that masses have been added. Dante said that this was a nice touch. The abstracted view simulator has different colours for each of the waves which allows the user to differentiate between them. This is all evidence to suggest that test 22 was a success.

On the other hand, Ian said that it may make the simulator more appealing if the background was something other than pure white. This can be done easily in future development by changing the fill(WHITE) procedure with another colour or with another procedure that draws the background onto the screen.

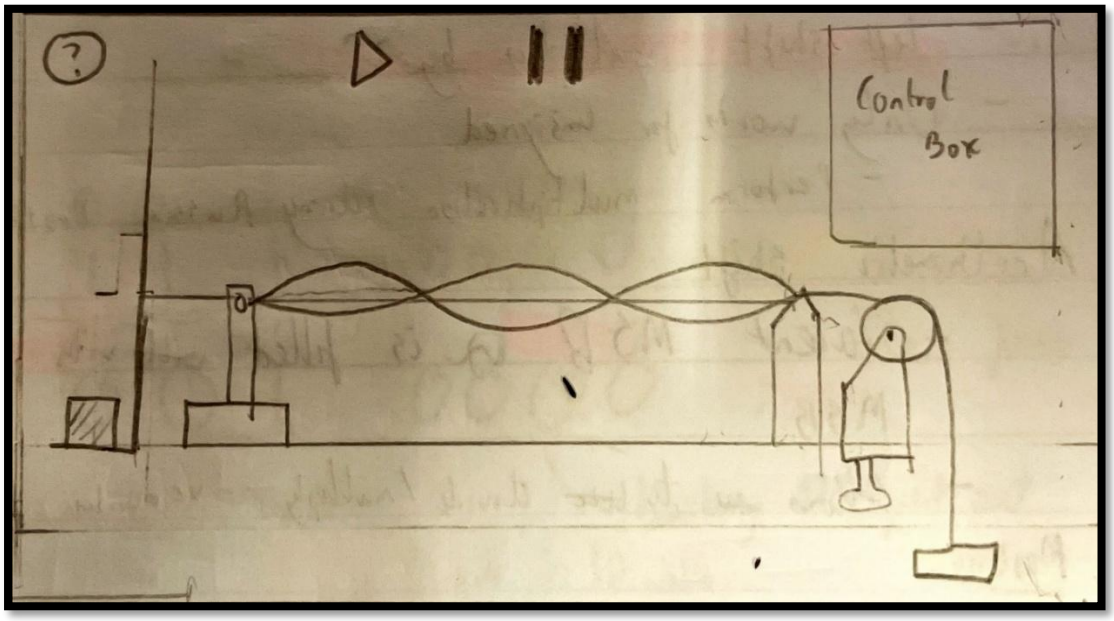
This is only a minor point in what is a mostly successful test so test 22 was a success overall.

## Test 23




This is a screenshot of the realistic view simulator.

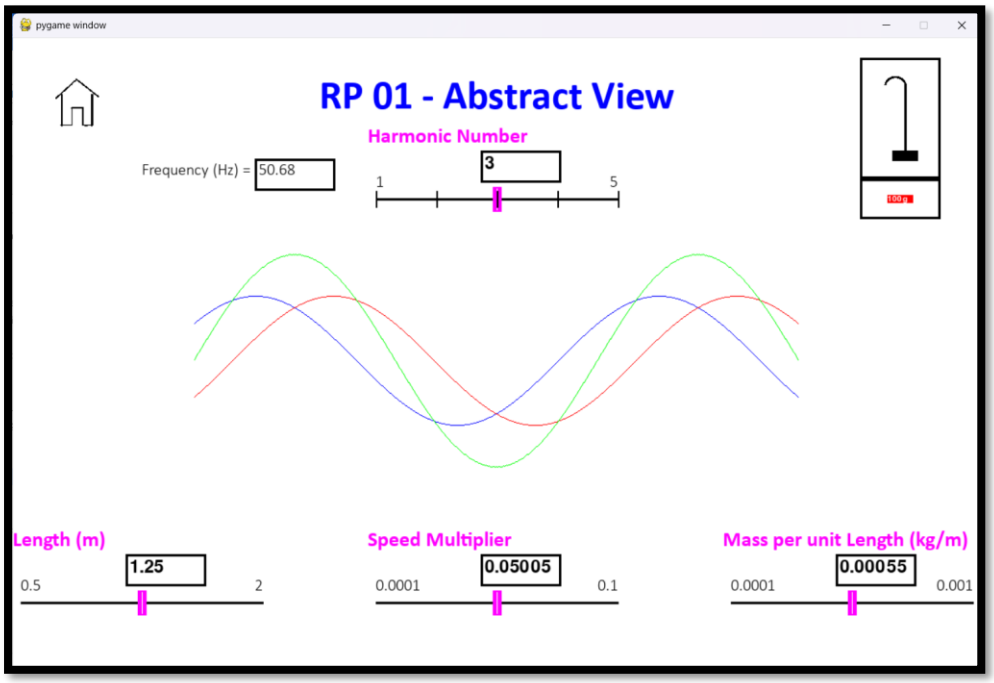
This is the design of this realistic view I created in the Usability Features section of my Design:




As you can see there are some similarities in that the stationary wave is shown in the middle at full speed and there is a control/ equipment box in the top right for both. However, I have not included the equipment like the clamp stand and vibration generator shown in this diagram in my final solution. This is because I had a limited timeframe and had to focus on the important parts of my solution like the waves themselves. This will, however, be done in future development.

"The fact that the waves are positioned in the centre of the screen and sliders are around the edges makes the interface clearer" said Dante. This is evidence to support test 23 being a success.

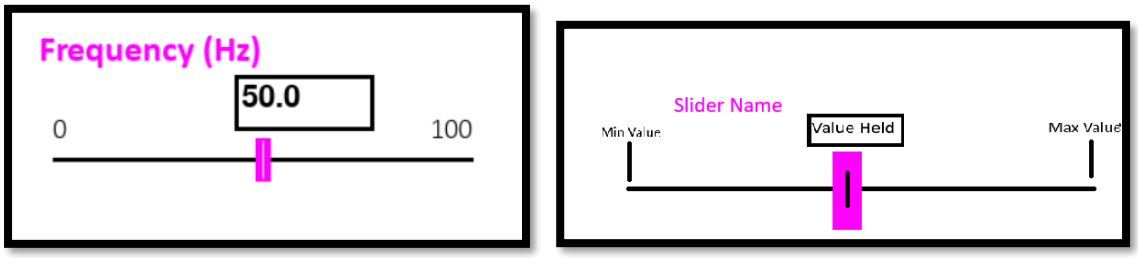
### *Abstract View*




This is a screenshot of the Abstract View.

When looking at this simulator, Ian said that it was not immediately obvious what the green, red and blue waves represent. This means that the system is less intuitive than desired. To fix this problem in the future, I will add a key on the side of the screen that shows which colour means what.

### *Sliders*




The image above, to the left shows the slider user in my final solution and the one on the right is the one designed in my design section after Ian told me that the old slider did not look like a slider. The two images are very similar and Ian still agrees that both are instantly recognisable as a slider.

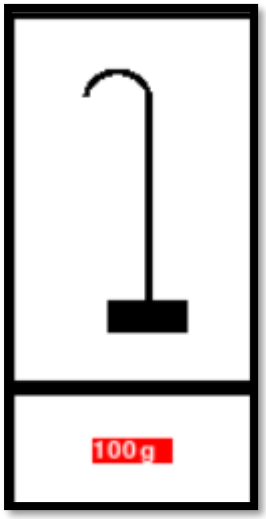
This is evidence for test 23 being a success because this is instantly recognisable.

However, after going through post-development testing with Ian, he said that it would be clearer if the sliders were separated from the waves by putting a box around them. I decided not to do this previously because it would add clutter to the screen but this may have to be reconsidered to meet stakeholder demands. This can easily be done by adding a draw rectangle procedure to the dispRP1 procedure.

Overall, test 23 was a partial success because most of it is clear but there are a couple of things like a key and a box around sliders that need to be added in future development.

### Test 24

The only equipment used was the mass hook and the masses:




Both Ian and Dante agreed that they are both instantly recognisable and the 100g label on the mass helps identify it as a mass.

However, they both did not like how it was always positioned in the top right corner of the screen rather than at the end of the string and this can be confusing for the user.

The reason why I developed it like that was to allow for future development of the setup of the practical activities so the hook can be dragged into position. How this will be developed in the future will be included in the limitations section of my evaluation.

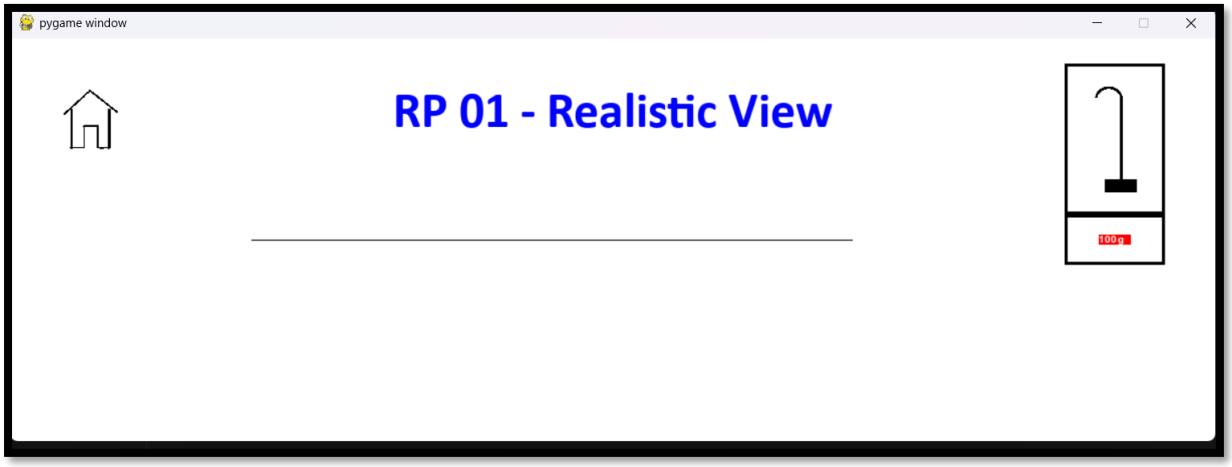
Overall, this test was a partial success because the visual aspect of the equipment is recognisable but they are not positioned sensibly.

# Limitations of the Solution

## Functionality Limitations

### *Display two simulations simultaneously*

This is one of the success criteria I have not met but I would like to allow users to experiment with two simulations side by side. This could be done in future development by creating a new procedure that displays two side by side and it calls the createRP1Real twice and dispRP1Real twice every frame but both with smaller screen sizes. I halved the constant SCREEN_SIZE to test if when this constant is changed, everything is still in the correct position and this is what was displayed:




As you can see, the sliders are not displayed on this screen but the stationary wave is still in the middle of the screen. This shows that the wave has been developed well as its position depends on the SCREEN_SIZE constant so the code is easily maintainable if the screen size needs to be changed. However, the position of the sliders has not been developed as well and have a fixed position no matter the size of the screen. This means that if two simultaneous simulations were to be developed in the future, the procedure that instantiates the sliders in their position would have to be re-written to use the SCREEN_SIZE constant instead of fixed values.

### *Equipment Setup*

I have not developed a setup of the equipment module. In my analysis, I said that it was important that users are able to setup the equipment themselves to get a true lab-like experience. However, I have only created a hook that can be moved but there is nothing checking that it is in the correct position.

It should be easy in further development to change this and create a new module that checks that the hook is in the correct position. My code for the MassHook class is already ready for this to be added because it has the auto setup attribute which when True means that the hook cannot be moved. This means that no parts of the MassHook class will have to be re-programmed.

Equipment other than the mass hook has not been developed yet.

### *Systematic Errors*

Systematic errors have not been added to the program despite this being part of my success criteria. This is because there are not many errors that occur in the required practical activity one which is the only one that was developed. However, I could add errors to the frequency generator/slider in future development.

In future development this would be done by editing the slider class so that if errors are added, it can add a random value to the value presented to the user. This means that only this module needs to be changed and I do not need to change other modules like the display of waves.

## Usability Limitations

### *Undo button*

An undo button would undo the previous change to any slider or piece of equipment.

This means that changes would have to be tracked every time something is changed. This could be done by storing the previous value in all classes that hold a value as an attribute before it is changed. For example, when a slider is clicked and self.moving is set to True, the previous value attribute is set to whatever the value held is at that moment.

This would allow each individual object to revert to what it was before a change but when the undo button is clicked, it needs to know which object it was that was used last otherwise all values would be switched back. This could be done by the changing procedure that allows values to be changed in objects to keep a variable that holds the last object that was moved.

# Maintenance

## Corrective Maintenance

The code is well commented: each sub-routine has a comment at the beginning saying what the sub-routine is designed to do, any code that does not have an obvious function is also commented.

This allows the code to have corrective maintenance so modules can be corrected easily without having top figure out what certain lines of code do.

The code is modular and most sub-routines only perform one function. The only sub-routine where this is not as good is the editValue method within the Sliders class which could be split into multiple methods: one that checks if the box has been clicked and one that allows a number to be input.

The code being modular means that corrective maintenance is easier because one sub-routine can be changed without worrying if it affects another sub-routine.

## Adaptive maintenance

This is how well the software will adapt to modern hardware.

Again, the code is modular and well commented as well as having good variable names so that it can be easily advanced for future machines.

As computers in the future will speed up, this does not mean that the waves in my simulator will also speed up as long as the in-built python library time is kept up top date. This is because my simulator uses the time to find where the graph should be at each point rather than the number of the frame.

## Perfective Maintenance

There is space for my program to become more efficient because I have not made the most of concurrency in my project. I could have used concurrency to get the program to check if sliders are adjusted at the same time that the waves get displayed.

# 5. Bibliography

AQA. (n.d.). *A-Level Physics Specification*. Retrieved September 19, 2024, from AQA: https://www.aqa.org.uk/subjects/science/as-and-a-level/physics-7407-7408/specification-at-a-glance

Geeks for Geeks. (2021, March 26). *How to create a text input box with Pygame?* Retrieved January 12, 2025, from https://www.geeksforgeeks.org/how-to-create-a-text-input-box-with-pygame/

Geeks For Geeks. (2022, November 17). *Python - Moving an object in PyGame*. Retrieved January 23, 2025, from Geeks For Geeks: https://www.geeksforgeeks.org/python-moving-an-object-in-pygame/

Gilbert Bett, T. O. (n.d.). *GeoGebra Standing Waves*. Retrieved September 19, 2024, from GeoGebra: https://www.geogebra.org/m/dwjwhv8z

PhET Colorado. (2006). *Look and Feel*. Retrieved October 6, 2024, from https://phet.colorado.edu/publications/PhET%20Look%20and%20Feel.pdf

PhET Colorado. (2025). *Generator Simulator*. Retrieved February 18, 2025, from PhET Simulations: https://phet.colorado.edu/sims/html/generator/latest/generator_all.html

PhET Colorado. (n.d.). *PhET Simulation Design Prcoess*. Retrieved October 6, 2024, from https://phet.colorado.edu/publications/phet_design_process.pdf

PhET. (n.d.). *PhET Interactive Simulations*. Retrieved April 2, 2025, from Circuit Construction Kit: https://phet.colorado.edu/en/simulations/wave-interference

PhET. (n.d.). *Wave Interference*. Retrieved April 2, 2025, from PhET Interactive Simulations: https://phet.colorado.edu/en/simulations/wave-interference

Physics & Maths Tutor. (2024). *Practical Skills*. Retrieved August 16, 2024, from https://www.physicsandmathstutor.com/physics-revision/a-level-aqa/practical-skills/

Physics and Maths Tutor. (2024). *AQA Physics A-Level Required Practical 1.*

*Physics and Maths Tutor*. (2025). Retrieved February 23, 2025, from AQA Physics A-Level 6.1 Further mechanics Notes: https://pmt.physicsandmathstutor.com/download/Physics/A-level/Notes/AQA/06-Further-Mechanics/Detailed%20Notes%20-%20Section%2006%20Further%20Mechanics%20-%20AQA%20Physics%20A-level.pdf

PyGame. (2025). *pygame.Rect documentation*. Retrieved January 13, 2025, from PyGame documentation: https://www.pygame.org/docs/ref/rect.html

*SimPhy Micrometer Simulator*. (n.d.). Retrieved September 19, 2024, from SimPhy: https://simphy.com/view_simulation?micrometer-simulator&PHPSESSID

teachyourselfpython.com. (2025). Mastering Python and Pygame. Retrieved January 6, 2025

University of Colorado. (2024). Retrieved August 24, 2024, from PhET Colorado: https://phet.colorado.edu/

W3 Schools. (n.d.). *Python String isnumeric() Method*. Retrieved January 25, 2025, from W3 Schools: https://www.w3schools.com/python/ref_string_isnumeric.asp

# 6. Appendices

Explanation of method of all Required Practical Activities - Practical Skills Revision - AQA Physics A-level - PMT (physicsandmathstutor.com)

## Appendix A - PyGame Methods

This is a summary of some PyGame in-built methods and what they do. This is not a list of all of the methods but just the ones where it is not immediately obvious as to what they do. Information has been fetched from the PyGame documentation (PyGame, 2025)

`Screen.blit(source,dest) –`

Draws a source Surface onto this Surface. The draw can be positioned with the dest argument. The dest argument can either be a pair of coordinates representing the position of the upper left corner of the blit or a Rect, where the upper left corner of the rectangle will be used as the position for the blit. The size of the destination rectangle does not affect the blit.

`Pygame.Rect(left,top,width,height):`

Instantiates a rectangle object with coordinates (left,top) and dimensions (width,height)

`Pygame.font.SysFont(name,size,bold=False)`

Return a new Font object that is loaded from the system fonts.

`Render(text, antialias, colour)`

Creates a new surface with the text rendered on it. Antialias is set to True to smooth out the lettering.

## Appendix B – Glossary

- **RP** – Required Practical activity. This is one of the practical activities performed by students in the AQA A-level Physics course
- **RP 01** – This is the stationary waves required pratcial activity that my program simulates.
- **Stationary wave** – A wave formed by two progressive waves moving in opposite directions interfering and superposing to create one stationary wave.

## Appendix C - Code Listing

### Waves.py

This contains the code for the display of the waves.

```
import time, pygame
import numpy as np #needed for maths calculations and manipulation of
arrays

BLACK=(0,0,0)
WHITE=(255,255,255)
RED=(255,0,0)
GREEN =(0,255,0)
BLUE=(0,0,255)
YELLOW  = (255,255,0)

def calcFreq(T,mu,l):
    # calculates the frequency of the first harmonic

    if not isinstance(T, (int,float)) or not isinstance(mu,(int,float)) or
not isinstance(l,(int,float)):
        raise ValueError("T, mu and l must be integers or floats")
        #Included to make errors more obvious if it has been caused by
passing an argument of the wromg type
    h1Freq = (1/(2*l))*np.sqrt(T/mu)
    return h1Freq

#endFunction

def calcSpeed(T,mu):
    # calculates the speed of the stationary wave
    return np.sqrt(T/mu)
#endFunction

def closestHarmonic(T: float,mu: float,l: float,f: float):
    # This function returns the number of the harmonic
    # closest to the frequency the user has set the frequency generator to
    # f = frequency input
    # T = tension in string
    # mu = mass per unit length of string
    # l = length of string
    harm1Freq = calcFreq(T,mu,l) #calculate frequency of wave
    closestHarmonic = round(f/harm1Freq)
    #divides the frequency by the frequency of the 1st harmonic and rounds
to nearest integer
    if closestHarmonic == 0:
        closestHarmonic = 1
        # A stationary wave of harmonic 0 cannot be shown so changed to 1
    #endIf
    return closestHarmonic
#endFunction


def dispReal(T: float,mu: float,l: float,f: float ,A: int ,screenSize:
tuple, surface):
    #displays the resultant stationary wave in real time

    SCREEN_LENGTH = 2 #metres  How many metres the screen width represents

    metresPerSecond = calcSpeed(T,mu)
```

```
    v = metresPerSecond * screenSize[1] * SCREEN_LENGTH# calculates speed
in pixels/second

    pixelL = (l * screenSize[0])/SCREEN_LENGTH #finds length of string in
pixels
    start = screenSize[0]/5
    end = screenSize[0]/5 + pixelL
    # graph is left adjusted


    heightRatio = screenSize[1] / 500  #multiplier so amplitude is adjusted
with screen height
    ratioL = l/SCREEN_LENGTH  #makes the string the correct length


    firstHarmonic = calcFreq(T,mu,l) #finds frequency of first harmonic
    harmonic = closestHarmonic(T,mu,l,f) # finds number of the harmonic to
be displayed
    harmFreq = harmonic* firstHarmonic #finds frequency of harmonic to be
shown

    x = np.arange(1,pixelL) # creates np array with all x values

    t = time.time() #time used to ensure no latency between frames
    b = 0.2 #dampening factor

    l2rWaveY = np.round(heightRatio*A*np.sin((harmonic*np.pi*(x)-
v*t)/(screenSize[0]*ratioL))/(np.sqrt((f**2 - harmFreq**2)**2+(b*f)**2)))
    # Calculates y values for wave moving left to right
    r2lwaveY = np.round(heightRatio*-A*np.sin((harmonic*np.pi*(-x)-
v*t)/(screenSize[0]*ratioL))/(np.sqrt((f**2 - harmFreq**2)**2+(b*f)**2)))
# np.round(-A*np.sin((harmonic*np.pi*(-x)-v*t)/screenSize[1]) +
screenSize[0]/2)
    # Calculates y values for wave moving right to left
    resultantY = l2rWaveY + r2lwaveY + screenSize[1]//2
    # Calculates y values for resultant stationary wave

    x = np.arange(start,end+1)

    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph


    pygame.draw.aalines(surface, BLACK, False, resultant)
    #draws lines on screen
#endProcedure

def dispAbtract(T: float,mu: float,l: float,harmonic: int,A:
int,screenSize: tuple,surface,multiplier: float, progressive: bool = True):
    # Displays 3 waves. One resultant sationary  wave and two progressive
waves
    # Takes harmonic as parameter because frequency has to be at a harmonic

    SCREEN_LENGTH = 2 #metres  How many metres the screen width represents

    metresPerSecond = calcSpeed(T,mu) # speed in metres per second
    v = metresPerSecond * screenSize[1] * multiplier * SCREEN_LENGTH
    #multiplied by screenSixe to get speed in pixels per second
    #multiplier is the fraction of the speed it moves
```

```
    pixelL = (l * screenSize[0])/SCREEN_LENGTH
    start = (screenSize[0]-pixelL)/2
    end = (screenSize[0]+pixelL)/2
    #centres the graph

    heightRatio = screenSize[1] / 500 #multiplier so amplitude is adjusted
with screen height
    ratioL = l/SCREEN_LENGTH #makes the string the correct length

    x = np.arange(1,pixelL) # creates np array with all x values

    t = time.time() #time used to ensure no latency between frames

    l2rWaveY = np.round(heightRatio*A*np.sin((harmonic*x*np.pi-
v*t)/(screenSize[0]*ratioL)) + screenSize[1]/2)
    # Calculates y values for wave moving left to right
    r2lwaveY = np.round(heightRatio*-A*np.sin((harmonic*np.pi*(-x)-
v*t)/(screenSize[0]*ratioL)) + screenSize[1]/2)
    # Calculates y values for wave moving right to left
    resultantY = l2rWaveY + r2lwaveY - screenSize[1]//2
    # Calculates y values for resultant stationary wave

    x = np.arange(start,end+1)

    l2rWave = list(zip(x,l2rWaveY))
    r2lWave = list(zip(x,r2lwaveY))
    resultant = list(zip(x,resultantY))
    # Creates a list of tuples for every point on graph

    if progressive: #only shows 2 progressive waves if selected by user
        pygame.draw.aalines(surface, BLUE, False, l2rWave)
        pygame.draw.aalines(surface, RED, False, r2lWave)
    #endif

    pygame.draw.aalines(surface, GREEN, False, resultant)
    #draws lines on screen
#endProcedure
```

### Sliders.py

This contains the code for the slider classes.

```
import pygame
from pygame.locals import *

pygame.font.init()

BLACK=(0,0,0)
WHITE=(255,255,255)
RED=(255,0,0)
GREEN =(0,255,0)
BLUE=(0,0,255)
YELLOW  = (255,255,0)
MAGENTA = (255,0,255)
#define colours



#creates the surface
```

```
def isNumber(stringInput):
    #checks if a string can be casted to a float
    try:
        float(stringInput)
        return True
    except ValueError:
        return False


class Slider:
    #This class allows sliders to be instatiated and moved
    def __init__(self,givenPos, givenLength, givenMinValue,givenMaxValue,
givenName = ""):
        self.POS = givenPos
        self.length = givenLength
        self.minValue = givenMinValue
        self.maxValue = givenMaxValue
        self.valueHeld = (givenMaxValue + givenMinValue)/2
        self.moving = False
        self.posPointer = list(givenPos)
        self.boxColour = BLACK
        self.pointer = pygame.Rect(self.POS[0]-5, self.POS[1]-15,11,31)
#creates a Rect object for the pointer

        self.inputBox = pygame.Rect(self.POS[0]-20,self.POS[1]-60,40,40)
        self.userText = str(self.valueHeld)
        self.active = False
        self.valueSurface = pygame.font.SysFont("calibri",28,bold =
True).render(self.userText,True, BLACK)
        # Text surface object which contains the value

        topLeftx = self.POS[0]-self.length/2

        self.minValSurface =
pygame.font.SysFont("calibri",20).render(str(self.minValue),True, BLACK)
        self.maxValSurface =
pygame.font.SysFont("calibri",20).render(str(self.maxValue),True, BLACK)


        self.lineRect = pygame.Rect(topLeftx, self.POS[1]-2,self.length, 5)

        if givenName:
            self.name = givenName
            self.nameSurface = pygame.font.SysFont("calibri",26,bold =
True).render(self.name,True, MAGENTA)
        else:
            self.nameSurface = False


    #endfunction



    def move(self,events):
        # This procedure changes the pointer location to the position
selected by the user
        for event in events:
            if event.type == MOUSEBUTTONDOWN and
self.pointer.collidepoint(event.pos):
                #checks if pointer is being clicked
```

```
                self.moving = True
            elif event.type ==  MOUSEBUTTONUP:
                #stops the pointer from being able to be moved after mouse
is raised
                self.moving = False
            elif event.type == MOUSEMOTION and self.moving:
                #if the pointer is being drageed it goes through the
following validation
                if abs(event.pos[0]-self.POS[0]) < self.length/2:
                    self.posPointer[0] = event.pos[0]
                elif event.pos[0] <= self.POS[0] -self.length/2:
                    self.posPointer[0] = self.POS[0] -self.length/2
                else:
                    self.posPointer[0] = self.POS[0] + self.length/2
                #endif
            elif event.type == MOUSEBUTTONDOWN and
self.lineRect.collidepoint(event.pos):
                #checks if a point on the line has ben clicked
                self.posPointer[0] = event.pos[0]


            #endif
    #endProcedure




    def setValueHeld(self):
        #This procedure changes the value held to the one indicated to by
the pointer
        range = self.maxValue - self.minValue
        relPosPointer = self.posPointer[0] - (self.POS[0] - self.length/2)
        unitsPerPixel = range/ self.length
        self.valueHeld = self.minValue + relPosPointer * unitsPerPixel

    def getValueHeld(self):
        return self.valueHeld

    def adjustValue(self):
        # move slider to correct position:
        range = self.maxValue - self.minValue
        pixelsPerUnit = self.length/range
        relPosPointer = pixelsPerUnit * (self.valueHeld - self.minValue)
        self.posPointer[0] = relPosPointer + self.POS[0] - self.length/2

    def editValue(self,events):
        #allows the user to type in a number that slider moves to

        for event in events:
            if event.type == pygame.MOUSEBUTTONDOWN :
                if self.inputBox.collidepoint(event.pos):
                    self.active = True
                else:
                    self.active = False

            elif self.active and event.type == pygame.KEYDOWN:
                if event.key == pygame.K_BACKSPACE:
                    self.userText = self.userText[:-1]
                    #deletes text if backspace hit
                elif event.key == pygame.K_RETURN:
```

```
                    self.active = False
                    if self.userText and (isNumber(self.userText) and
float(self.userText) <= self.maxValue and float(self.userText) >=
self.minValue):
                        #presence and range check
                        self.valueHeld = float(self.userText)
                        #updates value held
                        self.adjustValue()

                    else:
                        self.boxColour = (255,0,0)

                else:
                    self.userText += event.unicode


                #endif
            #endif

        if self.active:
            #sets colour to red if currently typing
            self.boxColour = RED
        else:
            self.boxColour = BLACK
            self.userText = str(round(self.valueHeld,5))
        #endif

        self.valueSurface =
pygame.font.Font(None,32).render(self.userText,True, self.boxColour)
        #creates a surface of with text on it size 32 font



        self.inputBox.width = max(100,self.valueSurface.get_width()+10)
        #adjusts the width of the box appropriately with the length of the
text

        self.setValueHeld()
        #changes the position of the pointer appropriately
    #endfunction









    def display(self,surface):
        #displays the slider on the screen
        startPos = (self.POS[0]-self.length/2, self.POS[1])
        endPos = (self.POS[0]+self.length/2, self.POS[1])
        pygame.draw.line(surface, BLACK,startPos, endPos, width = 3)

        self.pointer.topleft = (self.posPointer[0] - 5, self.posPointer[1]
- 15)

        pygame.draw.rect(surface, MAGENTA, self.pointer)
        pygame.draw.rect(surface, self.boxColour, self.inputBox,width = 3 )
```

```
        surface.blit(self.valueSurface, (self.inputBox.x+5,
self.inputBox.y+5))
        if self.nameSurface:
            surface.blit(self.nameSurface,(self.inputBox.x-self.length/2
+10, self.inputBox.y-30))

        #self.move()

        surface.blit(self.minValSurface,(self.POS[0]-self.length/2,
self.POS[1] - 30))

        surface.blit(self.maxValSurface,(self.POS[0]+self.length/2 -
self.maxValSurface.get_width(), self.POS[1] - 30))









        self.pointer.topleft = (self.posPointer[0] - 5, self.posPointer[1]
- 15)

pygame.draw.line(surface,WHITE,(self.posPointer[0],self.posPointer[1]-
13),(self.posPointer[0],self.posPointer[1]+13) )




    #endprocedure
#endclass


class discSlider(Slider):

    def setValueHeld(self):
        #changes the value held to the one indicated by the slider
        super().setValueHeld()
        self.valueHeld = int(round(self.valueHeld))
        #round the value to nearest integer as its discrete
        super().adjustValue()
    #endProcedure

    def display(self,surface):
        #displays the slider on the screen
        super().display(surface)

        farLeft = self.POS[0]-self.length/2
        bottom = self.POS[1]-10
        top = self.POS[1] +10
        spacing = self.length /(self.maxValue-self.minValue)
        for i in range(self.maxValue):
            xPos = farLeft + i * spacing
            pygame.draw.line(surface, BLACK, (xPos,bottom),(xPos,top),
width= 2)
    #endProcedure
#endClass
```

## Masses.py

This contains the code for the masses and the mass hook.

```
import pygame # used as GUI
from pygame.locals import * # access to Mouse motions


BLACK=(0,0,0)
WHITE=(255,255,255)
RED=(255,0,0)
GREEN =(0,255,0)
BLUE=(0,0,255)
YELLOW  = (255,0,255)


class MassHook:
    def __init__(self, givenPos:tuple, givenAutoSetup:bool = False):
        self.pos = givenPos #position of centre of hook mass
        self.numOfMasses = 0
        self.autoSetup = givenAutoSetup
        self.MAX_PLATES = 8
        self.moving = False
        hookImageBig = pygame.image.load("Mass Hook Image.png")
        self.resizeFactor = 2
        self.hookImage =
pygame.transform.scale(hookImageBig,(hookImageBig.get_width()/self.resizeFa
ctor,hookImageBig.get_height()/self.resizeFactor))
        self.hookImageRect = pygame.Rect(self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-
self.hookImage.get_height()/2,self.hookImage.get_width(),
self.hookImage.get_height())
        self.topLeftPos = list((self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-self.hookImage.get_height()/2))
#position of top left


        width = 27 #values from microsoft paint giving the width and height
of the hook
        height = 186

        self.relStackTopx = width/self.resizeFactor
        self.relStackTopy = height/self.resizeFactor

        self.stackBase = ((self.topLeftPos[0] + self.relStackTopx,
self.topLeftPos[1] +self.relStackTopy))
        self.maxItems = 1
    #endProcedure

    def move(self,events):
        # This procedure changes the mass Hook location to the position
selected by the user
        if not self.autoSetup:
            SCREEN_SIZE = (1200,800)
            for event in events:
                if event.type == MOUSEBUTTONDOWN and event.button == 1 and
self.hookImageRect.collidepoint(event.pos):
                    self.moving = True
                elif event.type ==  MOUSEBUTTONUP:
                    self.moving = False
                elif event.type == MOUSEMOTION and self.moving:
```

```
                    mousePos = pygame.mouse.get_pos()
                    if mousePos[0] > 0 and mousePos[1] > 0 and mousePos[1]
< SCREEN_SIZE[1] and mousePos[0] < SCREEN_SIZE[0]:
                        #validation to check that it is still within the
window
                        self.pos = mousePos
                    #endif
                elif event.type == MOUSEBUTTONDOWN and event.button == 3
and  self.hookImageRect.collidepoint(event.pos):
                    self.removeMass()
                #endif
        #endif
    #endProcedure


    def addMass(self):
        #adds masses to top of stack

        if self.numOfMasses < self.MAX_PLATES:
            #checks that stack is not full
            self.numOfMasses += 1

        #endif
    #endProcedure

    def removeMass(self):
        #removes a mass from the top of the stack
        if self.numOfMasses > 0 :
            #checks if stack is empty
            self.numOfMasses -= 1

        #endif
    #endFunction

    def display(self,screen):
        #displays hook on the screen

        self.topLeftPos = (self.pos[0]-
self.hookImage.get_width()/2,self.pos[1]-self.hookImage.get_height()/2)
        #position of top lefft of hook
        screen.blit(self.hookImage,self.topLeftPos) #puts hook on screen
        self.hookImageRect.topleft = self.topLeftPos #chages op left of
Rect object

        if self.numOfMasses > 0 :
            self.stackBase = ((self.topLeftPos[0] + self.relStackTopx,
self.topLeftPos[1] +self.relStackTopy))
            #resests pos of stackbase if it has been moved
            RECT_HEIGHT = 10
            RECT_WIDTH = 65 #width from ms paint
            for i in range(1,self.numOfMasses+1):
                #iterates through the masses added, drawing each mass in
turn
                topLeft = (self.stackBase[0],self.stackBase[1] - i *11)
                mass =
pygame.Rect(topLeft[0],topLeft[1],RECT_WIDTH/self.resizeFactor,
RECT_HEIGHT)
                pygame.draw.rect(screen, BLUE, mass)
        #endif
```

```
    #endFunction


    def getTension(self):
        mass = 0.1 *(self.numOfMasses+1) #1 must be added because it must
start with 100g
        return 9.81* mass
    #endfunction
#endClass


class Mass:
    def __init__(self,givenPos):
        self.pos = givenPos
        self.moving = False
        self.show = True
        RECT_HEIGHT = 10
        RECT_WIDTH = 65/2
        topLeftX = self.pos[0] - RECT_WIDTH/2
        topLeftY = self.pos[1] - RECT_HEIGHT/2
        self.massRect  =
pygame.Rect(topLeftX,topLeftY,RECT_WIDTH,RECT_HEIGHT)
        self.maxItems = 24


    def move(self,events):
        # Accepts user input and moves the hook to the correct position
        SCREEN_SIZE = (1200,800)

        for event in events:
            if event.type == MOUSEBUTTONDOWN and event.button == 1 and
self.massRect.collidepoint(event.pos):
                self.moving = True
            elif event.type ==  MOUSEBUTTONUP:
                self.moving = False
            elif event.type == MOUSEMOTION and self.moving:
                mousePos = pygame.mouse.get_pos()
                if mousePos[0] > 0 and mousePos[1] > 0 and mousePos[1] <
SCREEN_SIZE[1] and mousePos[0] < SCREEN_SIZE[0]:
                    #validation to check that it is still on
                    self.pos = mousePos
                #endif
            #endif
        #endFor
    #endProcedure


    def checkCorrectPos(self,hook):
        #checks if placed over hook and adds to hook if so
        if hook.hookImageRect.collidepoint(self.pos)  and hook.numOfMasses
< hook.MAX_PLATES:
            hook.addMass()
            self.show = False
            self.pos=(-20,-20)
         #endif
```

```
    def display(self,screen,hook):
        #displays the mass
        if self.show:
            self.massRect.center = self.pos
            pygame.draw.rect(screen,RED, self.massRect)
            self.checkCorrectPos(hook)

            textSurface = pygame.font.Font(None,15).render("100g",True,
WHITE)
            screen.blit(textSurface,self.massRect)
            #writes 100g on the mass
        else:
            pass
    #endFunction

    def getPos(self):
        return self.pos
    #endFunction
#endClass



class DragBox:
    def __init__(self, givenPos, itemType):
        self.pos = givenPos # centre of box
        self.itemType = itemType #the class of the item to be held in the
box
        NewItem = itemType(givenPos)
        self.items = [] # a list of all active items
        self.items.append(NewItem)
        if itemType == MassHook:
            #mass hook is a larger item
            self.height = 150
        else:
            self.height = 50
        self.width = 100
        self.maxItems = NewItem.maxItems # gets the max items from the
class

        self.box = pygame.Rect(givenPos[0],givenPos[1], self.width,
self.height)
        self.box.center = (givenPos[0],givenPos[1])

    def checkMore(self):
        #checks if more items need to be added and returns a list of al the
masses
        if self.items[-1].pos != self.pos and  len(self.items) <
self.maxItems:
            self.items.append(self.itemType(self.pos))
        #endif

        return self.items
    #endfunction

#endClass
```

```
class EquipmentBox:

    def __init__(self,givenPos):
        self.width = 100
        self.height = 200
        self.pos = givenPos
        self.massesBox =
DragBox((self.pos[0],self.pos[1]+self.height/4),Mass)
        self.massHookBox = DragBox((self.pos[0],self.pos[1]-self.height/4),
MassHook)
        self.boxes = [self.massHookBox,self.massesBox] # a list of all
items in box

    #endProcedure

    def display(self,screen):
        #displays each of the smaller boxes
        for box in self.boxes:
            items = box.checkMore()
            #gets a list of all the items
            if box.itemType == MassHook:
                hook = box.items[0]
                hook.display(screen)

            elif box.itemType == Mass:
                #mass needs an extra parameter
                for item in items:
                   item.display(screen,hook)
            else:
                for item in items:
                    item.display(screen)
            #endif

            pygame.draw.rect(screen, BLACK, box.box, width = 3 )
            #draws each item to the screen

        #endfor
    #endProcedure


    def getItems(self,itemType):
        #returns a list of the masses so that they can be moved
        return itemType.checkMore()

        #pygame.draw.rect(screen, BLACK, self.rectangle, width = 3 )
    #endFunction

    def getHook(self):
        #returns the hook
        return self.massHookBox.checkMore()[0]
    #endFunction

    def checkInput(self,events):
        #allows each of the objects in the box to be moved
        for box in self.boxes:
            for item in box.checkMore():
                item.move(events)
    #endFunction

#endClass
```

## Menu.py

This contains the code for the main menu.

```
import pygame
from pygame.locals import *

BLACK=(0,0,0)
WHITE=(255,255,255)
RED=(255,0,0)
GREEN =(0,255,0)
BLUE=(0,0,255)
YELLOW  = (255,255,0)
MAGENTA = (255,0,255)
#define colours

def checkClicked(rectObj,event):
    #returns true if rect object is clicked
    if event.type == MOUSEBUTTONDOWN and rectObj.collidepoint(event.pos):
        return True
    else:
        return False
    #endif
 #endFunction



def createMenu(SCREEN_SIZE):
    #displays the main menu with 2 options
    leftCentre = (SCREEN_SIZE[0]/4,SCREEN_SIZE[1]/2)
    rightCentre = (3*SCREEN_SIZE[0]/4,SCREEN_SIZE[1]/2)
    Left = MenuOption(leftCentre,1)
    Right = MenuOption(rightCentre,2)
    return Left,Right
#endFunction


class MenuOption:
    #one of the options on the main menu
    def __init__(self,pos,ID):
        #id is 1 if realistic view is shown
        #id is 2 if abstract view is shown
        height = 300
        width = 400

        topLeft = (pos[0]-width/2,pos[1]-height/2)

        self.box = pygame.Rect(topLeft[0],topLeft[1], width, height)

        borderSize = 5
        self.biggerBox = pygame.Rect(topLeft[0]-borderSize,topLeft[1]-
borderSize, width +borderSize*2, height + borderSize*2)
        #This is the bigger box that will have a border around the one
withg the image displayed

        if ID == 1:
            imageBig = pygame.image.load("Display Real Example.png")
```

```
            self.textSurface =
pygame.font.SysFont("calibri",20).render("RP 01 Stationary Waves -
Realistic View",True, BLACK)
        elif ID == 2:
            imageBig = pygame.image.load("Display Abstract Example.png")
            self.textSurface =
pygame.font.SysFont("calibri",20).render("RP 01 Stationary Waves -
Abstracted View",True, BLACK)


        self.image = pygame.transform.scale(imageBig,(width,height))
    #endProcedure


    def checkOptionClicked(self,event):
        #returns true if box is clicked
        return checkClicked(self.box,event)
    #endFunction

    def display(self,screen):
        #draws the menu to the screen
        pygame.draw.rect(screen,BLACK,self.biggerBox)
        screen.blit(self.image, self.box)
        #puts the image on the screen in the box

screen.blit(self.textSurface,(self.biggerBox.bottomleft[0],self.biggerBox.b
ottomleft[1]+20))
        #writes the text below the image
    #endProcedure
#endClass


def dispMenu(screen, leftOption, rightOption, SCREEN_SIZE):
    #displays the main menu on the screen
    leftOption.display(screen)
    rightOption.display(screen)
    textSurface =  pygame.font.SysFont("calibri",50,True).render("Physics
Practical Simulator",True, BLUE)
    screen.blit(textSurface,(SCREEN_SIZE[0]/2 -
textSurface.get_width()/2,20) )
#endProcedure


class HomeIcon:
    def __init__(self,pos):
        bigImage = pygame.image.load("Home Icon.png")
        width = 60
        height = 60
        self.image = pygame.transform.scale(bigImage,(width,height))
        self.box = pygame.Rect(pos[0]-width/2,pos[1]-height/2, width,
height)
    #endProcedure


    def checkIconClicked(self,events):
        #checks if the icon is clicked
        for event in events:
            if checkClicked(self.box,event):
                return True
        return False
```

```
    #endFunction

    def display(self,screen):
        #draws the icon on the screen
        screen.blit(self.image, self.box.topleft)
    #endProcedure
#endClass


def checkOptions(events,leftOption,rightOption):
    #returns 1 if option 1 is clicked and 2 if option 2 is clicked
    for event in events:
        if leftOption.checkOptionClicked(event):
            return 1
        elif rightOption.checkOptionClicked(event):
            return 2

        #end if
    #next event
#endFunction

def checkEnded(events):
    #returns True if program has been quit
    for event in events:
        if event.type == pygame.QUIT:
            #checks if window is closed

            return True
        #end if

    return False

    #next event
#endFunction




def menu(screen,SCREEN_SIZE):
    #displays the main menu and returns 1 if option 1 is clicked and 2 if 2
is clicked
    running = True

    leftOption, rightOption = createMenu(SCREEN_SIZE)

    clock = pygame.time.Clock()

    while running:
        events = pygame.event.get()

        screen.fill(WHITE)


        clicked = checkOptions(events,leftOption,rightOption)

        if checkEnded(events):
            #checks if program closed
            return False
        elif clicked:
            #checks if an option has been clicked
```

```
            return clicked
        else:
            dispMenu(screen,leftOption,rightOption,SCREEN_SIZE)
        #endif
        clock.tick(60)

        pygame.display.flip()

    #endWile
#endFunction
```

## RP 01.py

This is the main code that is run. It includes the code that displays each of the simulations.

```
import pygame, Sliders, Menu, Masses, Waves
from pygame.locals import * #needed to get mouse controls

BLUE = (0,0,255)
WHITE = (255,255,255)
BLACK = (0,0,0)

def displayFreq(surface,harmonic,l,T,mu):
    #displays the frequency for RP 01 abstract view
    frequency = round(Waves.calcFreq(T,mu,l) * harmonic,2)
    #gets frequency rounded to 2dp
    pos = (300,150) #position of top left of box

    frequencyBox = pygame.Rect(pos[0],pos[1],100,40)
    #the box around the value
    frequencySurface =
pygame.font.SysFont("calibri",20).render(str(frequency),True,BLACK)
    #a text surface holding the value
    frequencyBox.width = max(100,frequencySurface.get_width()+10)
    surface.blit(frequencySurface,(frequencyBox.x+5,frequencyBox.y+5))
    #puts the text on the screen
    pygame.draw.rect(surface,BLACK,frequencyBox,width = 3)
    textSurface = pygame.font.SysFont("calibri",20).render("Frequency (Hz)
= ",True, BLACK)
    # Heading before the box
    surface.blit(textSurface,(frequencyBox.x-textSurface.get_width(),
frequencyBox.y+5))
#endProcedure


def createRP1Real(SCREEN_SIZE):
    #instantiates and returns sliders, mass hook, ebox and title text

    f = Sliders.Slider((SCREEN_SIZE[0]/2,700),300,0,100,"Frequency (Hz)")
    mu = Sliders.Slider((SCREEN_SIZE[0]-160,700),300,0.0001,0.001, "Mass
per unit Length (kg/m)")
    l = Sliders.Slider((160,700),300,0.5,1.5,"Length (m)")

    eBox = Masses.EquipmentBox((SCREEN_SIZE[0]-100,150))

    homeIcon = Menu.HomeIcon((80,80))

    text = pygame.font.SysFont("calibri",50,bold = True)
    textSurface = text.render("RP 01 - Realistic View", True, BLUE)

    return [mu,l,f], eBox, textSurface, homeIcon
#endFunction


def displayRP1Real(screen, SCREEN_SIZE,sliders, eBox, textSurface,
homeIcon):
    #displays the Rp 01 Screen
    for slider in sliders:
        slider.display(screen)

    eBox.display(screen)
    homeIcon.display(screen)
```

```
    T = eBox.getHook().getTension()

    mu = sliders[0].valueHeld
    l = sliders[1].valueHeld
    f = sliders[2].valueHeld

    Waves.dispReal(T,mu,l,f,100,SCREEN_SIZE, screen)

    screen.blit(textSurface, (SCREEN_SIZE[0]/2-textSurface.get_width()/2,
50))
#endProcedure


def createRP1Abstract(SCREEN_SIZE):
    #instantiates and returns sliders, mass hook, ebox and title text

    multiplier = Sliders.Slider((SCREEN_SIZE[0]/2,700),300,0.0001,0.1,
"Speed Multiplier")
    harmonic = Sliders.discSlider((600,200),300,1,5, "Harmonic Number")
    mu = Sliders.Slider((SCREEN_SIZE[0]-160,700),300,0.0001,0.001, "Mass
per unit Length (kg/m)")
    l = Sliders.Slider((160,700),300,0.5,2,"Length (m)")

    eBox = Masses.EquipmentBox((SCREEN_SIZE[0]-100,150))

    homeIcon = Menu.HomeIcon((80,80))

    text = pygame.font.SysFont("calibri",50,bold = True)
    textSurface = text.render("RP 01 - Abstract View", True, BLUE)

    return [mu,l,harmonic,multiplier], eBox,textSurface, homeIcon
#endFunction

def displayRP1Abstract(screen,SCREEN_SIZE, sliders, eBox, textSurface,
homeIcon):
    #displays the Rp 01 Screen
    for slider in sliders:
        slider.display(screen)
    eBox.display(screen)
    homeIcon.display(screen)

    T = eBox.getHook().getTension()

    mu = sliders[0].valueHeld
    l = sliders[1].valueHeld
    harmonic = sliders[2].valueHeld
    multiplier = sliders[3].valueHeld

    displayFreq(screen,harmonic, l, T, mu)

    Waves.dispAbtract(T,mu,l,harmonic,50,SCREEN_SIZE,screen, multiplier)
    screen.blit(textSurface, (SCREEN_SIZE[0]/2-textSurface.get_width()/2,
50))
#endProcedure


def rp1(screen,SCREEN_SIZE, view):
    #the main code that is run when RP 1 realistic is displayed
    #returns False if quit and true if menu option clicked
```

```
    running = True

    if view == 1:
        sliders, eBox, textSurface, homeIcon = createRP1Real(SCREEN_SIZE)
    elif view == 2:
        sliders, eBox, textSurface, homeIcon =
createRP1Abstract(SCREEN_SIZE)
    #endif


    clock = pygame.time.Clock()

    while running:
        screen.fill(WHITE)

        events = pygame.event.get()
        #gets mouse events

        if Menu.checkEnded(events):
            running = False
            return False
        elif homeIcon.checkIconClicked(events):
            return True
        else:
            for slider in sliders:
                slider.move(events)
                slider.editValue(events)
            eBox.checkInput(events) #allows masses to be moved

        #endif

        if view == 1:
            displayRP1Real(screen, SCREEN_SIZE,sliders,eBox, textSurface,
homeIcon)
        else:
            displayRP1Abstract(screen, SCREEN_SIZE, sliders, eBox,
textSurface, homeIcon)
        #endif


        clock.tick(60)
        pygame.display.flip()
    #endWhile
#endFunction



if __name__ == "__main__":
    pygame.init()
    SCREEN_SIZE = (1200,800)
    screen = pygame.display.set_mode(SCREEN_SIZE)

    option = True
    while option:
        option = Menu.menu(screen,SCREEN_SIZE) # returns 1 if option 1
clicked and 2 if option 2 clicked
        if option:
            option = rp1(screen,SCREEN_SIZE,option) # returns False if quit
and True if menu icon clicked
```

```
    #endWhile

    pygame.quit() #closes pygame tab
```